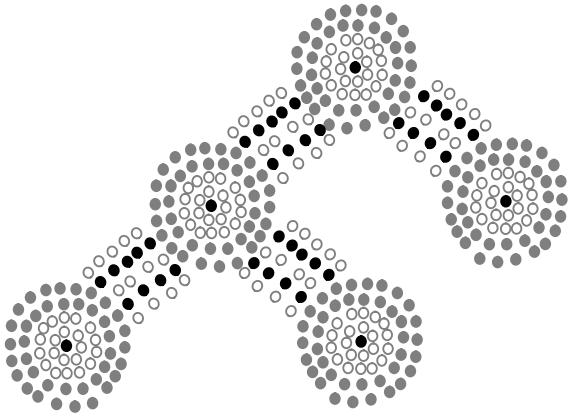
# Chapter 8 Trees



## Contents


**8.1 General Trees . . . . . 300**
8.1.1 Tree Definitions and Properties . . . . . 301
8.1.2 The Tree Abstract Data Type . . . . . 305
8.1.3 Computing Depth and Height . . . . . 308
**8.2 Binary Trees . . . . . 311**
8.2.1 The Binary Tree Abstract Data Type . . . . . 313
8.2.2 Properties of Binary Trees . . . . . 315
**8.3 Implementing Trees . . . . . 317**
8.3.1 Linked Structure for Binary Trees . . . . . 317
8.3.2 Array-Based Representation of a Binary Tree . . . . . 325
8.3.3 Linked Structure for General Trees . . . . . 327
**8.4 Tree Traversal Algorithms . . . . . 328**
8.4.1 Preorder and Postorder Traversals of General Trees . . . . 328
8.4.2 Breadth-First Tree Traversal . . . . . 330
8.4.3 Inorder Traversal of a Binary Tree . . . . . 331
8.4.4 Implementing Tree Traversals in Python . . . . . 333
8.4.5 Applications of Tree Traversals . . . . . 337
8.4.6 Euler Tours and the Template Method Pattern ★ . . . . . 341
**8.5 Case Study: An Expression Tree . . . . . 348**
**8.6 Exercises . . . . . 352**

# 8.1 General Trees

Productivity experts say that breakthroughs come by thinking "nonlinearly." In this chapter, we discuss one of the most important nonlinear data structures in computing—***trees***. Tree structures are indeed a breakthrough in data organization, for they allow us to implement a host of algorithms much faster than when using linear data structures, such as array-based lists or linked lists. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, Web sites, and other computer systems.

It is not always clear what productivity experts mean by "nonlinear" thinking, but when we say that trees are "nonlinear," we are referring to an organizational relationship that is richer than the simple "before" and "after" relationships between objects in sequences. The relationships in a tree are ***hierarchical***, with some objects being "above" and some "below" others. Actually, the main terminology for tree data structures comes from family trees, with the terms "parent," "child," "ancestor," and "descendant" being the most common words used to describe relationships. We show an example of a family tree in Figure 8.1.

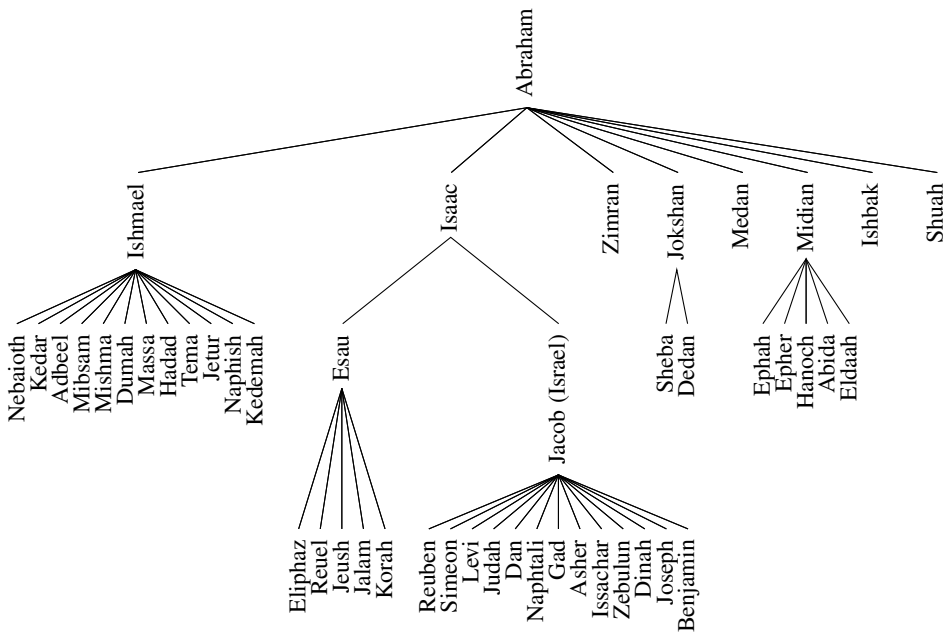



**Figure 8.1:** A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.

### 8.1.1 Tree Definitions and Properties

A ***tree*** is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a ***parent*** element and zero or more ***children*** elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. (See Figure 8.2.) We typically call the top element the ***root*** of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).

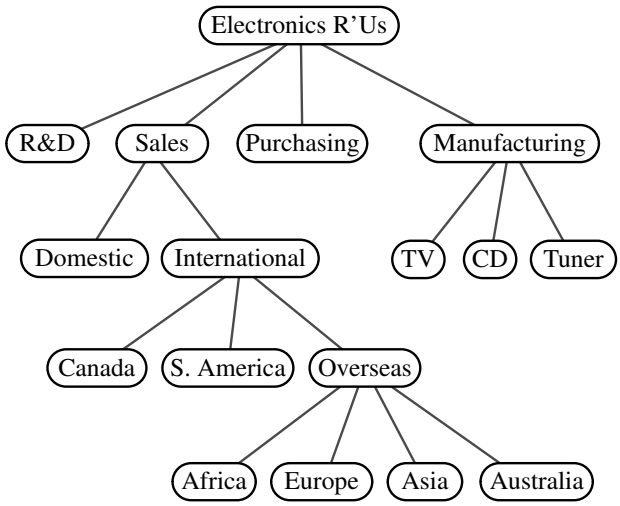



**Figure 8.2:** A tree with 17 nodes representing the organization of a fictitious corporation. The root stores *Electronics R'Us*. The children of the root store *R&D*, *Sales*, *Purchasing*, and *Manufacturing*. The internal nodes store *Sales*, *International*, *Overseas*, *Electronics R'Us*, and *Manufacturing*.

#### Formal Tree Definition

Formally, we define a ***tree*** $T$ as a set of ***nodes*** storing elements such that the nodes have a ***parent-child*** relationship that satisfies the following properties:

- If $T$ is nonempty, it has a special node, called the ***root*** of $T$, that has no parent.
- Each node $v$ of $T$ different from the root has a unique ***parent*** node $w$; every node with parent $w$ is a ***child*** of $w$.

Note that according to our definition, a tree can be empty, meaning that it does not have any nodes. This convention also allows us to define a tree recursively such that a tree $T$ is either empty or consists of a node $r$, called the root of $T$, and a (possibly empty) set of subtrees whose roots are the children of $r$.

## Other Node Relationships

Two nodes that are children of the same parent are ***siblings***. A node $v$ is ***external*** if $v$ has no children. A node $v$ is ***internal*** if it has one or more children. External nodes are also known as ***leaves***.

**Example 8.1:** *In Section 4.1.4, we discussed the hierarchical relationship between files and directories in a computer's file system, although at the time we did not emphasize the nomenclature of a file system as a tree. In Figure 8.3, we revisit an earlier example. We see that the internal nodes of the tree are associated with directories and the leaves are associated with regular files. In the UNIX and Linux operating systems, the root of the tree is appropriately called the "root directory," and is represented by the symbol "/."*

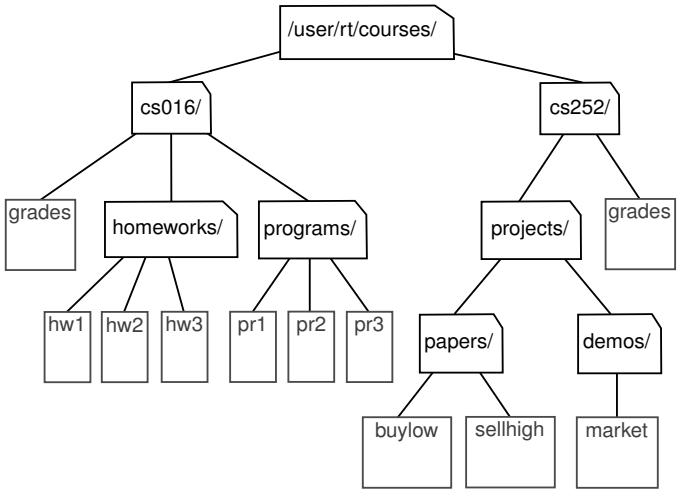



**Figure 8.3:** Tree representing a portion of a file system.

A node $u$ is an ***ancestor*** of a node $v$ if $u = v$ or $u$ is an ancestor of the parent of $v$. Conversely, we say that a node $v$ is a ***descendant*** of a node $u$ if $u$ is an ancestor of $v$. For example, in Figure 8.3, cs252/ is an ancestor of papers/, and pr3 is a descendant of cs016/. The ***subtree*** of $T$ ***rooted*** at a node $v$ is the tree consisting of all the descendants of $v$ in $T$ (including $v$ itself). In Figure 8.3, the subtree rooted at cs016/ consists of the nodes cs016/, grades, homeworks/, programs/, hw1, hw2, hw3, pr1, pr2, and pr3.

## Edges and Paths in Trees

An ***edge*** of tree $T$ is a pair of nodes $(u,v)$ such that $u$ is the parent of $v$, or vice versa. A ***path*** of $T$ is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example, the tree in Figure 8.3 contains the path (cs252/, projects/, demos/, market).

**Example 8.2:** *The inheritance relation between classes in a Python program forms a tree when single inheritance is used. For example, in Section 2.4 we provided a summary of the hierarchy for Python's exception types, as portrayed in Figure 8.4 (originally Figure 2.5). The BaseException class is the root of that hierarchy, while all user-defined exception classes should conventionally be declared as descendants of the more specific Exception class. (See, for example, the Empty class we introduced in Code Fragment 6.1 of Chapter 6.)*

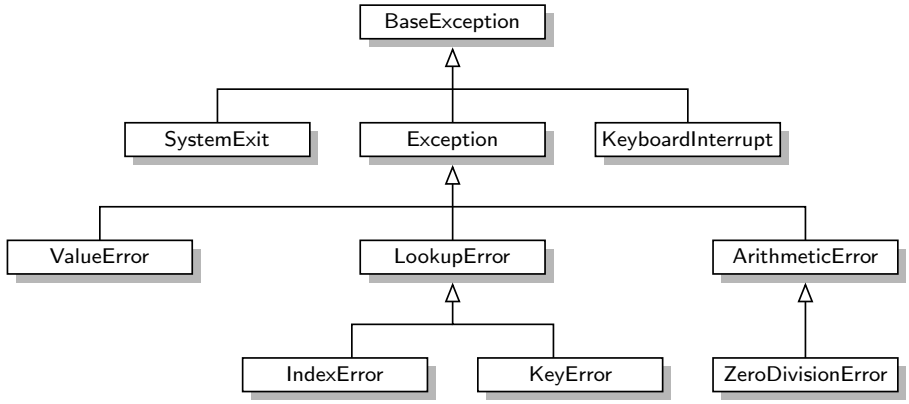



**Figure 8.4:** A portion of Python's hierarchy of exception types.

In Python, all classes are organized into a single hierarchy, as there exists a built-in class named object as the ultimate base class. It is a direct or indirect base class of all other types in Python (even if not declared as such when defining a new class). Therefore, the hierarchy pictured in Figure 8.4 is only a portion of Python's complete class hierarchy.

As a preview of the remainder of this chapter, Figure 8.5 portrays our own hierarchy of classes for representing various forms of a tree.

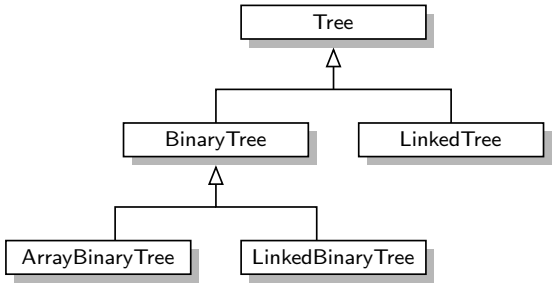



**Figure 8.5:** Our own inheritance hierarchy for modeling various abstractions and implementations of tree data structures. In the remainder of this chapter, we provide implementations of Tree, BinaryTree, and LinkedBinaryTree classes, and high-level sketches for how LinkedTree and ArrayBinaryTree might be designed.

## Ordered Trees

A tree is ***ordered*** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

**Example 8.3:** *The components of a structured document, such as a book, are hierarchically organized as a tree whose internal nodes are parts, chapters, and sections, and whose leaves are paragraphs, tables, figures, and so on. (See Figure 8.6.) The root of the tree corresponds to the book itself. We could, in fact, consider expanding the tree further to show paragraphs consisting of sentences, sentences consisting of words, and words consisting of characters. Such a tree is an example of an ordered tree, because there is a well-defined order among the children of each node.*

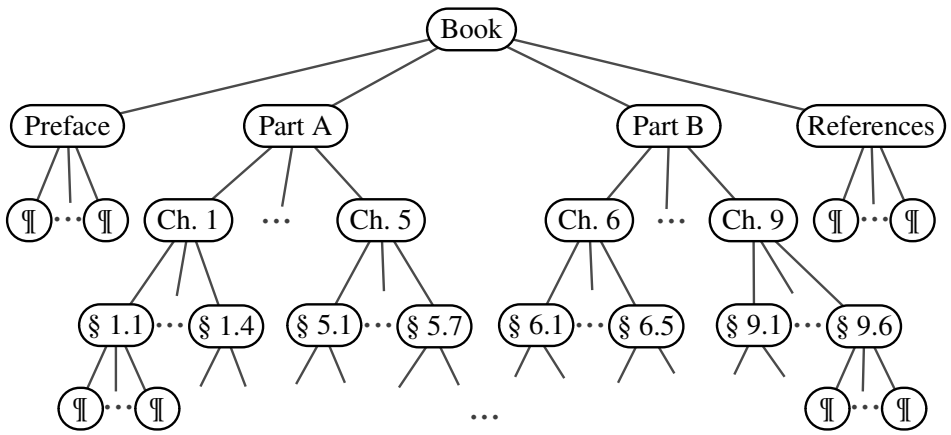



**Figure 8.6:** An ordered tree associated with a book.

Let's look back at the other examples of trees that we have described thus far, and consider whether the order of children is significant. A family tree that describes generational relationships, as in Figure 8.1, is often modeled as an ordered tree, with siblings ordered according to their birth.

In contrast, an organizational chart for a company, as in Figure 8.2, is typically considered an unordered tree. Likewise, when using a tree to describe an inheritance hierarchy, as in Figure 8.4, there is no particular significance to the order among the subclasses of a parent class. Finally, we consider the use of a tree in modeling a computer's file system, as in Figure 8.3. Although an operating system often displays entries of a directory in a particular order (e.g., alphabetical, chronological), such an order is not typically inherent to the file system's representation.

### 8.1.2 The Tree Abstract Data Type

As we did with positional lists in Section 7.4, we define a tree ADT using the concept of a ***position*** as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

**p.element( ):** Return the element stored at position p.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree:

**T.root( ):** Return the position of the root of tree T, or None if T is empty.

**T.is_root(p):** Return True if position p is the root of Tree T.

**T.parent(p):** Return the position of the parent of position p, or None if p is the root of T.

**T.num_children(p):** Return the number of children of position p.

**T.children(p):** Generate an iteration of the children of position p.

**T.is_leaf(p):** Return True if position p does not have any children.

**len(T):** Return the number of positions (and hence elements) that are contained in tree T.

**T.is_empty( ):** Return True if tree T does not contain any positions.

**T.positions( ):** Generate an iteration of all *positions* of tree T.

**iter(T):** Generate an iteration of all *elements* stored within tree T.

Any of the above methods that accepts a position as an argument should generate a ValueError if that position is invalid for T.

If a tree T is ordered, then T.children(p) reports the children of p in the natural order. If p is a leaf, then T.children(p) generates an empty iteration. In similar regard, if tree T is empty, then both T.positions( ) and iter(T) generate empty iterations. We will discuss general means for iterating through all positions of a tree in Sections 8.4.

We do not define any methods for creating or modifying trees at this point. We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

## A Tree Abstract Base Class in Python

In discussing the object-oriented design principle of abstraction in Section 2.1.2, we noted that a public interface for an abstract data type is often managed in Python via ***duck typing***. For example, we defined the notion of the public interface for a queue ADT in Section 6.2, and have since presented several classes that implement the queue interface (e.g., ArrayQueue in Section 6.2.2, LinkedQueue in Section 7.1.2, CircularQueue in Section 7.2.2). However, we never gave any formal definition of the queue ADT in Python; all of the concrete implementations were self-contained classes that just happen to adhere to the same public interface. A more formal mechanism to designate the relationships between different implementations of the same abstraction is through the definition of one class that serves as an ***abstract base class***, via inheritance, for one or more ***concrete classes***. (See Section 2.4.3.)

We choose to define a Tree class, in Code Fragment 8.1, that serves as an abstract base class corresponding to the tree ADT. Our reason for doing so is that there is quite a bit of useful code that we can provide, even at this level of abstraction, allowing greater code reuse in the concrete tree implementations we later define. The Tree class provides a definition of a nested Position class (which is also abstract), and declarations of many of the accessor methods included in the tree ADT.

However, our Tree class does not define any internal representation for storing a tree, and five of the methods given in that code fragment remain ***abstract*** (root, parent, num_children, children, and __len__); each of these methods raises a NotImplementedError. (A more formal approach for defining abstract base classes and abstract methods, using Python's abc module, is described in Section 2.4.3.) The subclasses are responsible for overriding abstract methods, such as children, to provide a working implementation for each behavior, based on their chosen internal representation.

Although the Tree class is an abstract base class, it includes several ***concrete*** methods with implementations that rely on calls to the abstract methods of the class. In defining the tree ADT in the previous section, we declare ten accessor methods. Five of those are the ones we left as abstract, in Code Fragment 8.1. The other five can be implemented based on the former. Code Fragment 8.2 provides concrete implementations for methods is_root, is_leaf, and is_empty. In Section 8.4, we will explore general algorithms for traversing a tree that can be used to provide concrete implementations of the positions and __iter__ methods within the Tree class. The beauty of this design is that the concrete methods defined within the Tree abstract base class will be inherited by all subclasses. This promotes greater code reuse, as there will be no need for those subclasses to reimplement such behaviors.

We note that, with the Tree class being abstract, there is no reason to create a direct instance of it, nor would such an instance be useful. The class exists to serve as a base for inheritance, and users will create instances of concrete subclasses.

```
class Tree:
  """Abstract base class representing a tree structure."""

  #------------------------------ nested Position class ------------------------------
  class Position:
    """An abstraction representing the location of a single element."""

    def element(self):
      """Return the element stored at this Position."""
      raise NotImplementedError('must be implemented by subclass')

    def __eq__(self, other):
      """Return True if other Position represents the same location."""
      raise NotImplementedError('must be implemented by subclass')

    def __ne__(self, other):
      """Return True if other does not represent the same location."""
      return not (self == other)                # opposite of __eq__

  # ---------- abstract methods that concrete subclass must support ----------
  def root(self):
    """Return Position representing the tree's root (or None if empty)."""
    raise NotImplementedError('must be implemented by subclass')

  def parent(self, p):
    """Return Position representing p's parent (or None if p is root)."""
    raise NotImplementedError('must be implemented by subclass')

  def num_children(self, p):
    """Return the number of children that Position p has."""
    raise NotImplementedError('must be implemented by subclass')

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    raise NotImplementedError('must be implemented by subclass')

  def __len__(self):
    """Return the total number of elements in the tree."""
    raise NotImplementedError('must be implemented by subclass')
```

**Code Fragment 8.1:** A portion of our Tree abstract base class (continued in Code Fragment 8.2).

```
40    # ---------- concrete methods implemented in this class ----------
41    def is_root(self, p):
42      """Return True if Position p represents the root of the tree."""
43      return self.root( ) == p
44
45    def is_leaf(self, p):
46      """Return True if Position p does not have any children."""
47      return self.num_children(p) == 0
48
49    def is_empty(self):
50      """Return True if the tree is empty."""
51      return len(self) == 0
```

**Code Fragment 8.2:** Some concrete methods of our Tree abstract base class.

### 8.1.3 Computing Depth and Height

Let $p$ be the position of a node of a tree $T$. The ***depth*** of $p$ is the number of ancestors of $p$, excluding $p$ itself. For example, in the tree of Figure 8.2, the node storing *International* has depth 2. Note that this definition implies that the depth of the root of $T$ is 0. The depth of $p$ can also be recursively defined as follows:

- If $p$ is the root, then the depth of $p$ is 0.
- Otherwise, the depth of $p$ is one plus the depth of the parent of $p$.

Based on this definition, we present a simple, recursive algorithm, depth, in Code Fragment 8.3, for computing the depth of a position $p$ in Tree $T$. This method calls itself recursively on the parent of $p$, and adds 1 to the value returned.

```
52    def depth(self, p):
53      """Return the number of levels separating Position p from the root."""
54      if self.is_root(p):
55        return 0
56      else:
57        return 1 + self.depth(self.parent(p))
```

**Code Fragment 8.3:** Method depth of the Tree class.

The running time of T.depth($p$) for position $p$ is $O(d_p + 1)$, where $d_p$ denotes the depth of $p$ in the tree $T$, because the algorithm performs a constant-time recursive step for each ancestor of $p$. Thus, algorithm T.depth($p$) runs in $O(n)$ worst-case time, where $n$ is the total number of positions of $T$, because a position of $T$ may have depth $n-1$ if all nodes form a single branch. Although such a running time is a function of the input size, it is more informative to characterize the running time in terms of the parameter $d_p$, as this parameter may be much smaller than $n$.

### Height

The ***height*** of a position $p$ in a tree $T$ is also defined recursively:

- If $p$ is a leaf, then the height of $p$ is 0.
- Otherwise, the height of $p$ is one more than the maximum of the heights of $p$'s children.

The ***height*** of a nonempty tree $T$ is the height of the root of $T$. For example, the tree of Figure 8.2 has height 4. In addition, height can also be viewed as follows.

**Proposition 8.4:** *The height of a nonempty tree $T$ is equal to the maximum of the depths of its leaf positions.*

We leave the justification of this fact to an exercise (R-8.3). We present an algorithm, height1, implemented in Code Fragment 8.4 as a nonpublic method _height1 of the Tree class. It computes the height of a nonempty tree $T$ based on Proposition 8.4 and the algorithm depth from Code Fragment 8.3.

```
58   def _height1(self):                    # works, but O(n^2) worst-case time
59     """Return the height of the tree."""
60     return max(self.depth(p) for p in self.positions( ) if self.is_leaf(p))
```

**Code Fragment 8.4:** Method _height1 of the Tree class. Note that this method calls the depth method.

Unfortunately, algorithm height1 is not very efficient. We have not yet defined the positions( ) method; we will see that it can be implemented to run in $O(n)$ time, where $n$ is the number of positions of $T$. Because height1 calls algorithm depth($p$) on each leaf of $T$, its running time is $O(n + \sum_{p \in L}(d_p + 1))$, where $L$ is the set of leaf positions of $T$. In the worst case, the sum $\sum_{p \in L}(d_p + 1)$ is proportional to $n^2$. (See Exercise C-8.33.) Thus, algorithm height1 runs in $O(n^2)$ worst-case time.

We can compute the height of a tree more efficiently, in $O(n)$ worst-case time, by relying instead on the original recursive definition. To do this, we will parameterize a function based on a position within the tree, and calculate the height of the subtree rooted at that position. Algorithm height2, shown as nonpublic method _height2 in Code Fragment 8.5, computes the height of tree $T$ in this way.

```
61   def _height2(self, p):                 # time is linear in size of subtree
62     """Return the height of the subtree rooted at Position p."""
63     if self.is_leaf(p):
64       return 0
65     else:
66       return 1 + max(self._height2(c) for c in self.children(p))
```

**Code Fragment 8.5:** Method _height2 for computing the height of a subtree rooted at a position $p$ of a Tree.

It is important to understand why algorithm height2 is more efficient than height1. The algorithm is recursive, and it progresses in a top-down fashion. If the method is initially called on the root of $T$, it will eventually be called once for each position of $T$. This is because the root eventually invokes the recursion on each of its children, which in turn invokes the recursion on each of their children, and so on.

We can determine the running time of the height2 algorithm by summing, over all the positions, the amount of time spent on the nonrecursive part of each call. (Review Section 4.2 for analyses of recursive processes.) In our implementation, there is a constant amount of work per position, plus the overhead of computing the maximum over the iteration of children. Although we do not yet have a concrete implementation of children(p), we assume that such an iteration is generated in $O(c_p + 1)$ time, where $c_p$ denotes the number of children of $p$. Algorithm height2 spends $O(c_p + 1)$ time at each position $p$ to compute the maximum, and its overall running time is $O(\sum_p (c_p + 1)) = O(n + \sum_p c_p)$. In order to complete the analysis, we make use of the following property.

**Proposition 8.5:** *Let $T$ be a tree with $n$ positions, and let $c_p$ denote the number of children of a position $p$ of $T$. Then, summing over the positions of $T$, $\sum_p c_p = n - 1$.*

**Justification:** Each position of $T$, with the exception of the root, is a child of another position, and thus contributes one unit to the above sum. ■

By Proposition 8.5, the running time of algorithm height2, when called on the root of $T$, is $O(n)$, where $n$ is the number of positions of $T$.

Revisiting the public interface for our Tree class, the ability to compute heights of subtrees is beneficial, but a user might expect to be able to compute the height of the entire tree without explicitly designating the tree root. We can wrap the non-public _height2 in our implementation with a public height method that provides a default interpretation when invoked on tree $T$ with syntax T.height( ). Such an implementation is given in Code Fragment 8.6.

```
67   def height(self, p=None):
68     """Return the height of the subtree rooted at Position p.
69
70     If p is None, return the height of the entire tree.
71     """
72     if p is None:
73       p = self.root( )
74     return self._height2(p)          # start _height2 recursion
```

**Code Fragment 8.6:** Public method Tree.height that computes the height of the entire tree by default, or a subtree rooted at given position, if specified.

## 8.2 Binary Trees

A ***binary tree*** is an ordered tree with the following properties:

1. Every node has at most two children.
2. Each child node is labeled as being either a ***left child*** or a ***right child***.
3. A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of an internal node $v$ is called a ***left subtree*** or ***right subtree***, respectively, of $v$. A binary tree is ***proper*** if each node has either zero or two children. Some people also refer to such trees as being ***full*** binary trees. Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is ***improper***.

**Example 8.6:** *An important class of binary trees arises in contexts where we wish to represent a number of different outcomes that can result from answering a series of yes-or-no questions. Each internal node is associated with a question. Starting at the root, we go to the left or right child of the current node, depending on whether the answer to the question is "Yes" or "No." With each decision, we follow an edge from a parent to a child, eventually tracing a path in the tree from the root to a leaf. Such binary trees are known as* **decision trees**, *because a leaf position $p$ in such a tree represents a decision of what to do if the questions associated with $p$'s ancestors are answered in a way that leads to $p$. A decision tree is a proper binary tree. Figure 8.7 illustrates a decision tree that provides recommendations to a prospective investor.*

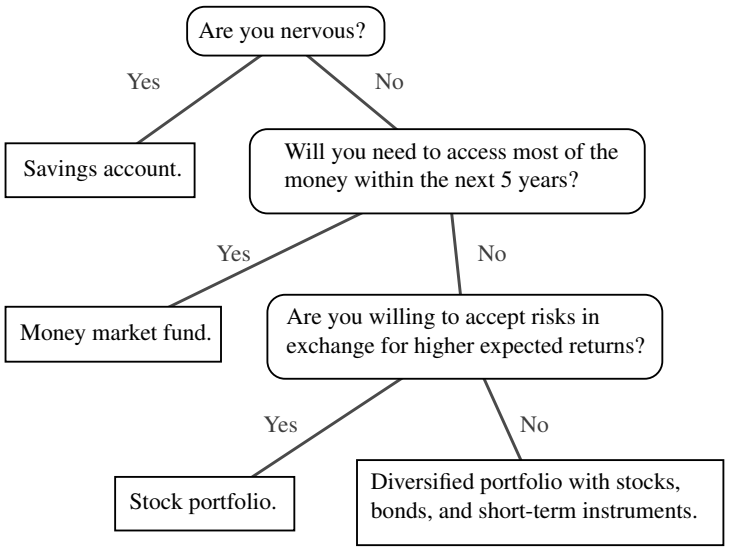



**Figure 8.7:** A decision tree providing investment advice.

**Example 8.7:** *An arithmetic expression can be represented by a binary tree whose leaves are associated with variables or constants, and whose internal nodes are associated with one of the operators $+$, $-$, $\times$, and $/$. (See Figure 8.8.) Each node in such a tree has a value associated with it.*

- *If a node is leaf, then its value is that of its variable or constant.*
- *If a node is internal, then its value is defined by applying its operation to the values of its children.*

*An arithmetic expression tree is a proper binary tree, since each operator $+$, $-$, $\times$, and $/$ takes exactly two operands. Of course, if we were to allow unary operators, like negation ($-$), as in "$-x$," then we could have an improper binary tree.*

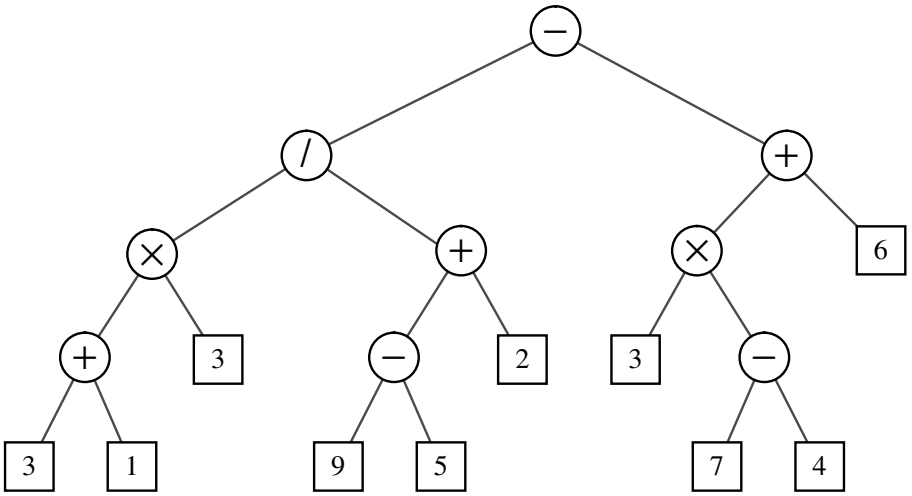



**Figure 8.8:** A binary tree representing an arithmetic expression. This tree represents the expression $((((3+1)\times 3)/((9-5)+2)) - ((3\times(7-4))+6))$. The value associated with the internal node labeled "/" is 2.

## A Recursive Binary Tree Definition

Incidentally, we can also define a binary tree in a recursive way such that a binary tree is either empty or consists of:

- A node $r$, called the root of $T$, that stores an element
- A binary tree (possibly empty), called the left subtree of $T$
- A binary tree (possibly empty), called the right subtree of $T$

### 8.2.1 The Binary Tree Abstract Data Type

As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

**T.left(p):** Return the position that represents the left child of p, or None if p has no left child.

**T.right(p):** Return the position that represents the right child of p, or None if p has no right child.

**T.sibling(p):** Return the position that represents the sibling of p, or None if p has no sibling.

Just as in Section 8.1.2 for the tree ADT, we do not define specialized update methods for binary trees here. Instead, we will consider some possible update methods when we describe specific implementations and applications of binary trees.

#### The BinaryTree Abstract Base Class in Python

Just as Tree was defined as an abstract base class in Section 8.1.2, we define a new BinaryTree class associated with the binary tree ADT. We rely on inheritance to define the BinaryTree class based upon the existing Tree class. However, our BinaryTree class remains *abstract*, as we still do not provide complete specifications for how such a structure will be represented internally, nor implementations for some necessary behaviors.

Our Python implementation of the BinaryTree class is given in Code Fragment 8.7. By using inheritance, a binary tree supports all the functionality that was defined for general trees (e.g., parent, is_leaf, root). Our new class also inherits the nested Position class that was originally defined within the Tree class definition. In addition, the new class provides declarations for new abstract methods left and right that should be supported by concrete subclasses of BinaryTree.

Our new class also provides two concrete implementations of methods. The new sibling method is derived from the combination of left, right, and parent. Typically, we identify the sibling of a position $p$ as the "other" child of $p$'s parent. However, if $p$ is the root, it has no parent, and thus no sibling. Also, $p$ may be the only child of its parent, and thus does not have a sibling.

Finally, Code Fragment 8.7 provides a concrete implementation of the children method; this method is abstract in the Tree class. Although we have still not specified how the children of a node will be stored, we derive a generator for the ordered children based upon the implied behavior of abstract methods left and right.

```
class BinaryTree(Tree):
  """Abstract base class representing a binary tree structure."""

  # --------------------- additional abstract methods ---------------------
  def left(self, p):
    """Return a Position representing p's left child.

    Return None if p does not have a left child.
    """
    raise NotImplementedError('must be implemented by subclass')

  def right(self, p):
    """Return a Position representing p's right child.

    Return None if p does not have a right child.
    """
    raise NotImplementedError('must be implemented by subclass')

  # ---------- concrete methods implemented in this class ----------
  def sibling(self, p):
    """Return a Position representing p's sibling (or None if no sibling)."""
    parent = self.parent(p)
    if parent is None:                    # p must be the root
      return None                         # root has no sibling
    else:
      if p == self.left(parent):
        return self.right(parent)         # possibly None
      else:
        return self.left(parent)          # possibly None

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    if self.left(p) is not None:
      yield self.left(p)
    if self.right(p) is not None:
      yield self.right(p)
```

**Code Fragment 8.7:** A BinaryTree abstract base class that extends the existing Tree abstract base class from Code Fragments 8.1 and 8.2.

### 8.2.2 Properties of Binary Trees

Binary trees have several interesting properties dealing with relationships between their heights and number of nodes. We denote the set of all nodes of a tree $T$ at the same depth $d$ as ***level*** $d$ of $T$. In a binary tree, level 0 has at most one node (the root), level 1 has at most two nodes (the children of the root), level 2 has at most four nodes, and so on. (See Figure 8.9.) In general, level $d$ has at most $2^d$ nodes.

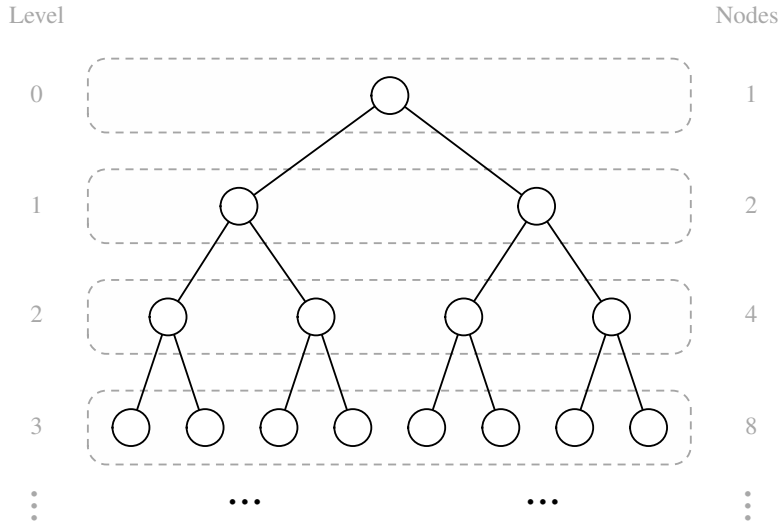



**Figure 8.9:** Maximum number of nodes in the levels of a binary tree.

We can see that the maximum number of nodes on the levels of a binary tree grows exponentially as we go down the tree. From this simple observation, we can derive the following properties relating the height of a binary tree $T$ with its number of nodes. A detailed justification of these properties is left as Exercise R-8.8.

**Proposition 8.8:** *Let $T$ be a nonempty binary tree, and let $n$, $n_E$, $n_I$ and $h$ denote the number of nodes, number of external nodes, number of internal nodes, and height of $T$, respectively. Then $T$ has the following properties:*

1. $h+1 \leq n \leq 2^{h+1}-1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq n-1$

*Also, if $T$ is proper, then $T$ has the following properties:*

1. $2h+1 \leq n \leq 2^{h+1}-1$
2. $h+1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq (n-1)/2$

## Relating Internal Nodes to External Nodes in a Proper Binary Tree

In addition to the earlier binary tree properties, the following relationship exists between the number of internal nodes and external nodes in a proper binary tree.

**Proposition 8.9:** *In a nonempty proper binary tree $T$, with $n_E$ external nodes and $n_I$ internal nodes, we have $n_E = n_I + 1$.*

**Justification:** We justify this proposition by removing nodes from $T$ and dividing them up into two "piles," an internal-node pile and an external-node pile, until $T$ becomes empty. The piles are initially empty. By the end, we will show that the external-node pile has one more node than the internal-node pile. We consider two cases:

***Case 1:*** If $T$ has only one node $v$, we remove $v$ and place it on the external-node pile. Thus, the external-node pile has one node and the internal-node pile is empty.

***Case 2:*** Otherwise ($T$ has more than one node), we remove from $T$ an (arbitrary) external node $w$ and its parent $v$, which is an internal node. We place $w$ on the external-node pile and $v$ on the internal-node pile. If $v$ has a parent $u$, then we reconnect $u$ with the former sibling $z$ of $w$, as shown in Figure 8.10. This operation, removes one internal node and one external node, and leaves the tree being a proper binary tree.

Repeating this operation, we eventually are left with a final tree consisting of a single node. Note that the same number of external and internal nodes have been removed and placed on their respective piles by the sequence of operations leading to this final tree. Now, we remove the node of the final tree and we place it on the external-node pile. Thus, the the external-node pile has one more node than the internal-node pile.

■

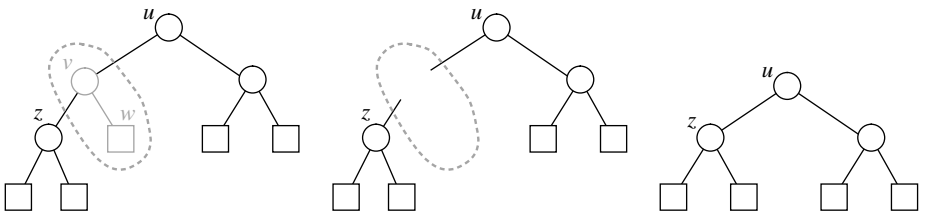



**Figure 8.10:** Operation that removes an external node and its parent node, used in the justification of Proposition 8.9.

Note that the above relationship does not hold, in general, for improper binary trees and nonbinary trees, although there are other interesting relationships that do hold. (See Exercises C-8.32 through C-8.34.)

# 8.3 Implementing Trees

The Tree and BinaryTree classes that we have defined thus far in this chapter are both formally ***abstract base classes***. Although they provide a great deal of support, neither of them can be directly instantiated. We have not yet defined key implementation details for how a tree will be represented internally, and how we can effectively navigate between parents and children. Specifically, a concrete implementation of a tree must provide methods root, parent, num_children, children, __len__, and in the case of BinaryTree, the additional accessors left and right.

There are several choices for the internal representation of trees. We describe the most common representations in this section. We begin with the case of a ***binary tree***, since its shape is more narrowly defined.

## 8.3.1 Linked Structure for Binary Trees

A natural way to realize a binary tree $T$ is to use a ***linked structure***, with a node (see Figure 8.11a) that maintains references to the element stored at a position $p$ and to the nodes associated with the children and parent of $p$. If $p$ is the root of $T$, then the parent field of $p$ is None. Likewise, if $p$ does not have a left child (respectively, right child), the associated field is None. The tree itself maintains an instance variable storing a reference to the root node (if any), and a variable, called size, that represents the overall number of nodes of $T$. We show such a linked structure representation of a binary tree in Figure 8.11b.

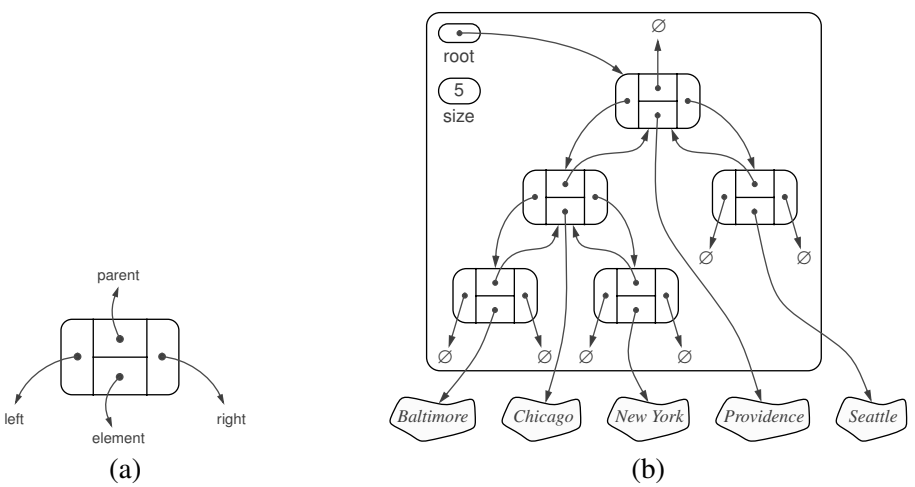



**Figure 8.11:** A linked structure for representing: (a) a single node; (b) a binary tree.

### Python Implementation of a Linked Binary Tree Structure

In this section, we define a concrete LinkedBinaryTree class that implements the binary tree ADT by subclassing the BinaryTree class. Our general approach is very similar to what we used when developing the PositionalList in Section 7.4: We define a simple, nonpublic _Node class to represent a node, and a public Position class that wraps a node. We provide a _validate utility for robustly checking the validity of a given position instance when unwrapping it, and a _make_position utility for wrapping a node as a position to return to a caller.

Those definitions are provided in Code Fragment 8.8. As a formality, the new Position class is declared to inherit immediately from BinaryTree.Position. Technically, the BinaryTree class definition (see Code Fragment 8.7) does not formally declare such a nested class; it trivially inherits it from Tree.Position. A minor benefit from this design is that our position class inherits the __ne__ special method so that syntax p != q is derived appropriately relative to __eq__.

Our class definition continues, in Code Fragment 8.9, with a constructor and with concrete implementations for the methods that remain abstract in the Tree and BinaryTree classes. The constructor creates an empty tree by initializing _root to None and _size to zero. These accessor methods are implemented with careful use of the _validate and _make_position utilities to safeguard against boundary cases.

### Operations for Updating a Linked Binary Tree

Thus far, we have provided functionality for examining an existing binary tree. However, the constructor for our LinkedBinaryTree class results in an empty tree and we have not provided any means for changing the structure or content of a tree.

We chose not to declare update methods as part of the Tree or BinaryTree abstract base classes for several reasons. First, although the principle of encapsulation suggests that the outward behaviors of a class need not depend on the internal representation, the *efficiency* of the operations depends greatly upon the representation. We prefer to have each concrete implementation of a tree class offer the most suitable options for updating a tree.

The second reason we do not provide update methods in the base class is that we may not want such update methods to be part of a public interface. There are many applications of trees, and some forms of update operations that are suitable for one application may be unacceptable in another. However, if we place an update method in a base class, any class that inherits from that base will inherit the update method. Consider, for example, the possibility of a method T.replace(p, e) that replaces the element stored at position $p$ with another element $e$. Such a general method may be unacceptable in the context of an ***arithmetic expression tree*** (see Example 8.7 on page 312, and a later case study in Section 8.5), because we may want to enforce that internal nodes store only operators as elements.

For linked binary trees, a reasonable set of update methods to support for general usage are the following:

**T.add_root(e):** Create a root for an empty tree, storing e as the element, and return the position of that root; an error occurs if the tree is not empty.

**T.add_left(p, e):** Create a new node storing element e, link the node as the left child of position p, and return the resulting position; an error occurs if p already has a left child.

**T.add_right(p, e):** Create a new node storing element e, link the node as the right child of position p, and return the resulting position; an error occurs if p already has a right child.

**T.replace(p, e):** Replace the element stored at position p with element e, and return the previously stored element.

**T.delete(p):** Remove the node at position p, replacing it with its child, if any, and return the element that had been stored at p; an error occurs if p has two children.

**T.attach(p, T1, T2):** Attach the internal structure of trees T1 and T2, respectively, as the left and right subtrees of leaf position p of T, and reset T1 and T2 to empty trees; an error condition occurs if p is not a leaf.

We have specifically chosen this collection of operations because each can be implemented in $O(1)$ worst-case time with our linked representation. The most complex of these are delete and attach, due to the case analyses involving the various parent-child relationships and boundary conditions, yet there remains only a constant number of operations to perform. (The implementation of both methods could be greatly simplified if we used a tree representation with a sentinel node, akin to our treatment of positional lists; see Exercise C-8.40).

To avoid the problem of undesirable update methods being inherited by subclasses of LinkedBinaryTree, we have chosen an implementation in which none of the above methods are publicly supported. Instead, we provide *nonpublic* versions of each, for example, providing the underscored _delete in lieu of a public delete. Our implementations of these six update methods are provided in Code Fragments 8.10 and 8.11.

In particular applications, subclasses of LinkedBinaryTree can invoke the nonpublic methods internally, while preserving a public interface that is appropriate for the application. A subclass may also choose to wrap one or more of the nonpublic update methods with a public method to expose it to the user. We leave as an exercise (R-8.15), the task of defining a MutableLinkedBinaryTree subclass that provides public methods wrapping each of these six update methods.

```
class LinkedBinaryTree(BinaryTree):
  """Linked representation of a binary tree structure."""

  class _Node:          # Lightweight, nonpublic class for storing a node.
    __slots__ = '_element', '_parent', '_left', '_right'
    def __init__(self, element, parent=None, left=None, right=None):
      self._element = element
      self._parent = parent
      self._left = left
      self._right = right

  class Position(BinaryTree.Position):
    """An abstraction representing the location of a single element."""

    def __init__(self, container, node):
      """Constructor should not be invoked by user."""
      self._container = container
      self._node = node

    def element(self):
      """Return the element stored at this Position."""
      return self._node._element

    def __eq__(self, other):
      """Return True if other is a Position representing the same location."""
      return type(other) is type(self) and other._node is self._node

  def _validate(self, p):
    """Return associated node, if position is valid."""
    if not isinstance(p, self.Position):
      raise TypeError('p must be proper Position type')
    if p._container is not self:
      raise ValueError('p does not belong to this container')
    if p._node._parent is p._node:          # convention for deprecated nodes
      raise ValueError('p is no longer valid')
    return p._node

  def _make_position(self, node):
    """Return Position instance for given node (or None if no node)."""
    return self.Position(self, node) if node is not None else None
```

**Code Fragment 8.8:** The beginning of our LinkedBinaryTree class (continued in Code Fragments 8.9 through 8.11).

```
41    #---------------------------- binary tree constructor ----------------------------
42    def __init__(self):
43      """Create an initially empty binary tree."""
44      self._root = None
45      self._size = 0
46
47    #---------------------------- public accessors ----------------------------
48    def __len__(self):
49      """Return the total number of elements in the tree."""
50      return self._size
51
52    def root(self):
53      """Return the root Position of the tree (or None if tree is empty)."""
54      return self._make_position(self._root)
55
56    def parent(self, p):
57      """Return the Position of p's parent (or None if p is root)."""
58      node = self._validate(p)
59      return self._make_position(node._parent)
60
61    def left(self, p):
62      """Return the Position of p's left child (or None if no left child)."""
63      node = self._validate(p)
64      return self._make_position(node._left)
65
66    def right(self, p):
67      """Return the Position of p's right child (or None if no right child)."""
68      node = self._validate(p)
69      return self._make_position(node._right)
70
71    def num_children(self, p):
72      """Return the number of children of Position p."""
73      node = self._validate(p)
74      count = 0
75      if node._left is not None:          # left child exists
76        count += 1
77      if node._right is not None:         # right child exists
78        count += 1
79      return count
```

**Code Fragment 8.9:** Public accessors for our LinkedBinaryTree class. The class begins in Code Fragment 8.8 and continues in Code Fragments 8.10 and 8.11.

```
80    def _add_root(self, e):
81      """Place element e at the root of an empty tree and return new Position.
82
83      Raise ValueError if tree nonempty.
84      """
85      if self._root is not None: raise ValueError('Root exists')
86      self._size = 1
87      self._root = self._Node(e)
88      return self._make_position(self._root)
89
90    def _add_left(self, p, e):
91      """Create a new left child for Position p, storing element e.
92
93      Return the Position of new node.
94      Raise ValueError if Position p is invalid or p already has a left child.
95      """
96      node = self._validate(p)
97      if node._left is not None: raise ValueError('Left child exists')
98      self._size += 1
99      node._left = self._Node(e, node)                    # node is its parent
100     return self._make_position(node._left)
101
102   def _add_right(self, p, e):
103     """Create a new right child for Position p, storing element e.
104
105     Return the Position of new node.
106     Raise ValueError if Position p is invalid or p already has a right child.
107     """
108     node = self._validate(p)
109     if node._right is not None: raise ValueError('Right child exists')
110     self._size += 1
111     node._right = self._Node(e, node)                   # node is its parent
112     return self._make_position(node._right)
113
114   def _replace(self, p, e):
115     """Replace the element at position p with e, and return old element."""
116     node = self._validate(p)
117     old = node._element
118     node._element = e
119     return old
```

**Code Fragment 8.10:** Nonpublic update methods for the LinkedBinaryTree class (continued in Code Fragment 8.11).

```
120   def _delete(self, p):
121     """Delete the node at Position p, and replace it with its child, if any.
122
123     Return the element that had been stored at Position p.
124     Raise ValueError if Position p is invalid or p has two children.
125     """
126     node = self._validate(p)
127     if self.num_children(p) == 2: raise ValueError('p has two children')
128     child = node._left if node._left else node._right        # might be None
129     if child is not None:
130       child._parent = node._parent      # child's grandparent becomes parent
131     if node is self._root:
132       self._root = child                # child becomes root
133     else:
134       parent = node._parent
135       if node is parent._left:
136         parent._left = child
137       else:
138         parent._right = child
139     self._size -= 1
140     node._parent = node                 # convention for deprecated node
141     return node._element
142
143   def _attach(self, p, t1, t2):
144     """Attach trees t1 and t2 as left and right subtrees of external p."""
145     node = self._validate(p)
146     if not self.is_leaf(p): raise ValueError('position must be leaf')
147     if not type(self) is type(t1) is type(t2):   # all 3 trees must be same type
148       raise TypeError('Tree types must match')
149     self._size += len(t1) + len(t2)
150     if not t1.is_empty():               # attached t1 as left subtree of node
151       t1._root._parent = node
152       node._left = t1._root
153       t1._root = None                   # set t1 instance to empty
154       t1._size = 0
155     if not t2.is_empty():               # attached t2 as right subtree of node
156       t2._root._parent = node
157       node._right = t2._root
158       t2._root = None                   # set t2 instance to empty
159       t2._size = 0
```

**Code Fragment 8.11:** Nonpublic update methods for the LinkedBinaryTree class (continued from Code Fragment 8.10).

### Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the LinkedBinaryTree methods, including derived methods that are inherited from the Tree and BinaryTree classes:

- The len method, implemented in LinkedBinaryTree, uses an instance variable storing the number of nodes of $T$ and takes $O(1)$ time. Method is_empty, inherited from Tree, relies on a single call to len and thus takes $O(1)$ time.
- The accessor methods root, left, right, parent, and num_children are implemented directly in LinkedBinaryTree and take $O(1)$ time. The sibling and children methods are derived in BinaryTree based on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The is_root and is_leaf methods, from the Tree class, both run in $O(1)$ time, as is_root calls root and then relies on equivalence testing of positions, while is_leaf calls left and right and verifies that None is returned by both.
- Methods depth and height were each analyzed in Section 8.1.3. The depth method at position $p$ runs in $O(d_p + 1)$ time where $d_p$ is its depth; the height method on the root of the tree runs in $O(n)$ time.
- The various update methods add_root, add_left, add_right, replace, delete, and attach (that is, their nonpublic implementations) each run in $O(1)$ time, as they involve relinking only a constant number of nodes per operation.

Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

| **Operation** | **Running Time** |
|---|---|
| len, is_empty | $O(1)$ |
| root, parent, left, right, sibling, children, num_children | $O(1)$ |
| is_root, is_leaf | $O(1)$ |
| depth(p) | $O(d_p + 1)$ |
| height | $O(n)$ |
| add_root, add_left, add_right, replace, delete, attach | $O(1)$ |

**Table 8.1:** Running times for the methods of an $n$-node binary tree implemented with a linked structure. The space usage is $O(n)$.

### 8.3.2 Array-Based Representation of a Binary Tree

An alternative representation of a binary tree $T$ is based on a way of numbering the positions of $T$. For every position $p$ of $T$, let $f(p)$ be the integer defined as follows.

- If $p$ is the root of $T$, then $f(p) = 0$.
- If $p$ is the left child of position $q$, then $f(p) = 2f(q) + 1$.
- If $p$ is the right child of position $q$, then $f(p) = 2f(q) + 2$.

The numbering function $f$ is known as a ***level numbering*** of the positions in a binary tree $T$, for it numbers the positions on each level of $T$ in increasing order from left to right. (See Figure 8.12.) Note well that the level numbering is based on *potential* positions within the tree, not actual positions of a given tree, so they are not necessarily consecutive. For example, in Figure 8.12(b), there are no nodes with level numbering 13 or 14, because the node with level numbering 6 has no children.

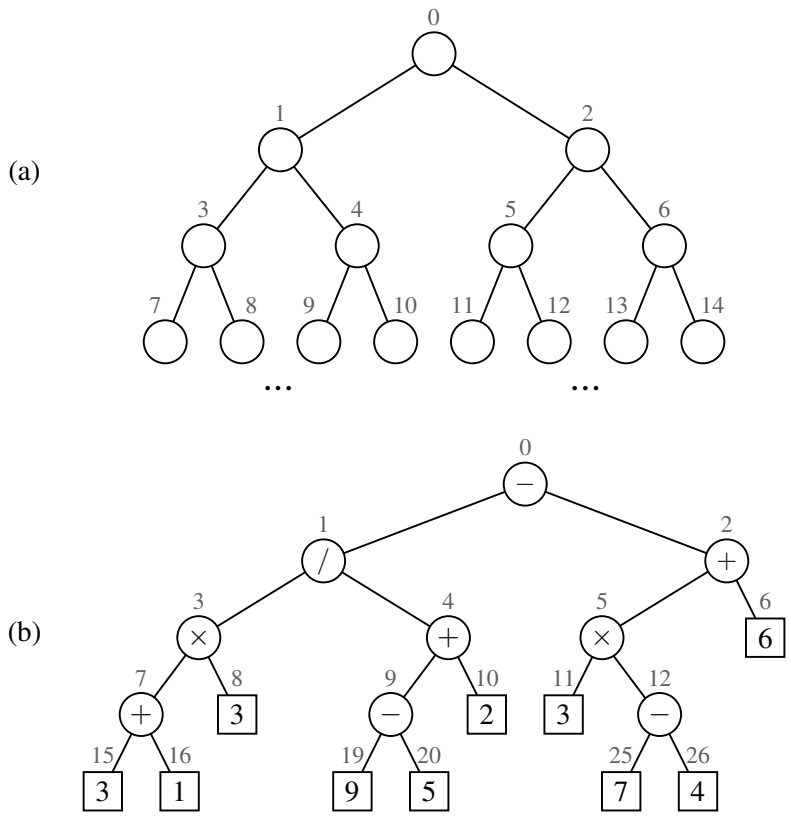



**Figure 8.12:** Binary tree level numbering: (a) general scheme; (b) an example.

The level numbering function $f$ suggests a representation of a binary tree $T$ by means of an array-based structure $A$ (such as a Python list), with the element at position $p$ of $T$ stored at index $f(p)$ of the array. We show an example of an array-based representation of a binary tree in Figure 8.13.

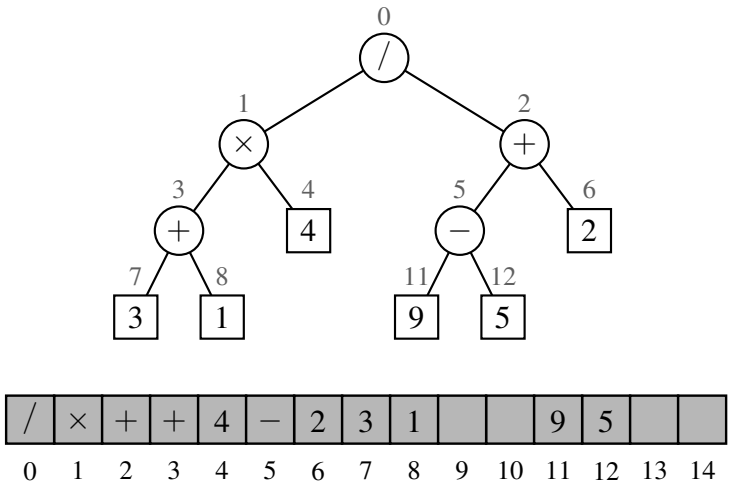



**Figure 8.13:** Representation of a binary tree by means of an array.

One advantage of an array-based representation of a binary tree is that a position $p$ can be represented by the single integer $f(p)$, and that position-based methods such as root, parent, left, and right can be implemented using simple arithmetic operations on the number $f(p)$. Based on our formula for the level numbering, the left child of $p$ has index $2f(p)+1$, the right child of $p$ has index $2f(p)+2$, and the parent of $p$ has index $\lfloor (f(p)-1)/2 \rfloor$. We leave the details of a complete implementation as an exercise (R-8.18).

The space usage of an array-based representation depends greatly on the shape of the tree. Let $n$ be the number of nodes of $T$, and let $f_M$ be the maximum value of $f(p)$ over all the nodes of $T$. The array $A$ requires length $N = 1 + f_M$, since elements range from $A[0]$ to $A[f_M]$. Note that $A$ may have a number of empty cells that do not refer to existing nodes of $T$. In fact, in the worst case, $N = 2^n - 1$, the justification of which is left as an exercise (R-8.16). In Section 9.3, we will see a class of binary trees, called "heaps" for which $N = n$. Thus, in spite of the worst-case space usage, there are applications for which the array representation of a binary tree is space efficient. Still, for general binary trees, the exponential worst-case space requirement of this representation is prohibitive.

Another drawback of an array representation is that some update operations for trees cannot be efficiently supported. For example, deleting a node and promoting its child takes $O(n)$ time because it is not just the child that moves locations within the array, but all descendants of that child.

### 8.3.3 Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree $T$ as a linked structure is to have each node store a single *container* of references to its children. For example, a children field of a node can be a Python list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.14.

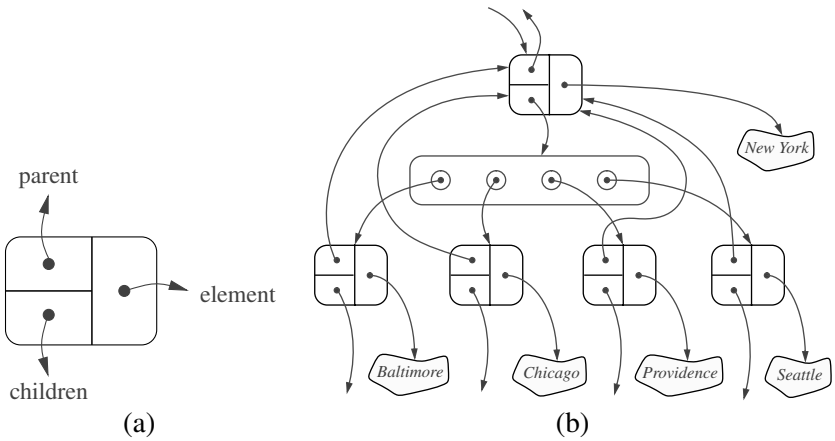



**Figure 8.14:** The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.14), but we note that, by using a collection to store the children of each position $p$, we can implement children($p$) by simply iterating that collection.

| Operation | Running Time |
|---:|---|
| len, is_empty | $O(1)$ |
| root, parent, is_root, is_leaf | $O(1)$ |
| children($p$) | $O(c_p+1)$ |
| depth(p) | $O(d_p+1)$ |
| height | $O(n)$ |

**Table 8.2:** Running times of the accessor methods of an $n$-node general tree implemented with a linked structure. We let $c_p$ denote the number of children of a position $p$. The space usage is $O(n)$.

# 8.4 Tree Traversal Algorithms

A ***traversal*** of a tree $T$ is a systematic way of accessing, or "visiting," all the positions of $T$. The specific action associated with the "visit" of a position $p$ depends on the application of this traversal, and could involve anything from incrementing a counter to performing some complex computation for $p$. In this section, we describe several common traversal schemes for trees, implement them in the context of our various tree classes, and discuss several common applications of tree traversals.

## 8.4.1 Preorder and Postorder Traversals of General Trees

In a ***preorder traversal*** of a tree $T$, the root of $T$ is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudo-code for the preorder traversal of the subtree rooted at a position $p$ is shown in Code Fragment 8.12.

```
Algorithm preorder(T, p):
  perform the "visit" action for position p
  for each child c in T.children(p) do
    preorder(T, c)                {recursively traverse the subtree rooted at c}
```

**Code Fragment 8.12:** Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree T.

Figure 8.15 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

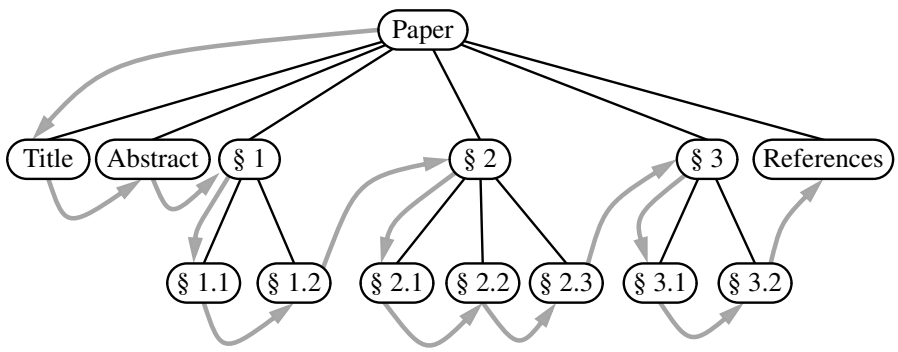



**Figure 8.15:** Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

### Postorder Traversal

Another important tree traversal algorithm is the ***postorder traversal***. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name "postorder"). Pseudo-code for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.16.

```
Algorithm postorder(T, p):
  for each child c in T.children(p) do
    postorder(T, c)                    {recursively traverse the subtree rooted at c}
  perform the "visit" action for position p
```

**Code Fragment 8.13:** Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree T.

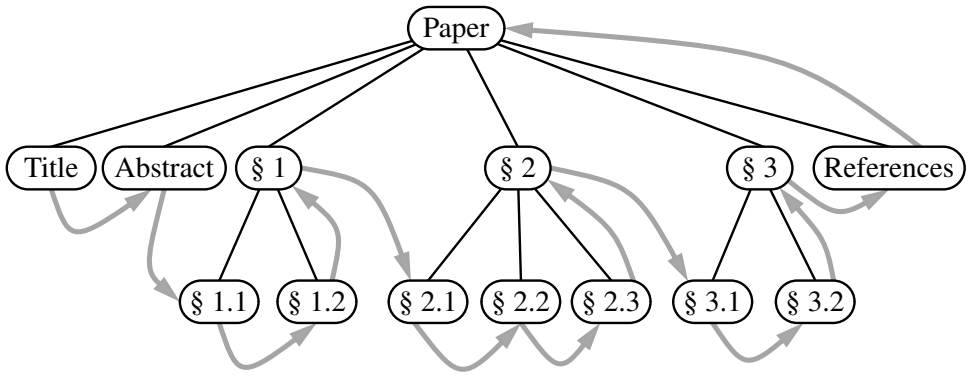



**Figure 8.16:** Postorder traversal of the ordered tree of Figure 8.15.

### Running-Time Analysis

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height2, given in Code Fragment 8.5 of Section 8.1.3. At each position $p$, the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where $c_p$ is the number of children of $p$, under the assumption that the "visit" itself takes $O(1)$ time. By Proposition 8.5, the overall running time for the traversal of tree $T$ is $O(n)$, where $n$ is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all the $n$ positions of the tree.

### 8.4.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another common approach is to traverse a tree so that we visit all the positions at depth $d$ before we visit the positions at depth $d+1$. Such an algorithm is known as a ***breadth-first traversal***.

A breadth-first traversal is a common approach used in software for playing games. A ***game tree*** represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.17 displays a partial game tree for Tic-Tac-Toe.

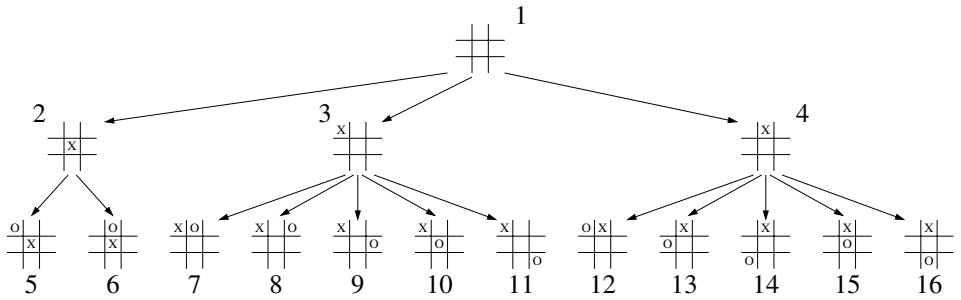



**Figure 8.17:** Partial game tree for Tic-Tac-Toe, with annotations displaying the order in which positions are visited in a breadth-first traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudo-code for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the $n$ calls to enqueue and $n$ calls to dequeue.

```
Algorithm breadthfirst(T):
  Initialize queue Q to contain T.root( )
  while Q not empty do
    p = Q.dequeue( )                    {p is the oldest entry in the queue}
    perform the "visit" action for position p
    for each child c in T.children(p) do
      Q.enqueue(c)     {add p's children to the end of the queue for later visits}
```

**Code Fragment 8.14:** Algorithm for performing a breadth-first traversal of a tree.

### 8.4.3 Inorder Traversal of a Binary Tree

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees, can be directly applied to binary trees. In this section, we introduce another common traversal algorithm specifically for a binary tree.

During an ***inorder traversal***, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree $T$ can be informally viewed as visiting the nodes of $T$ "from left to right." Indeed, for every position $p$, the inorder traversal visits $p$ after all the positions in the left subtree of $p$ and before all the positions in the right subtree of $p$. Pseudo-code for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.18.

```
Algorithm inorder(p):
  if p has a left child lc then
    inorder(lc)                    {recursively traverse the left subtree of p}
  perform the "visit" action for position p
  if p has a right child rc then
    inorder(rc)                    {recursively traverse the right subtree of p}
```

**Code Fragment 8.15:** Algorithm inorder for performing an inorder traversal of a subtree rooted at position p of a binary tree.

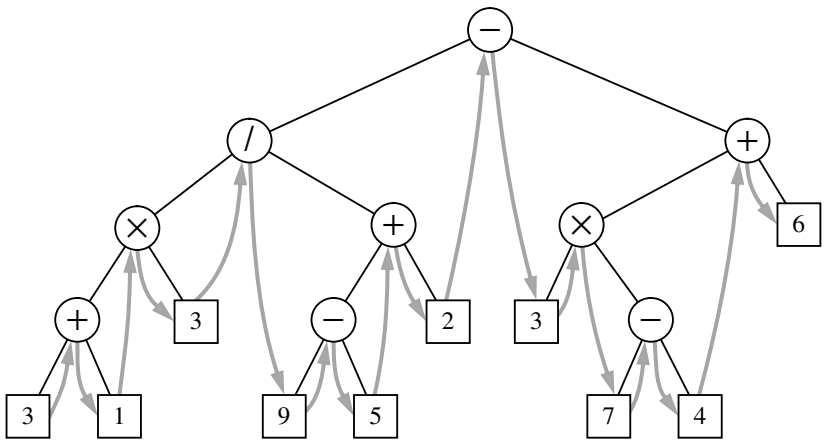



**Figure 8.18:** Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.18, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3+1\times 3/9-5+2\ldots$ (albeit without parentheses).

## Binary Search Trees

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a ***binary search tree***. Let $S$ be a set whose unique elements have an order relation. For example, $S$ could be a set of integers. A binary search tree for $S$ is a binary tree $T$ such that, for each position $p$ of $T$:

- Position $p$ stores an element of $S$, denoted as $e(p)$.
- Elements stored in the left subtree of $p$ (if any) are less than $e(p)$.
- Elements stored in the right subtree of $p$ (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.19. The above properties assure that an inorder traversal of a binary search tree $T$ visits the elements in nondecreasing order.

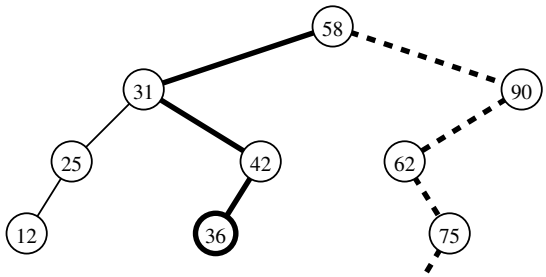



**Figure 8.19:** A binary search tree storing integers. The solid path is traversed when searching (successfully) for 36. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree $T$ for set $S$ to find whether a given search value $v$ is in $S$, by traversing a path down the tree $T$, starting at the root. At each internal position $p$ encountered, we compare our search value $v$ with the element $e(p)$ stored at $p$. If $v < e(p)$, then the search continues in the left subtree of $p$. If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of $p$. Finally, if we reach an empty subtree, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.6), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.19.

Note that the running time of searching in a binary search tree $T$ is proportional to the height of $T$. Recall from Proposition 8.8 that the height of a binary tree with $n$ nodes can be as small as $\log(n+1)-1$ or as large as $n-1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

### 8.4.4 Implementing Tree Traversals in Python

When first defining the tree ADT in Section 8.1.2, we stated that tree T should include support for the following methods:

**T.positions( ):** Generate an iteration of all *positions* of tree T.

**iter(T):** Generate an iteration of all *elements* stored within tree T.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we demonstrate how any of the tree traversal algorithms we have introduced could be used to produce these iterations.

To begin, we note that it is easy to produce an iteration of all elements of a tree, if we rely on a presumed iteration of all positions. Therefore, support for the iter(T) syntax can be formally provided by a concrete implementation of the special method __iter__ within the abstract base class Tree. We rely on Python's generator syntax as the mechanism for producing iterations. (See Section 1.8.) Our implementation of Tree.__iter__ is given in Code Fragment 8.16.

```
75  def __iter__(self):
76    """Generate an iteration of the tree's elements."""
77    for p in self.positions( ):        # use same order as positions()
78      yield p.element( )               # but yield each element
```

**Code Fragment 8.16:** Iterating all elements of a Tree instance, based upon an iteration of the positions of the tree. This code should be included in the body of the Tree class.

To implement the positions method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we will provide independent implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the positions method of the tree ADT.

#### Preorder Traversal

We begin by considering the ***preorder traversal*** algorithm. We will support a public method with calling signature T.preorder( ) for tree T, which generates a preorder iteration of all positions within the tree. However, the recursive algorithm for generating a preorder traversal, as originally described in Code Fragment 8.12, must be parameterized by a specific position within the tree that serves as the root of a subtree to traverse. A standard solution for such a circumstance is to define a nonpublic utility method with the desired recursive parameterization, and then to have the public method preorder invoke the nonpublic method upon the root of the tree. Our implementation of such a design is given in Code Fragment 8.17.

```
79   def preorder(self):
80     """Generate a preorder iteration of positions in the tree."""
81     if not self.is_empty( ):
82       for p in self._subtree_preorder(self.root( )):       # start recursion
83         yield p
84
85   def _subtree_preorder(self, p):
86     """Generate a preorder iteration of positions in subtree rooted at p."""
87     yield p                                    # visit p before its subtrees
88     for c in self.children(p):                 # for each child c
89       for other in self._subtree_preorder(c):  # do preorder of c's subtree
90         yield other                            # yielding each to our caller
```

**Code Fragment 8.17:** Support for performing a preorder traversal of a tree. This code should be included in the body of the Tree class.

Formally, both preorder and the utility _subtree_preorder are generators. Rather than perform a "visit" action from within this code, we yield each position to the caller and let the caller decide what action to perform at that position.

The _subtree_preorder method is the recursive one. However, because we are relying on generators rather than traditional functions, the recursion has a slightly different form. In order to yield all positions within the subtree of child $c$, we loop over the positions yielded by the recursive call self._subtree_preorder(c), and re-yield each position in the outer context. Note that if $p$ is a leaf, the for loop over self.children(p) is trivial (this is the base case for our recursion).

We rely on a similar technique in the public preorder method to re-yield all positions that are generated by the recursive process starting at the root of the tree; if the tree is empty, nothing is yielded. At this point, we have provided full support for the preorder generator. A user of the class can therefore write code such as

```
for p in T.preorder( ):
  # "visit" position p
```

The official tree ADT requires that all trees support a positions method as well. To use a preorder traversal as the default order of iteration, we include the definition shown in Code Fragment 8.18 within our Tree class. Rather than loop over the results returned by the preorder call, we return the entire iteration as an object.

```
91   def positions(self):
92     """Generate an iteration of the tree's positions."""
93     return self.preorder( )              # return entire preorder iteration
```

**Code Fragment 8.18:** An implementation of the positions method for the Tree class that relies on a preorder traversal to generate the results.

### Postorder Traversal

We can implement a postorder traversal using very similar technique as with a preorder traversal. The only difference is that within the recursive utility for a postorder we wait to yield position $p$ until *after* we have recursively yield the positions in its subtrees. An implementation is given in Code Fragment 8.19.

```
94   def postorder(self):
95     """Generate a postorder iteration of positions in the tree."""
96     if not self.is_empty( ):
97       for p in self._subtree_postorder(self.root( )):       # start recursion
98         yield p
99
100  def _subtree_postorder(self, p):
101    """Generate a postorder iteration of positions in subtree rooted at p."""
102    for c in self.children(p):                    # for each child c
103      for other in self._subtree_postorder(c):    # do postorder of c's subtree
104        yield other                               # yielding each to our caller
105    yield p                                       # visit p after its subtrees
```

**Code Fragment 8.19:** Support for performing a postorder traversal of a tree. This code should be included in the body of the Tree class.

### Breadth-First Traversal

In Code Fragment 8.20, we provide an implementation of the breadth-first traversal algorithm in the context of our Tree class. Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. Our implementation uses the LinkedQueue class from Section 7.1.2, although any implementation of the queue ADT would suffice.

### Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees, and so we include their implementations within the Tree abstract base class. Those methods are inherited by the abstract BinaryTree class, the concrete LinkedBinaryTree class, and any other dependent tree classes we might develop.

The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the BinaryTree class. We use a similar technique to implement an inorder traversal (Code Fragment 8.21) as we did with preorder and postorder traversals.

```
106   def breadthfirst(self):
107     """Generate a breadth-first iteration of the positions of the tree."""
108     if not self.is_empty( ):
109       fringe = LinkedQueue( )                # known positions not yet yielded
110       fringe.enqueue(self.root( ))           # starting with the root
111       while not fringe.is_empty( ):
112         p = fringe.dequeue( )                # remove from front of the queue
113         yield p                              # report this position
114         for c in self.children(p):
115           fringe.enqueue(c)                  # add children to back of queue
```

**Code Fragment 8.20:** An implementation of a breadth-first traversal of a tree. This code should be included in the body of the Tree class.

```
37   def inorder(self):
38     """Generate an inorder iteration of positions in the tree."""
39     if not self.is_empty( ):
40       for p in self._subtree_inorder(self.root( )):
41         yield p
42
43   def _subtree_inorder(self, p):
44     """Generate an inorder iteration of positions in subtree rooted at p."""
45     if self.left(p) is not None:          # if left child exists, traverse its subtree
46       for other in self._subtree_inorder(self.left(p)):
47         yield other
48     yield p                               # visit p between its subtrees
49     if self.right(p) is not None:         # if right child exists, traverse its subtree
50       for other in self._subtree_inorder(self.right(p)):
51         yield other
```

**Code Fragment 8.21:** Support for performing an inorder traversal of a binary tree. This code should be included in the BinaryTree class (given in Code Fragment 8.7).

For many applications of binary trees, an inorder traversal provides a natural iteration. We could make it the default for the BinaryTree class by overriding the positions method that was inherited from the Tree class (see Code Fragment 8.22).

```
52   # override inherited version to make inorder the default
53   def positions(self):
54     """Generate an iteration of the tree's positions."""
55     return self.inorder( )                # make inorder the default
```

**Code Fragment 8.22:** Defining the BinaryTree.position method so that positions are reported using inorder traversal.

### 8.4.5 Applications of Tree Traversals

In this section, we demonstrate several representative applications of tree traversals, including some customizations of the standard traversal algorithms.

#### Table of Contents

When using a tree to represent the hierarchical structure of a document, a preorder traversal of the tree can naturally be used to produce a table of contents for the document. For example, the table of contents associated with the tree from Figure 8.15 is displayed in Figure 8.20. Part (a) of that figure gives a simple presentation with one element per line; part (b) shows a more attractive presentation produced by indenting each element based on its depth within the tree. A similar presentation could be used to display the contents of a computer's file system, based on its tree representation (as in Figure 8.3).

```
Paper                Paper
Title                  Title
Abstract               Abstract
§1                     §1
§1.1                     §1.1
§1.2                     §1.2
§2                     §2
§2.1                     §2.1
...                      ...

(a)                  (b)
```

**Figure 8.20:** Table of contents for a document represented by the tree in Figure 8.15: (a) without indentation; (b) with indentation based on depth within the tree.

The unindented version of the table of contents, given a tree $T$, can be produced with the following code:

```
for p in T.preorder( ):
  print(p.element( ))
```

To produce the presentation of Figure 8.20(b), we indent each element with a number of spaces equal to twice the element's depth in the tree (hence, the root element was unindented). Although we could replace the body of the above loop with the statement `print(2*T.depth(p)*' ' + str(p.element( )))`, such an approach is unnecessarily inefficient. Although the work to produce the preorder traversal runs in $O(n)$ time, based on the analysis of Section 8.4.1, the calls to `depth` incur a hidden cost. Making a call to `depth` from every position of the tree results in $O(n^2)$ worst-case time, as noted when analyzing the algorithm `height1` in Section 8.1.3.

A preferred approach to producing an indented table of contents is to redesign a top-down recursion that includes the current depth as an additional parameter. Such an implementation is provided in Code Fragment 8.23. This implementation runs in worst-case $O(n)$ time (except, technically, the time it takes to print strings of increasing lengths).

```
1  def preorder_indent(T, p, d):
2    """Print preorder representation of subtree of T rooted at p at depth d."""
3    print(2*d*' ' + str(p.element( )))          # use depth for indentation
4    for c in T.children(p):
5      preorder_indent(T, c, d+1)                # child depth is d+1
```

**Code Fragment 8.23:** Efficient recursion for printing indented version of a preorder traversal. On a complete tree $T$, the recursion should be started with form preorder_indent(T, T.root( ), 0).

In the example of Figure 8.20, we were fortunate in that the numbering was embedded within the elements of the tree. More generally, we might be interested in using a preorder traversal to display the structure of a tree, with indentation and also explicit numbering that was not present in the tree. For example, we might display the tree from Figure 8.2 beginning as:

```
Electronics R'Us
  1 R&D
  2 Sales
    2.1 Domestic
    2.2 International
      2.2.1 Canada
      2.2.2 S. America
```

This is more challenging, because the numbers used as labels are implicit in the structure of the tree. A label depends on the index of each position, relative to its siblings, along the path from the root to the current position. To accomplish the task, we add a representation of that path as an additional parameter to the recursive signature. Specifically, we use a list of zero-indexed numbers, one for each position along the downward path, other than the root. (We convert those numbers to one-indexed form when printing.)

At the implementation level, we wish to avoid the inefficiency of duplicating such lists when sending a new parameter from one level of the recursion to the next. A standard solution is to share the same list instance throughout the recursion. At one level of the recursion, a new entry is temporarily added to the end of the list before making further recursive calls. In order to "leave no trace," that same block of code must remove the extraneous entry from the list before completing its task. An implementation based on this approach is given in Code Fragment 8.24.

```
1  def preorder_label(T, p, d, path):
2    """Print labeled representation of subtree of T rooted at p at depth d."""
3    label = '.'.join(str(j+1) for j in path)    # displayed labels are one-indexed
4    print(2*d*'  ' + label, p.element())
5    path.append(0)                              # path entries are zero-indexed
6    for c in T.children(p):
7      preorder_label(T, c, d+1, path)           # child depth is d+1
8      path[-1] += 1
9    path.pop()
```

**Code Fragment 8.24:** Efficient recursion for printing an indented and *labeled* presentation of a preorder traversal.

### Parenthetic Representations of a Tree

It is not possible to reconstruct a general tree, given only the preorder sequence of elements, as in Figure 8.20(a). Some additional context is necessary for the structure of the tree to be well defined. The use of indentation or numbered labels provides such context, with a very human-friendly presentation. However, there are more concise string representations of trees that are computer-friendly.

In this section, we explore one such representation. The ***parenthetic string representation*** $P(T)$ of tree $T$ is recursively defined as follows. If $T$ consists of a single position $p$, then

$$P(T) = \mathsf{str}(p.\mathsf{element}()).$$

Otherwise, it is defined recursively as,

$$P(T) = \mathsf{str}(p.\mathsf{element}()) + \texttt{'('} + P(T_1) + \texttt{', '} + \cdots + \texttt{', '} + P(T_k) + \texttt{')'}$$

where $p$ is the root of $T$ and $T_1, T_2, \ldots, T_k$ are the subtrees rooted at the children of $p$, which are given in order if $T$ is an ordered tree. We are using "+" here to denote string concatenation. As an example, the parenthetic representation of the tree of Figure 8.2 would appear as follows (line breaks are cosmetic):

```
Electronics R'Us (R&D, Sales (Domestic, International (Canada,
S. America, Overseas (Africa, Europe, Asia, Australia))),
Purchasing, Manufacturing (TV, CD, Tuner))
```

Although the parenthetic representation is essentially a preorder traversal, we cannot easily produce the additional punctuation using the formal implementation of preorder, as given in Code Fragment 8.17. The opening parenthesis must be produced just before the loop over a position's children and the closing parenthesis must be produced just after that loop. Furthermore, the separating commas must be produced. The Python function parenthesize, shown in Code Fragment 8.25, is a custom traversal that prints such a parenthetic string representation of a tree $T$.

```
def parenthesize(T, p):
  """Print parenthesized representation of subtree of T rooted at p."""
  print(p.element( ), end='')          # use of end avoids trailing newline
  if not T.is_leaf(p):
    first_time = True
    for c in T.children(p):
      sep = ' (' if first_time else ', '          # determine proper separator
      print(sep, end='')
      first_time = False                # any future passes will not be the first
      parenthesize(T, c)                # recur on child
    print(')', end='')                  # include closing parenthesis
```

**Code Fragment 8.25:** Function that prints parenthetic string representation of a tree.

## Computing Disk Space

In Example 8.1, we considered the use of a tree as a model for a file-system structure, with internal positions representing directories and leaves representing files. In fact, when introducing the use of recursion back in Chapter 4, we specifically examined the topic of file systems (see Section 4.1.4). Although we did not explicitly model it as a tree at that time, we gave an implementation of an algorithm for computing the disk usage (Code Fragment 4.5).

The recursive computation of disk space is emblematic of a *postorder* traversal, as we cannot effectively compute the total space used by a directory until *after* we know the space that is used by its children directories. Unfortunately, the formal implementation of postorder, as given in Code Fragment 8.19 does not suffice for this purpose. As it visits the position of a directory, there is no easy way to discern which of the previous positions represent children of that directory, nor how much recursive disk space was allocated.

We would like to have a mechanism for children to return information to the parent as part of the traversal process. A custom solution to the disk space problem, with each level of recursion providing a return value to the (parent) caller, is provided in Code Fragment 8.26.

```
def disk_space(T, p):
  """Return total disk space for subtree of T rooted at p."""
  subtotal = p.element( ).space( )          # space used at position p
  for c in T.children(p):
    subtotal += disk_space(T, c)            # add child's space to subtotal
  return subtotal
```

**Code Fragment 8.26:** Recursive computation of disk space for a tree. We assume that a space( ) method of each tree element reports the local space used at that position.

### 8.4.6 Euler Tours and the Template Method Pattern ⋆

The various applications described in Section 8.4.5 demonstrate the great power of recursive tree traversals. Unfortunately, they also show that the specific implementations of the preorder and postorder methods of our Tree class, or the inorder method of the BinaryTree class, are not general enough to capture the range of computations we desire. In some cases, we need more of a blending of the approaches, with initial work performed before recurring on subtrees, additional work performed after those recursions, and in the case of a binary tree, work performed between the two possible recursions. Furthermore, in some contexts it was important to know the depth of a position, or the complete path from the root to that position, or to return information from one level of the recursion to another. For each of the previous applications, we were able to develop a custom implementation to properly adapt the recursive ideas, but the great principles of object-oriented programming introduced in Section 2.1.1 include ***adaptability*** and ***reusability***.

In this section, we develop a more general framework for implementing tree traversals based on a concept known as an ***Euler tour traversal***. The Euler tour traversal of a general tree $T$ can be informally defined as a "walk" around $T$, where we start by going from the root toward its leftmost child, viewing the edges of $T$ as being "walls" that we always keep to our left. (See Figure 8.21.)

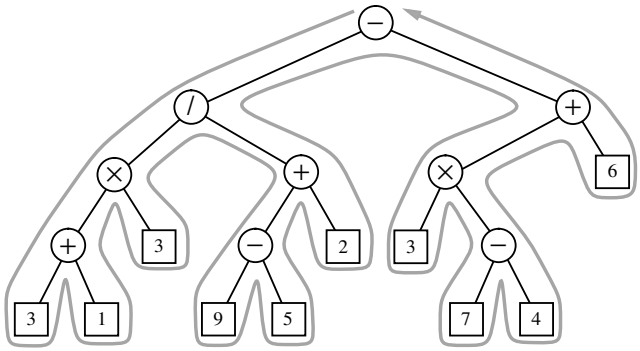



**Figure 8.21:** Euler tour traversal of a tree.

The complexity of the walk is $O(n)$, because it progresses exactly two times along each of the $n-1$ edges of the tree—once going downward along the edge, and later going upward along the edge. To unify the concept of preorder and postorder traversals, we can think of there being two notable "visits" to each position $p$:

- A "pre visit" occurs when first reaching the position, that is, when the walk passes immediately left of the node in our visualization.
- A "post visit" occurs when the walk later proceeds upward from that position, that is, when the walk passes to the right of the node in our visualization.

The process of an Euler tour can easily be viewed recursively. In between the "pre visit" and "post visit" of a given position will be a recursive tour of each of its subtrees. Looking at Figure 8.21 as an example, there is a contiguous portion of the entire tour that is itself an Euler tour of the subtree of the node with element "/". That tour contains two contiguous subtours, one traversing that position's left subtree and another traversing the right subtree. The pseudo-code for an Euler tour traversal of a subtree rooted at a position $p$ is shown in Code Fragment 8.27.

```
Algorithm eulertour(T, p):
  perform the "pre visit" action for position p
  for each child c in T.children(p) do
    eulertour(T, c)                    {recursively tour the subtree rooted at c}
  perform the "post visit" action for position p
```

**Code Fragment 8.27:** Algorithm eulertour for performing an Euler tour traversal of a subtree rooted at position p of a tree.

## The Template Method Pattern

To provide a framework that is reusable and adaptable, we rely on an interesting object-oriented software design pattern, the ***template method pattern***. The template method pattern describes a generic computation mechanism that can be specialized for a particular application by redefining certain steps. To allow customization, the primary algorithm calls auxiliary functions known as ***hooks*** at designated steps of the process.

In the context of an Euler tour traversal, we define two separate hooks, a previsit hook that is called before the subtrees are traversed, and a postvisit hook that is called after the completion of the subtree traversals. Our implementation will take the form of an EulerTour class that manages the process, and defines trivial definitions for the hooks that do nothing. The traversal can be customized by defining a subclass of EulerTour and overriding one or both hooks to provide specialized behavior.

## Python Implementation

Our implementation of an EulerTour class is provided in Code Fragment 8.28. The primary recursive process is defined in the nonpublic _tour method. A tour instance is created by sending a reference to a specific tree to the constructor, and then by calling the public execute method, which beings the tour and returns a final result of the computation.

```
1  class EulerTour:
2    """Abstract base class for performing Euler tour of a tree.
3
4    _hook_previsit and _hook_postvisit may be overridden by subclasses.
5    """
6    def __init__(self, tree):
7      """Prepare an Euler tour template for given tree."""
8      self._tree = tree
9
10   def tree(self):
11     """Return reference to the tree being traversed."""
12     return self._tree
13
14   def execute(self):
15     """Perform the tour and return any result from post visit of root."""
16     if len(self._tree) > 0:
17       return self._tour(self._tree.root( ), 0, [ ])          # start the recursion
18
19   def _tour(self, p, d, path):
20     """Perform tour of subtree rooted at Position p.
21
22     p          Position of current node being visited
23     d          depth of p in the tree
24     path       list of indices of children on path from root to p
25     """
26     self._hook_previsit(p, d, path)                          # "pre visit" p
27     results = [ ]
28     path.append(0)             # add new index to end of path before recursion
29     for c in self._tree.children(p):
30       results.append(self._tour(c, d+1, path))      # recur on child's subtree
31       path[-1] += 1             # increment index
32     path.pop( )                 # remove extraneous index from end of path
33     answer = self._hook_postvisit(p, d, path, results)       # "post visit" p
34     return answer
35
36   def _hook_previsit(self, p, d, path):                      # can be overridden
37     pass
38
39   def _hook_postvisit(self, p, d, path, results):            # can be overridden
40     pass
```

**Code Fragment 8.28:** An EulerTour base class providing a framework for performing Euler tour traversals of a tree.

Based on our experience of customizing traversals for sample applications Section 8.4.5, we build support into the primary EulerTour for maintaining the recursive depth and the representation of the recursive path through a tree, using the approach that we introduced in Code Fragment 8.24. We also provide a mechanism for one recursive level to return a value to another when post-processing. Formally, our framework relies on the following two hooks that can be specialized:

- method _hook_previsit(p, d, path)
  This function is called once for each position, immediately before its subtrees (if any) are traversed. Parameter p is a position in the tree, d is the depth of that position, and path is a list of indices, using the convention described in the discussion of Code Fragment 8.24. No return value is expected from this function.
- method _hook_postvisit(p, d, path, results)
  This function is called once for each position, immediately after its subtrees (if any) are traversed. The first three parameters use the same convention as did _hook_previsit. The final parameter is a list of objects that were provided as return values from the post visits of the respective subtrees of p. Any value returned by this call will be available to the parent of p during its postvisit.

For more complex tasks, subclasses of EulerTour may also choose to initialize and maintain additional state in the form of instance variables that can be accessed within the bodies of the hooks.

### Using the Euler Tour Framework

To demonstrate the flexibility of our Euler tour framework, we revisit the sample applications from Section 8.4.5. As a simple example, an indented preorder traversal, akin to that originally produced by Code Fragment 8.23, can be generated with the simple subclass given in Code Fragment 8.29.

```
1 class PreorderPrintIndentedTour(EulerTour):
2   def _hook_previsit(self, p, d, path):
3     print(2*d*' ' + str(p.element( )))
```

**Code Fragment 8.29:** A subclass of EulerTour that produces an indented preorder list of a tree's elements.

Such a tour would be started by creating an instance of the subclass for a given tree T, and invoking its execute method. This could be expressed as follows:

```
tour = PreorderPrintIndentedTour(T)
tour.execute( )
```

A labeled version of an indented, preorder presentation, akin to Code Fragment 8.24, could be generated by the new subclass of EulerTour shown in Code Fragment 8.30.

```
1  class PreorderPrintIndentedLabeledTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      label = '.'.join(str(j+1) for j in path)    # labels are one-indexed
4      print(2*d*' ' + label, p.element( ))
```

**Code Fragment 8.30:** A subclass of EulerTour that produces a labeled and indented, preorder list of a tree's elements.

To produce the parenthetic string representation, originally achieved with Code Fragment 8.25, we define a subclass that overrides both the previsit and postvisit hooks. Our new implementation is given in Code Fragment 8.31.

```
1  class ParenthesizeTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      if path and path[-1] > 0:                   # p follows a sibling
4        print(', ', end='')                       # so preface with comma
5      print(p.element( ), end='')                 # then print element
6      if not self.tree( ).is_leaf(p):             # if p has children
7        print(' (', end='')                       # print opening parenthesis
8
9    def _hook_postvisit(self, p, d, path, results):
10     if not self.tree( ).is_leaf(p):             # if p has children
11       print(')', end='')                        # print closing parenthesis
```

**Code Fragment 8.31:** A subclass of EulerTour that prints a parenthetic string representation of a tree.

Notice that in this implementation, we need to invoke a method on the tree instance that is being traversed from within the hooks. The public tree( ) method of the EulerTour class serves as an accessor for that tree.

Finally, the task of computing disk space, as originally implemented in Code Fragment 8.26, can be performed quite easily with the EulerTour subclass shown in Code Fragment 8.32. The postvisit result of the root will be returned by the call to execute( ).

```
1  class DiskSpaceTour(EulerTour):
2    def _hook_postvisit(self, p, d, path, results):
3      # we simply add space associated with p to that of its subtrees
4      return p.element( ).space( ) + sum(results)
```

**Code Fragment 8.32:** A subclass of EulerTour that computes disk space for a tree.

## The Euler Tour Traversal of a Binary Tree

In Section 8.4.6, we introduced the concept of an Euler tour traversal of a general graph, using the template method pattern in designing the EulerTour class. That class provided methods _hook_previsit and _hook_postvisit that could be overridden to customize a tour. In Code Fragment 8.33 we provide a BinaryEulerTour specialization that includes an additional _hook_invisit that is called once for each position—after its left subtree is traversed, but before its right subtree is traversed.

Our implementation of BinaryEulerTour replaces the original _tour utility to specialize to the case in which a node has at most two children. If a node has only one child, a tour differentiates between whether that is a left child or a right child, with the "in visit" taking place after the visit of a sole left child, but before the visit of a sole right child. In the case of a leaf, the three hooks are called in succession.

```
1  class BinaryEulerTour(EulerTour):
2    """Abstract base class for performing Euler tour of a binary tree.
3
4    This version includes an additional _hook_invisit that is called after the tour
5    of the left subtree (if any), yet before the tour of the right subtree (if any).
6
7    Note: Right child is always assigned index 1 in path, even if no left sibling.
8    """
9    def _tour(self, p, d, path):
10     results = [None, None]          # will update with results of recursions
11     self._hook_previsit(p, d, path)                    # "pre visit" for p
12     if self._tree.left(p) is not None:                 # consider left child
13       path.append(0)
14       results[0] = self._tour(self._tree.left(p), d+1, path)
15       path.pop( )
16     self._hook_invisit(p, d, path)                     # "in visit" for p
17     if self._tree.right(p) is not None:                # consider right child
18       path.append(1)
19       results[1] = self._tour(self._tree.right(p), d+1, path)
20       path.pop( )
21     answer = self._hook_postvisit(p, d, path, results)       # "post visit" p
22     return answer
23
24   def _hook_invisit(self, p, d, path): pass            # can be overridden
```

**Code Fragment 8.33:** A BinaryEulerTour base class providing a specialized tour for binary trees. The original EulerTour base class was given in Code Fragment 8.28.

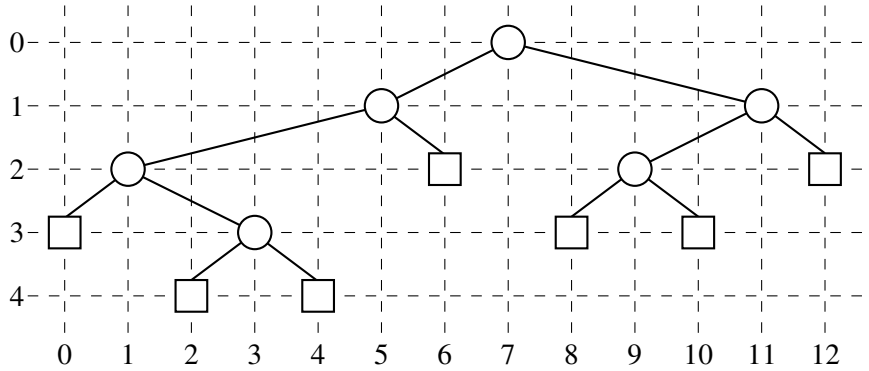



**Figure 8.22:** An inorder drawing of a binary tree.

To demonstrate use of the BinaryEulerTour framework, we develop a subclass that computes a graphical layout of a binary tree, as shown in Figure 8.22. The geometry is determined by an algorithm that assigns $x$- and $y$-coordinates to each position $p$ of a binary tree $T$ using the following two rules:

- $x(p)$ is the number of positions visited before $p$ in an inorder traversal of $T$.
- $y(p)$ is the depth of $p$ in $T$.

In this application, we take the convention common in computer graphics that $x$-coordinates increase left to right and $y$-coordinates increase top to bottom. So the origin is in the upper left corner of the computer screen.

Code Fragment 8.34 provides an implementation of a BinaryLayout subclass that implements the above algorithm for assigning $(x,y)$ coordinates to the element stored at each position of a binary tree. We adapt the BinaryEulerTour framework by introducing additional state in the form of a _count instance variable that represents the number of "in visits" that we have performed. The $x$-coordinate for each position is set according to that counter.

```
1  class BinaryLayout(BinaryEulerTour):
2    """Class for computing (x,y) coordinates for each node of a binary tree."""
3    def __init__(self, tree):
4      super().__init__(tree)                # must call the parent constructor
5      self._count = 0                       # initialize count of processed nodes
6
7    def _hook_invisit(self, p, d, path):
8      p.element().setX(self._count)         # x-coordinate serialized by count
9      p.element().setY(d)                   # y-coordinate is depth
10     self._count += 1                      # advance count of processed nodes
```

**Code Fragment 8.34:** A BinaryLayout class that computes coordinates at which to draw positions of a binary tree. We assume that the element type for the original tree supports setX and setY methods.

## 8.5 Case Study: An Expression Tree

In Example 8.7, we introduced the use of a binary tree to represent the structure of an arithmetic expression. In this section, we define a new ExpressionTree class that provides support for constructing such trees, and for displaying and evaluating the arithmetic expression that such a tree represents. Our ExpressionTree class is defined as a subclass of LinkedBinaryTree, and we rely on the nonpublic mutators to construct such trees. Each internal node must store a string that defines a binary operator (e.g., `'+'`), and each leaf must store a numeric value (or a string representing a numeric value).

Our eventual goal is to build arbitrarily complex expression trees for compound arithmetic expressions such as $(((3+1)\times 4)/((9-5)+2))$. However, it suffices for the ExpressionTree class to support two basic forms of initialization:

**ExpressionTree(value):** Create a tree storing the given value at the root.

**ExpressionTree(op, $E_1$, $E_2$):** Create a tree storing string op at the root (e.g., $+$), and with the structures of existing ExpressionTree instances $E_1$ and $E_2$ as the left and right subtrees of the root, respectively.

Such a constructor for the ExpressionTree class is given in Code Fragment 8.35. The class formally inherits from LinkedBinaryTree, so it has access to all the nonpublic update methods that were defined in Section 8.3.1. We use `_add_root` to create an initial root of the tree storing the token provided as the first parameter. Then we perform run-time checking of the parameters to determine whether the caller invoked the one-parameter version of the constructor (in which case, we are done), or the three-parameter form. In that case, we use the inherited `_attach` method to incorporate the structure of the existing trees as subtrees of the root.

### Composing a Parenthesized String Representation

A string representation of an existing expression tree instance, for example, as `'(((3+1)x4)/((9-5)+2))'`, can be produced by displaying tree elements using an inorder traversal, but with opening and closing parentheses inserted with a preorder and postorder step, respectively. In the context of an ExpressionTree class, we support a special `__str__` method (see Section 2.3.2) that returns the appropriate string. Because it is more efficient to first build a sequence of individual strings to be joined together (see discussion of "Composing Strings" in Section 5.4.2), the implementation of `__str__` relies on a nonpublic, recursive method named `_parenthesize_recur` that appends a series of strings to a list. These methods are included in Code 8.35.

```
class ExpressionTree(LinkedBinaryTree):
  """An arithmetic expression tree."""

  def __init__(self, token, left=None, right=None):
    """Create an expression tree.

    In a single parameter form, token should be a leaf value (e.g., '42'),
    and the expression tree will have that value at an isolated node.

    In a three-parameter version, token should be an operator,
    and left and right should be existing ExpressionTree instances
    that become the operands for the binary operator.
    """
    super().__init__( )                          # LinkedBinaryTree initialization
    if not isinstance(token, str):
      raise TypeError('Token must be a string')
    self._add_root(token)                        # use inherited, nonpublic method
    if left is not None:                         # presumably three-parameter form
      if token not in '+-*x/':
        raise ValueError('token must be valid operator')
      self._attach(self.root( ), left, right)    # use inherited, nonpublic method

  def __str__(self):
    """Return string representation of the expression."""
    pieces = [ ]                                 # sequence of piecewise strings to compose
    self._parenthesize_recur(self.root( ), pieces)
    return ''.join(pieces)

  def _parenthesize_recur(self, p, result):
    """Append piecewise representation of p's subtree to resulting list."""
    if self.is_leaf(p):
      result.append(str(p.element( )))                    # leaf value as a string
    else:
      result.append('(')                                  # opening parenthesis
      self._parenthesize_recur(self.left(p), result)      # left subtree
      result.append(p.element( ))                         # operator
      self._parenthesize_recur(self.right(p), result)     # right subtree
      result.append(')')                                  # closing parenthesis
```

**Code Fragment 8.35:** The beginning of an ExpressionTree class.

### Expression Tree Evaluation

The numeric evaluation of an expression tree can be accomplished with a simple application of a postorder traversal. If we know the values represented by the two subtrees of an internal position, we can calculate the result of the computation that position designates. Pseudo-code for the recursive evaluation of the value represented by a subtree rooted at position $p$ is given in Code Fragment 8.36.

```
Algorithm evaluate_recur(p):
  if p is a leaf then
    return the value stored at p
  else
    let ∘ be the operator stored at p
    x = evaluate_recur(left(p))
    y = evaluate_recur(right(p))
    return x ∘ y
```

**Code Fragment 8.36:** Algorithm evaluate_recur for evaluating the expression represented by a subtree of an arithmetic expression tree rooted at position p.

To implement this algorithm in the context of a Python ExpressionTree class, we provide a public evaluate method that is invoked on instance T as T.evaluate( ). Code Fragment 8.37 provides such an implementation, relying on a nonpublic _evaluate_recur method that computes the value of a designated subtree.

```
39  def evaluate(self):
40    """Return the numeric result of the expression."""
41    return self._evaluate_recur(self.root( ))
42
43  def _evaluate_recur(self, p):
44    """Return the numeric result of subtree rooted at p."""
45    if self.is_leaf(p):
46      return float(p.element( ))          # we assume element is numeric
47    else:
48      op = p.element( )
49      left_val = self._evaluate_recur(self.left(p))
50      right_val = self._evaluate_recur(self.right(p))
51      if op == '+': return left_val + right_val
52      elif op == '-': return left_val - right_val
53      elif op == '/': return left_val / right_val
54      else: return left_val * right_val       # treat 'x' or '*' as multiplication
```

**Code Fragment 8.37:** Support for evaluating an ExpressionTree instance.

### Building an Expression Tree

The constructor for the ExpressionTree class, from Code Fragment 8.35, provides basic functionality for combining existing trees to build larger expression trees. However, the question still remains how to construct a tree that represents an expression for a given string, such as `'(((3+1)x4)/((9-5)+2))'`.

To automate this process, we rely on a bottom-up construction algorithm, assuming that a string can first be tokenized so that multidigit numbers are treated atomically (see Exercise R-8.30), and that the expression is fully parenthesized. The algorithm uses a stack $S$ while scanning tokens of the input expression $E$ to find values, operators, and right parentheses. (Left parentheses are ignored.)

- When we see an operator $\circ$, we push that string on the stack.
- When we see a literal value $v$, we create a single-node expression tree $T$ storing $v$, and push $T$ on the stack.
- When we see a right parenthesis, `')'`, we pop the top three items from the stack $S$, which represent a subexpression $(E_1 \circ E_2)$. We then construct a tree $T$ using trees for $E_1$ and $E_2$ as subtrees of the root storing $\circ$, and push the resulting tree $T$ back on the stack.

We repeat this until the expression $E$ has been processed, at which time the top element on the stack is the expression tree for $E$. The total running time is $O(n)$.

An implementation of this algorithm is given in Code Fragment 8.38 in the form of a stand-alone function named build_expression_tree, which produces and returns an appropriate ExpressionTree instance, assuming the input has been tokenized.

```
1  def build_expression_tree(tokens):
2    """Returns an ExpressionTree based upon by a tokenized expression."""
3    S = [ ]                                    # we use Python list as stack
4    for t in tokens:
5      if t in '+-x*/':                         # t is an operator symbol
6        S.append(t)                            # push the operator symbol
7      elif t not in '()':                      # consider t to be a literal
8        S.append(ExpressionTree(t))            # push trivial tree storing value
9      elif t == ')':          # compose a new tree from three constituent parts
10       right = S.pop( )                       # right subtree as per LIFO
11       op = S.pop( )                          # operator symbol
12       left = S.pop( )                        # left subtree
13       S.append(ExpressionTree(op, left, right)) # repush tree
14     # we ignore a left parenthesis
15   return S.pop( )
```

**Code Fragment 8.38:** Implementation of a build_expression_tree that produces an ExpressionTree from a sequence of tokens representing an arithmetic expression.

## 8.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

### Reinforcement

**R-8.1** The following questions refer to the tree of Figure 8.3.
a. Which node is the root?
b. What are the internal nodes?
c. How many descendants does node cs016/ have?
d. How many ancestors does node cs016/ have?
e. What are the siblings of node homeworks/?
f. Which nodes are in the subtree rooted at node projects/?
g. What is the depth of node papers/?
h. What is the height of the tree?

**R-8.2** Show a tree achieving the worst-case running time for algorithm depth.

**R-8.3** Give a justification of Proposition 8.4.

**R-8.4** What is the running time of a call to T._height2(p) when called on a position p distinct from the root of T? (See Code Fragment 8.5.)

**R-8.5** Describe an algorithm, relying only on the BinaryTree operations, that counts the number of leaves in a binary tree that are the *left* child of their respective parent.

**R-8.6** Let $T$ be an $n$-node binary tree that may be improper. Describe how to represent $T$ by means of a ***proper*** binary tree $T'$ with $O(n)$ nodes.

**R-8.7** What are the minimum and maximum number of internal and external nodes in an improper binary tree with $n$ nodes?

**R-8.8** Answer the following questions so as to justify Proposition 8.8.
a. What is the minimum number of external nodes for a proper binary tree with height $h$? Justify your answer.
b. What is the maximum number of external nodes for a proper binary tree with height $h$? Justify your answer.
c. Let $T$ be a proper binary tree with height $h$ and $n$ nodes. Show that

$$\log(n+1) - 1 \leq h \leq (n-1)/2.$$

d. For which values of $n$ and $h$ can the above lower and upper bounds on $h$ be attained with equality?

**R-8.9** Give a proof by induction of Proposition 8.9.

**R-8.10** Give a direct implementation of the num_children method within the class BinaryTree.

**R-8.11** Find the value of the arithmetic expression associated with each subtree of the binary tree of Figure 8.8.

**R-8.12** Draw an arithmetic expression tree that has four external nodes, storing the numbers 1, 5, 6, and 7 (with each number stored in a distinct external node, but not necessarily in this order), and has three internal nodes, each storing an operator from the set $\{+, -, \times, /\}$, so that the value of the root is 21. The operators may return and act on fractions, and an operator may be used more than once.

**R-8.13** Draw the binary tree representation of the following arithmetic expression: "$(((5+2)*(2-1))/((2+9)+((7-2)-1))*8)$".

**R-8.14** Justify Table 8.2, summarizing the running time of the methods of a tree represented with a linked structure, by providing, for each method, a description of its implementation, and an analysis of its running time.

**R-8.15** The LinkedBinaryTree class provides only nonpublic versions of the update methods discussed on page 319. Implement a simple subclass named MutableLinkedBinaryTree that provides public wrapper functions for each of the inherited nonpublic update methods.

**R-8.16** Let $T$ be a binary tree with $n$ nodes, and let $f()$ be the level numbering function of the positions of $T$, as given in Section 8.3.2.

a. Show that, for every position $p$ of $T$, $f(p) \leq 2^n - 2$.
b. Show an example of a binary tree with seven nodes that attains the above upper bound on $f(p)$ for some position $p$.

**R-8.17** Show how to use the Euler tour traversal to compute the level number $f(p)$, as defined in Section 8.3.2, of each position in a binary tree $T$.

**R-8.18** Let $T$ be a binary tree with $n$ positions that is realized with an array representation $A$, and let $f()$ be the level numbering function of the positions of $T$, as given in Section 8.3.2. Give pseudo-code descriptions of each of the methods root, parent, left, right, is_leaf, and is_root.

**R-8.19** Our definition of the level numbering function $f(p)$, as given in Section 8.3.2, began with the root having number 0. Some authors prefer to use a level numbering $g(p)$ in which the root is assigned number 1, because it simplifies the arithmetic for finding neighboring positions. Redo Exercise R-8.18, but assuming that we use a level numbering $g(p)$ in which the root is assigned number 1.

**R-8.20** Draw a binary tree $T$ that simultaneously satisfies the following:

- Each internal node of $T$ stores a single character.
- A *preorder* traversal of $T$ yields EXAMFUN.
- An *inorder* traversal of $T$ yields MAFXUEN.

**R-8.21** In what order are positions visited during a preorder traversal of the tree of Figure 8.8?

**R-8.22** In what order are positions visited during a postorder traversal of the tree of Figure 8.8?

**R-8.23** Let $T$ be an ordered tree with more than one node. Is it possible that the preorder traversal of $T$ visits the nodes in the same order as the postorder traversal of $T$? If so, give an example; otherwise, explain why this cannot occur. Likewise, is it possible that the preorder traversal of $T$ visits the nodes in the reverse order of the postorder traversal of $T$? If so, give an example; otherwise, explain why this cannot occur.

**R-8.24** Answer the previous question for the case when $T$ is a proper binary tree with more than one node.

**R-8.25** Consider the example of a breadth-first traversal given in Figure 8.17. Using the annotated numbers from that figure, describe the contents of the queue before each pass of the while loop in Code Fragment 8.14. To get started, the queue has contents $\{1\}$ before the first pass, and contents $\{2,3,4\}$ before the second pass.

**R-8.26** The collections.deque class supports an extend method that adds a collection of elements to the end of the queue at once. Reimplement the breadthfirst method of the Tree class to take advantage of this feature.

**R-8.27** Give the output of the function parenthesize(T, T.root()), as described in Code Fragment 8.25, when $T$ is the tree of Figure 8.8.

**R-8.28** What is the running time of parenthesize(T, T.root()), as given in Code Fragment 8.25, for a tree $T$ with $n$ nodes?

**R-8.29** Describe, in pseudo-code, an algorithm for computing the number of descendants of each node of a binary tree. The algorithm should be based on the Euler tour traversal.

**R-8.30** The build_expression_tree method of the ExpressionTree class requires input that is an iterable of string tokens. We used a convenient example, `'(((3+1)x4)/((9-5)+2))'`, in which each character is its own token, so that the string itself sufficed as input to build_expression_tree. In general, a string, such as `'(35 + 14)'`, must be explicitly tokenized into list `['(', '35', '+', '14', ')']` so as to ignore whitespace and to recognize multidigit numbers as a single token. Write a utility method, tokenize(raw), that returns such a list of tokens for a raw string.

## Creativity

**C-8.31** Define the ***internal path length***, $I(T)$, of a tree $T$ to be the sum of the depths of all the internal positions in $T$. Likewise, define the ***external path length***, $E(T)$, of a tree $T$ to be the sum of the depths of all the external positions in $T$. Show that if $T$ is a proper binary tree with $n$ positions, then $E(T) = I(T) + n - 1$.

**C-8.32** Let $T$ be a (not necessarily proper) binary tree with $n$ nodes, and let $D$ be the sum of the depths of all the external nodes of $T$. Show that if $T$ has the minimum number of external nodes possible, then $D$ is $O(n)$ and if $T$ has the maximum number of external nodes possible, then $D$ is $O(n \log n)$.

**C-8.33** Let $T$ be a (possibly improper) binary tree with $n$ nodes, and let $D$ be the sum of the depths of all the external nodes of $T$. Describe a configuration for $T$ such that $D$ is $\Omega(n^2)$. Such a tree would be the worst case for the asymptotic running time of method _height1 (Code Fragment 8.4).

**C-8.34** For a tree $T$, let $n_I$ denote the number of its internal nodes, and let $n_E$ denote the number of its external nodes. Show that if every internal node in $T$ has exactly 3 children, then $n_E = 2n_I + 1$.

**C-8.35** Two ordered trees $T'$ and $T''$ are said to be ***isomorphic*** if one of the following holds:

- Both $T'$ and $T''$ are empty.
- The roots of $T'$ and $T''$ have the same number $k \geq 0$ of subtrees, and the $i^{th}$ such subtree of $T'$ is isomorphic to the $i^{th}$ such subtree of $T''$ for $i = 1, \ldots, k$.

Design an algorithm that tests whether two given ordered trees are isomorphic. What is the running time of your algorithm?

**C-8.36** Show that there are more than $2^n$ improper binary trees with $n$ internal nodes such that no pair are isomorphic (see Exercise C-8.35).

**C-8.37** If we exclude isomorphic trees (see Exercise C-8.35), exactly how many proper binary trees exist with exactly 4 leaves?

**C-8.38** Add support in LinkedBinaryTree for a method, _delete_subtree(p), that removes the entire subtree rooted at position $p$, making sure to maintain the count on the size of the tree. What is the running time of your implementation?

**C-8.39** Add support in LinkedBinaryTree for a method, _swap(p,q), that has the effect of restructuring the tree so that the node referenced by p takes the place of the node referenced by q, and vice versa. Make sure to properly handle the case when the nodes are adjacent.

**C-8.40** We can simplify parts of our LinkedBinaryTree implementation if we make use of of a single sentinel node, referenced as the _sentinel member of the tree instance, such that the sentinel is the parent of the real root of the tree, and the root is referenced as the left child of the sentinel. Furthermore, the sentinel will take the place of None as the value of the _left or _right member for a node without such a child. Give a new implementation of the update methods _delete and _attach, assuming such a representation.

**C-8.41** Describe how to clone a LinkedBinaryTree instance representing a proper binary tree, with use of the _attach method.

**C-8.42** Describe how to clone a LinkedBinaryTree instance representing a (not necessarily proper) binary tree, with use of the _add_left and add_right methods.

**C-8.43** We can define a ***binary tree representation*** $T'$ for an ordered general tree $T$ as follows (see Figure 8.23):

- For each position $p$ of $T$, there is an associated position $p'$ of $T'$.
- If $p$ is a leaf of $T$, then $p'$ in $T'$ does not have a left child; otherwise the left child of $p'$ is $q'$, where $q$ is the first child of $p$ in $T$.
- If $p$ has a sibling $q$ ordered immediately after it in $T$, then $q'$ is the right child of $p'$ in $T$; otherwise $p'$ does not have a right child.

Given such a representation $T'$ of a general ordered tree $T$, answer each of the following questions:

a. Is a preorder traversal of $T'$ equivalent to a preorder traversal of $T$?
b. Is a postorder traversal of $T'$ equivalent to a postorder traversal of $T$?
c. Is an inorder traversal of $T'$ equivalent to one of the standard traversals of $T$? If so, which one?

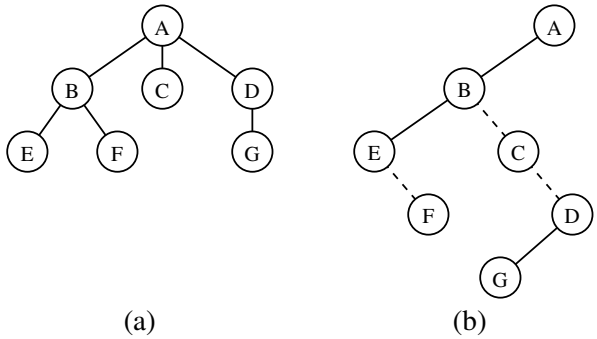



**Figure 8.23:** Representation of a tree with a binary tree: (a) tree $T$; (b) binary tree $T'$ for $T$. The dashed edges connect nodes of $T'$ that are siblings in $T$.

**C-8.44** Give an efficient algorithm that computes and prints, for every position $p$ of a tree $T$, the element of $p$ followed by the height of $p$'s subtree.

**C-8.45** Give an $O(n)$-time algorithm for computing the depths of all positions of a tree $T$, where $n$ is the number of nodes of $T$.

**C-8.46** The ***path length*** of a tree $T$ is the sum of the depths of all positions in $T$. Describe a linear-time method for computing the path length of a tree $T$.

**C-8.47** The ***balance factor*** of an internal position $p$ of a proper binary tree is the difference between the heights of the right and left subtrees of $p$. Show how to specialize the Euler tour traversal of Section 8.4.6 to print the balance factors of all the internal nodes of a proper binary tree.

**C-8.48** Given a proper binary tree $T$, define the ***reflection*** of $T$ to be the binary tree $T'$ such that each node $v$ in $T$ is also in $T'$, but the left child of $v$ in $T$ is $v$'s right child in $T'$ and the right child of $v$ in $T$ is $v$'s left child in $T'$. Show that a preorder traversal of a proper binary tree $T$ is the same as the postorder traversal of $T$'s reflection, but in reverse order.

**C-8.49** Let the ***rank*** of a position $p$ during a traversal be defined such that the first element visited has rank 1, the second element visited has rank 2, and so on. For each position $p$ in a tree $T$, let $\mathsf{pre}(p)$ be the rank of $p$ in a preorder traversal of $T$, let $\mathsf{post}(p)$ be the rank of $p$ in a postorder traversal of $T$, let $\mathsf{depth}(p)$ be the depth of $p$, and let $\mathsf{desc}(p)$ be the number of descendants of $p$, including $p$ itself. Derive a formula defining $\mathsf{post}(p)$ in terms of $\mathsf{desc}(p)$, $\mathsf{depth}(p)$, and $\mathsf{pre}(p)$, for each node $p$ in $T$.

**C-8.50** Design algorithms for the following operations for a binary tree $T$:

- preorder_next(p): Return the position visited after $p$ in a preorder traversal of $T$ (or None if $p$ is the last node visited).
- inorder_next(p): Return the position visited after $p$ in an inorder traversal of $T$ (or None if $p$ is the last node visited).
- postorder_next(p): Return the position visited after $p$ in a postorder traversal of $T$ (or None if $p$ is the last node visited).

What are the worst-case running times of your algorithms?

**C-8.51** To implement the preorder method of the LinkedBinaryTree class, we relied on the convenience of Python's generator syntax and the yield statement. Give an alternative implementation of preorder that returns an explicit instance of a nested iterator class. (See Section 2.3.4 for discussion of iterators.)

**C-8.52** Algorithm preorder_draw draws a binary tree $T$ by assigning $x$- and $y$-coordinates to each position $p$ such that $x(p)$ is the number of nodes preceding $p$ in the preorder traversal of $T$ and $y(p)$ is the depth of $p$ in $T$.

a. Show that the drawing of $T$ produced by preorder_draw has no pairs of crossing edges.
b. Redraw the binary tree of Figure 8.22 using preorder_draw.

**C-8.53** Redo the previous problem for the algorithm postorder_draw that is similar to preorder_draw except that it assigns $x(p)$ to be the number of nodes preceding position $p$ in the postorder traversal.

**C-8.54** Design an algorithm for drawing *general* trees, using a style similar to the inorder traversal approach for drawing binary trees.

**C-8.55** Exercise P-4.27 described the walk function of the os module. This function performs a traversal of the implicit tree represented by the file system. Read the formal documentation for the function, and in particular its use of an optional Boolean parameter named topdown. Describe how its behavior relates to tree traversal algorithms described in this chapter.

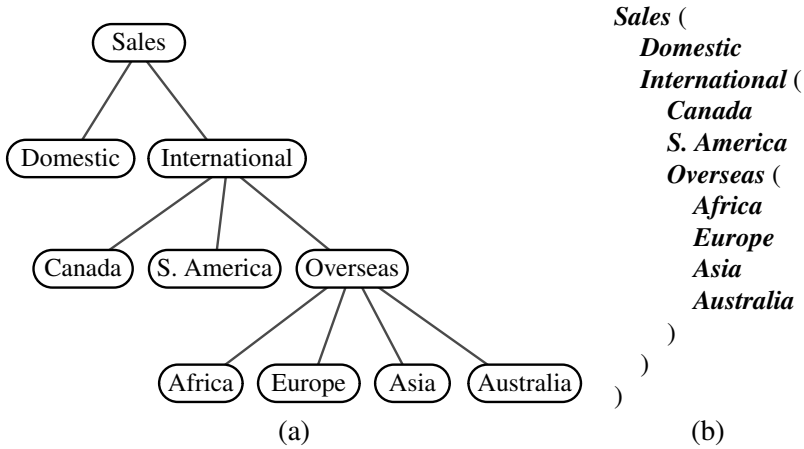



**Figure 8.24:** (a) Tree $T$; (b) indented parenthetic representation of $T$.

**C-8.56** The ***indented parenthetic representation*** of a tree $T$ is a variation of the parenthetic representation of $T$ (see Code Fragment 8.25) that uses indentation and line breaks as illustrated in Figure 8.24. Give an algorithm that prints this representation of a tree.

**C-8.57** Let $T$ be a binary tree with $n$ positions. Define a ***Roman position*** to be a position $p$ in $T$, such that the number of descendants in $p$'s left subtree differ from the number of descendants in $p$'s right subtree by at most 5. Describe a linear-time method for finding each position $p$ of $T$, such that $p$ is not a Roman position, but all of $p$'s descendants are Roman.

**C-8.58** Let $T$ be a tree with $n$ positions. Define the ***lowest common ancestor*** (LCA) between two positions $p$ and $q$ as the lowest position in $T$ that has both $p$ and $q$ as descendants (where we allow a position to be a descendant of itself). Given two positions $p$ and $q$, describe an efficient algorithm for finding the LCA of $p$ and $q$. What is the running time of your algorithm?

**C-8.59** Let $T$ be a binary tree with $n$ positions, and, for any position $p$ in $T$, let $d_p$ denote the depth of $p$ in $T$. The ***distance*** between two positions $p$ and $q$ in $T$ is $d_p + d_q - 2d_a$, where $a$ is the lowest common ancestor (LCA) of $p$ and $q$. The ***diameter*** of $T$ is the maximum distance between two positions in $T$. Describe an efficient algorithm for finding the diameter of $T$. What is the running time of your algorithm?

**C-8.60** Suppose each position $p$ of a binary tree $T$ is labeled with its value $f(p)$ in a level numbering of $T$. Design a fast method for determining $f(a)$ for the lowest common ancestor (LCA), $a$, of two positions $p$ and $q$ in $T$, given $f(p)$ and $f(q)$. You do not need to find position $a$, just value $f(a)$.

**C-8.61** Give an alternative implementation of the build_expression_tree method of the ExpressionTree class that relies on recursion to perform an implicit Euler tour of the tree that is being built.

**C-8.62** Note that the build_expression_tree function of the ExpressionTree class is written in such a way that a leaf token can be any string; for example, it parses the expression `'(a*(b+c))'`. However, within the evaluate method, an error would occur when attempting to convert a leaf token to a number. Modify the evaluate method to accept an optional Python dictionary that can be used to map such string variables to numeric values, with a syntax such as T.evaluate({`'a'`:3, `'b'`:1, `'c'`:5}). In this way, the same algebraic expression can be evaluated using different values.

**C-8.63** As mentioned in Exercise C-6.22, ***postfix notation*** is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if "$(exp_1)\,\mathbf{op}\,(exp_2)$" is a normal (infix) fully parenthesized expression with operation **op**, then its postfix equivalent is "$pexp_1\ pexp_2\ \mathbf{op}$", where $pexp_1$ is the postfix version of $exp_1$ and $pexp_2$ is the postfix version of $exp_2$. The postfix version of a single number or variable is just that number or variable. So, for example, the postfix version of the infix expression "$((5+2)*(8-3))/4$" is "$5\ 2\ +\ 8\ 3\ -\ *\ 4\ /$". Implement a postfix method of the ExpressionTree class of Section 8.5 that produces the postfix notation for the given expression.

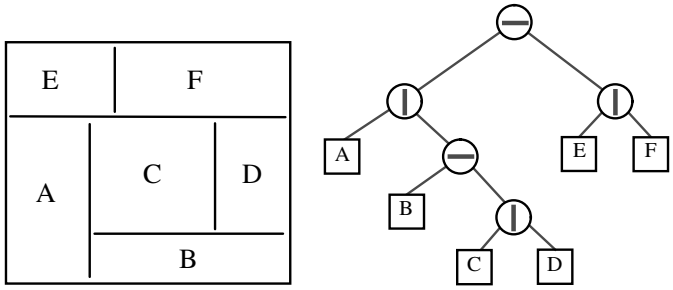
## Projects

**P-8.64** Implement the binary tree ADT using the array-based representation described in Section 8.3.2.

**P-8.65** Implement the tree ADT using a linked structure as described in Section 8.3.3. Provide a reasonable set of update methods for your tree.

**P-8.66** The memory usage for the LinkedBinaryTree class can be streamlined by removing the parent reference from each node, and instead having each Position instance keep a member, _path, that is a list of nodes representing the entire path from the root to that position. (This generally saves memory because there are typically relatively few stored position instances.) Reimplement the LinkedBinaryTree class using this strategy.

**P-8.67** A ***slicing floor plan*** divides a rectangle with horizontal and vertical sides using horizontal and vertical ***cuts***. (See Figure 8.25a.) A slicing floor plan can be represented by a proper binary tree, called a ***slicing tree***, whose internal nodes represent the cuts, and whose external nodes represent the ***basic rectangles*** into which the floor plan is decomposed by the cuts. (See Figure 8.25b.) The ***compaction problem*** for a slicing floor plan is defined as follows. Assume that each basic rectangle of a slicing floor plan is assigned a minimum width $w$ and a minimum height $h$. The compaction problem is to find the smallest possible height and width for each rectangle of the slicing floor plan that is compatible with the minimum dimensions




(a) (b)

**Figure 8.25:** (a) Slicing floor plan; (b) slicing tree associated with the floor plan.

of the basic rectangles. Namely, this problem requires the assignment of values $\mathsf{h}(p)$ and $\mathsf{w}(p)$ to each position $p$ of the slicing tree such that:

$$\mathsf{w}(p) = \begin{cases} w & \text{if } p \text{ is a leaf whose basic rectangle has minimum width } w \\ \max(\mathsf{w}(\ell), \mathsf{w}(r)) & \text{if } p \text{ is an internal position, associated with a horizontal cut, with left child } \ell \text{ and right child } r \\ \mathsf{w}(\ell) + \mathsf{w}(r) & \text{if } p \text{ is an internal position, associated with a vertical cut, with left child } \ell \text{ and right child } r \end{cases}$$

$$\mathsf{h}(p) = \begin{cases} h & \text{if } p \text{ is a leaf node whose basic rectangle has minimum height } h \\ \mathsf{h}(\ell) + \mathsf{h}(r) & \text{if } p \text{ is an internal position, associated with a horizontal cut, with left child } \ell \text{ and right child } r \\ \max(\mathsf{h}(\ell), \mathsf{h}(r)) & \text{if } p \text{ is an internal position, associated with a vertical cut, with left child } \ell \text{ and right child } r \end{cases}$$

Design a data structure for slicing floor plans that supports the operations:

- Create a floor plan consisting of a single basic rectangle.
- Decompose a basic rectangle by means of a horizontal cut.
- Decompose a basic rectangle by means of a vertical cut.
- Assign minimum height and width to a basic rectangle.
- Draw the slicing tree associated with the floor plan.
- Compact and draw the floor plan.

**P-8.68** Write a program that can play Tic-Tac-Toe effectively. (See Section 5.6.) To do this, you will need to create a ***game tree*** $T$, which is a tree where each position corresponds to a ***game configuration***, which, in this case, is a representation of the Tic-Tac-Toe board. (See Section 8.4.2.) The root corresponds to the initial configuration. For each internal position $p$ in $T$, the children of $p$ correspond to the game states we can reach from $p$'s game state in a single legal move for the appropriate player, $A$ (the first player) or $B$ (the second player). Positions at even depths correspond to moves for $A$ and positions at odd depths correspond to moves for $B$. Leaves are either final game states or are at a depth beyond which we do not want to explore. We score each leaf with a value that indicates how good this state is for player $A$. In large games, like chess, we have to use a heuristic scoring function, but for small games, like Tic-Tac-Toe, we can construct the entire game tree and score leaves as $+1$, $0$, $-1$, indicating whether player $A$ has a win, draw, or lose in that configuration. A good algorithm for choosing moves is ***minimax***. In this algorithm, we assign a score to each internal position $p$ in $T$, such that if $p$ represents $A$'s turn, we compute $p$'s score as the maximum of the scores of $p$'s children (which corresponds to $A$'s optimal play from $p$). If an internal node $p$ represents $B$'s turn, then we compute $p$'s score as the minimum of the scores of $p$'s children (which corresponds to $B$'s optimal play from $p$).

**P-8.69** Implement the tree ADT using the binary tree representation described in Exercise C-8.43. You may adapt the LinkedBinaryTree implementation.

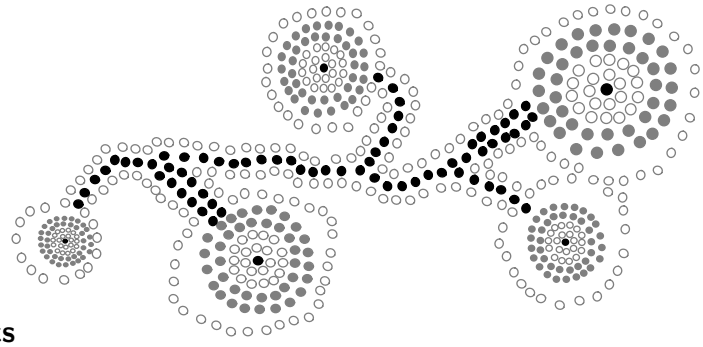
**P-8.70** Write a program that takes as input a general tree $T$ and a position $p$ of $T$ and converts $T$ to another tree with the same set of position adjacencies, but now with $p$ as its root.

# Chapter Notes

Discussions of the classic preorder, inorder, and postorder tree traversal methods can be found in Knuth's *Fundamental Algorithms* book [64]. The Euler tour traversal technique comes from the parallel algorithms community; it is introduced by Tarjan and Vishkin [93] and is discussed by JáJá [54] and by Karp and Ramachandran [58]. The algorithm for drawing a tree is generally considered to be a part of the "folklore" of graph-drawing algorithms. The reader interested in graph drawing is referred to the book by Di Battista, Eades, Tamassia, and Tollis [34] and the survey by Tamassia and Liotta [92]. The puzzle in Exercise R-8.12 was communicated by Micha Sharir.

Chapter

# 9 Priority Queues



## Contents


**9.1 The Priority Queue Abstract Data Type . . . . . 363**
9.1.1 Priorities . . . . . 363
9.1.2 The Priority Queue ADT . . . . . 364
**9.2 Implementing a Priority Queue . . . . . 365**
9.2.1 The Composition Design Pattern . . . . . 365
9.2.2 Implementation with an Unsorted List . . . . . 366
9.2.3 Implementation with a Sorted List . . . . . 368
**9.3 Heaps . . . . . 370**
9.3.1 The Heap Data Structure . . . . . 370
9.3.2 Implementing a Priority Queue with a Heap . . . . . 372
9.3.3 Array-Based Representation of a Complete Binary Tree . . 376
9.3.4 Python Heap Implementation . . . . . 376
9.3.5 Analysis of a Heap-Based Priority Queue . . . . . 379
9.3.6 Bottom-Up Heap Construction ★ . . . . . 380
9.3.7 Python's heapq Module . . . . . 384
**9.4 Sorting with a Priority Queue . . . . . 385**
9.4.1 Selection-Sort and Insertion-Sort . . . . . 386
9.4.2 Heap-Sort . . . . . 388
**9.5 Adaptable Priority Queues . . . . . 390**
9.5.1 Locators . . . . . 390
9.5.2 Implementing an Adaptable Priority Queue . . . . . 391
**9.6 Exercises . . . . . 395**

# 9.1 The Priority Queue Abstract Data Type

## 9.1.1 Priorities

In Chapter 6, we introduced the queue ADT as a collection of objects that are added and removed according to the ***first-in, first-out*** (***FIFO***) principle. A company's customer call center embodies such a model in which waiting customers are told "calls will be answered in the order that they were received." In that setting, a new call is added to the back of the queue, and each time a customer service representative becomes available, he or she is connected with the call that is removed from the front of the call queue.

In practice, there are many applications in which a queue-like structure is used to manage objects that must be processed in some way, but for which the first-in, first-out policy does not suffice. Consider, for example, an air-traffic control center that has to decide which flight to clear for landing from among many approaching the airport. This choice may be influenced by factors such as each plane's distance from the runway, time spent waiting in a holding pattern, or amount of remaining fuel. It is unlikely that the landing decisions are based purely on a FIFO policy.

There are other situations in which a "first come, first serve" policy might seem reasonable, yet for which other priorities come into play. To use another airline analogy, suppose a certain flight is fully booked an hour prior to departure. Because of the possibility of cancellations, the airline maintains a queue of standby passengers hoping to get a seat. Although the priority of a standby passenger is influenced by the check-in time of that passenger, other considerations include the fare paid and frequent-flyer status. So it may be that an available seat is given to a passenger who has arrived *later* than another, if such a passenger is assigned a better priority by the airline agent.

In this chapter, we introduce a new abstract data type known as a ***priority queue***. This is a collection of prioritized elements that allows arbitrary element insertion, and allows the removal of the element that has first priority. When an element is added to a priority queue, the user designates its priority by providing an associated ***key***. The element with the *minimum* key will be the next to be removed from the queue (thus, an element with key 1 will be given priority over an element with key 2). Although it is quite common for priorities to be expressed numerically, any Python object may be used as a key, as long as the object type supports a consistent meaning for the test `a < b`, for any instances `a` and `b`, so as to define a natural order of the keys. With such generality, applications may develop their own notion of priority for each element. For example, different financial analysts may assign different ratings (i.e., priorities) to a particular asset, such as a share of stock.

### 9.1.2 The Priority Queue ADT

Formally, we model an element and its priority as a key-value pair. We define the priority queue ADT to support the following methods for a priority queue P:

**P.add(k, v):** Insert an item with key k and value v into priority queue P.

**P.min( ):** Return a tuple, (k,v), representing the key and value of an item in priority queue P with minimum key (but do not remove the item); an error occurs if the priority queue is empty.

**P.remove_min( ):** Remove an item with minimum key from priority queue P, and return a tuple, (k,v), representing the key and value of the removed item; an error occurs if the priority queue is empty.

**P.is_empty( ):** Return True if priority queue P does not contain any items.

**len(P):** Return the number of items in priority queue P.

A priority queue may have multiple entries with equivalent keys, in which case methods min and remove_min may report an arbitrary choice of item having minimum key. Values may be any type of object.

In our initial model for a priority queue, we assume that an element's key remains fixed once it has been added to a priority queue. In Section 9.5, we consider an extension that allows a user to update an element's key within the priority queue.

**Example 9.1:** *The following table shows a series of operations and their effects on an initially empty priority queue P. The "Priority Queue" column is somewhat deceiving since it shows the entries as tuples and sorted by key. Such an internal representation is not required of a priority queue.*

| Operation | Return Value | Priority Queue |
|---|---|---|
| P.add(5,A) | | {(5,A)} |
| P.add(9,C) | | {(5,A), (9,C)} |
| P.add(3,B) | | {(3,B), (5,A), (9,C)} |
| P.add(7,D) | | {(3,B), (5,A), (7,D), (9,C)} |
| P.min( ) | (3,B) | {(3,B), (5,A), (7,D), (9,C)} |
| P.remove_min( ) | (3,B) | {(5,A), (7,D), (9,C)} |
| P.remove_min( ) | (5,A) | {(7,D), (9,C)} |
| len(P) | 2 | {(7,D), (9,C)} |
| P.remove_min( ) | (7,D) | {(9,C)} |
| P.remove_min( ) | (9,C) | { } |
| P.is_empty( ) | True | { } |
| P.remove_min( ) | "error" | { } |

# 9.2 Implementing a Priority Queue

In this section, we show how to implement a priority queue by storing its entries in a positional list $L$. (See Section 7.4.) We provide two realizations, depending on whether or not we keep the entries in $L$ sorted by key.

## 9.2.1 The Composition Design Pattern

One challenge in implementing a priority queue is that we must keep track of both an element and its key, even as items are relocated within our data structure. This is reminiscent of a case study from Section 7.6 in which we maintain access counts with each element. In that setting, we introduced the ***composition design pattern***, defining an _Item class that assured that each element remained paired with its associated count in our primary data structure.

For priority queues, we will use composition to store items internally as pairs consisting of a key $k$ and a value $v$. To implement this concept for all priority queue implementations, we provide a PriorityQueueBase class (see Code Fragment 9.1) that includes a definition for a nested class named _Item. We define the syntax a < b, for item instances a and b, to be based upon the keys.

```
1  class PriorityQueueBase:
2    """Abstract base class for a priority queue."""
3
4    class _Item:
5      """Lightweight composite to store priority queue items."""
6      __slots__ = '_key', '_value'
7
8      def __init__(self, k, v):
9        self._key = k
10       self._value = v
11
12     def __lt__(self, other):
13       return self._key < other._key       # compare items based on their keys
14
15   def is_empty(self):                     # concrete method assuming abstract len
16     """Return True if the priority queue is empty."""
17     return len(self) == 0
```

**Code Fragment 9.1:** A PriorityQueueBase class with a nested _Item class that composes a key and a value into a single object. For convenience, we provide a concrete implementation of is_empty that is based on a presumed __len__ impelementation.

### 9.2.2 Implementation with an Unsorted List

In our first concrete implementation of a priority queue, we store entries within an *unsorted list*. Our UnsortedPriorityQueue class is given in Code Fragment 9.2, inheriting from the PriorityQueueBase class introduced in Code Fragment 9.1. For internal storage, key-value pairs are represented as composites, using instances of the inherited _Item class. These items are stored within a PositionalList, identified as the _data member of our class. We assume that the positional list is implemented with a doubly-linked list, as in Section 7.4, so that all operations of that ADT execute in $O(1)$ time.

We begin with an empty list when a new priority queue is constructed. At all times, the size of the list equals the number of key-value pairs currently stored in the priority queue. For this reason, our priority queue __len__ method simply returns the length of the internal _data list. By the design of our PriorityQueueBase class, we inherit a concrete implementation of the is_empty method that relies on a call to our __len__ method.

Each time a key-value pair is added to the priority queue, via the add method, we create a new _Item composite for the given key and value, and add that item to the end of the list. Such an implementation takes $O(1)$ time.

The remaining challenge is that when min or remove_min is called, we must locate the item with minimum key. Because the items are not sorted, we must inspect all entries to find one with a minimum key. For convenience, we define a nonpublic _find_min utility that returns the *position* of an item with minimum key. Knowledge of the position allows the remove_min method to invoke the delete method on the positional list. The min method simply uses the position to retrieve the item when preparing a key-value tuple to return. Due to the loop for finding the minimum key, both min and remove_min methods run in $O(n)$ time, where $n$ is the number of entries in the priority queue.

A summary of the running times for the UnsortedPriorityQueue class is given in Table 9.1.

| Operation | Running Time |
|---|---|
| len | $O(1)$ |
| is_empty | $O(1)$ |
| add | $O(1)$ |
| min | $O(n)$ |
| remove_min | $O(n)$ |

**Table 9.1:** Worst-case running times of the methods of a priority queue of size $n$, realized by means of an unsorted, doubly linked list. The space requirement is $O(n)$.

```
1  class UnsortedPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with an unsorted list."""
3
4    def _find_min(self):                    # nonpublic utility
5      """Return Position of item with minimum key."""
6      if self.is_empty( ):                  # is_empty inherited from base class
7        raise Empty('Priority queue is empty')
8      small = self._data.first( )
9      walk = self._data.after(small)
10     while walk is not None:
11       if walk.element( ) < small.element( ):
12         small = walk
13       walk = self._data.after(walk)
14     return small
15
16   def __init__(self):
17     """Create a new empty Priority Queue."""
18     self._data = PositionalList( )
19
20   def __len__(self):
21     """Return the number of items in the priority queue."""
22     return len(self._data)
23
24   def add(self, key, value):
25     """Add a key-value pair."""
26     self._data.add_last(self._Item(key, value))
27
28   def min(self):
29     """Return but do not remove (k,v) tuple with minimum key."""
30     p = self._find_min( )
31     item = p.element( )
32     return (item._key, item._value)
33
34   def remove_min(self):
35     """Remove and return (k,v) tuple with minimum key."""
36     p = self._find_min( )
37     item = self._data.delete(p)
38     return (item._key, item._value)
```

**Code Fragment 9.2:** An implementation of a priority queue using an unsorted list. The parent class PriorityQueueBase is given in Code Fragment 9.1, and the PositionalList class is from Section 7.4.

### 9.2.3 Implementation with a Sorted List

An alternative implementation of a priority queue uses a positional list, yet maintaining entries sorted by nondecreasing keys. This ensures that the first element of the list is an entry with the smallest key.

Our SortedPriorityQueue class is given in Code Fragment 9.3. The implementation of min and remove_min are rather straightforward given knowledge that the first element of a list has a minimum key. We rely on the first method of the positional list to find the position of the first item, and the delete method to remove the entry from the list. Assuming that the list is implemented with a doubly linked list, operations min and remove_min take $O(1)$ time.

This benefit comes at a cost, however, for method add now requires that we scan the list to find the appropriate position to insert the new item. Our implementation starts at the end of the list, walking backward until the new key is smaller than an existing item; in the worst case, it progresses until reaching the front of the list. Therefore, the add method takes $O(n)$ worst-case time, where $n$ is the number of entries in the priority queue at the time the method is executed. In summary, when using a sorted list to implement a priority queue, insertion runs in linear time, whereas finding and removing the minimum can be done in constant time.

#### Comparing the Two List-Based Implementations

Table 9.2 compares the running times of the methods of a priority queue realized by means of a sorted and unsorted list, respectively. We see an interesting trade-off when we use a list to implement the priority queue ADT. An unsorted list supports fast insertions but slow queries and deletions, whereas a sorted list allows fast queries and deletions, but slow insertions.

| Operation | Unsorted List | Sorted List |
|---|---|---|
| len | $O(1)$ | $O(1)$ |
| is_empty | $O(1)$ | $O(1)$ |
| add | $O(1)$ | $O(n)$ |
| min | $O(n)$ | $O(1)$ |
| remove_min | $O(n)$ | $O(1)$ |

**Table 9.2:** Worst-case running times of the methods of a priority queue of size $n$, realized by means of an unsorted or sorted list, respectively. We assume that the list is implemented by a doubly linked list. The space requirement is $O(n)$.

```
1  class SortedPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with a sorted list."""
3
4    def __init__(self):
5      """Create a new empty Priority Queue."""
6      self._data = PositionalList( )
7
8    def __len__(self):
9      """Return the number of items in the priority queue."""
10     return len(self._data)
11
12   def add(self, key, value):
13     """Add a key-value pair."""
14     newest = self._Item(key, value)              # make new item instance
15     walk = self._data.last( )          # walk backward looking for smaller key
16     while walk is not None and newest < walk.element( ):
17       walk = self._data.before(walk)
18     if walk is None:
19       self._data.add_first(newest)               # new key is smallest
20     else:
21       self._data.add_after(walk, newest)         # newest goes after walk
22
23   def min(self):
24     """Return but do not remove (k,v) tuple with minimum key."""
25     if self.is_empty( ):
26       raise Empty('Priority queue is empty.')
27     p = self._data.first( )
28     item = p.element( )
29     return (item._key, item._value)
30
31   def remove_min(self):
32     """Remove and return (k,v) tuple with minimum key."""
33     if self.is_empty( ):
34       raise Empty('Priority queue is empty.')
35     item = self._data.delete(self._data.first( ))
36     return (item._key, item._value)
```

**Code Fragment 9.3:** An implementation of a priority queue using a sorted list. The parent class PriorityQueueBase is given in Code Fragment 9.1, and the PositionalList class is from Section 7.4.

# 9.3 Heaps

The two strategies for implementing a priority queue ADT in the previous section demonstrate an interesting trade-off. When using an *unsorted* list to store entries, we can perform insertions in $O(1)$ time, but finding or removing an element with minimum key requires an $O(n)$-time loop through the entire collection. In contrast, if using a *sorted* list, we can trivially find or remove the minimum element in $O(1)$ time, but adding a new element to the queue may require $O(n)$ time to restore the sorted order.

In this section, we provide a more efficient realization of a priority queue using a data structure called a ***binary heap***. This data structure allows us to perform both insertions and removals in logarithmic time, which is a significant improvement over the list-based implementations discussed in Section 9.2. The fundamental way the heap achieves this improvement is to use the structure of a binary tree to find a compromise between elements being entirely unsorted and perfectly sorted.

## 9.3.1 The Heap Data Structure

A heap (see Figure 9.1) is a binary tree $T$ that stores a collection of items at its positions and that satisfies two additional properties: a relational property defined in terms of the way keys are stored in $T$ and a structural property defined in terms of the shape of $T$ itself. The relational property is the following:

***Heap-Order Property*:** In a heap $T$, for every position $p$ other than the root, the key stored at $p$ is greater than or equal to the key stored at $p$'s parent.

As a consequence of the heap-order property, the keys encountered on a path from the root to a leaf of $T$ are in nondecreasing order. Also, a minimum key is always stored at the root of $T$. This makes it easy to locate such an item when min or remove_min is called, as it is informally said to be "at the top of the heap" (hence, the name "heap" for the data structure). By the way, the heap data structure defined here has nothing to do with the memory heap (Section 15.1.1) used in the run-time environment supporting a programming language like Python.

For the sake of efficiency, as will become clear later, we want the heap $T$ to have as small a height as possible. We enforce this requirement by insisting that the heap $T$ satisfy an additional structural property—it must be what we term ***complete***.

***Complete Binary Tree Property:*** A heap $T$ with height $h$ is a ***complete*** binary tree if levels $0, 1, 2, \ldots, h-1$ of $T$ have the maximum number of nodes possible (namely, level $i$ has $2^i$ nodes, for $0 \leq i \leq h-1$) and the remaining nodes at level $h$ reside in the leftmost possible positions at that level.

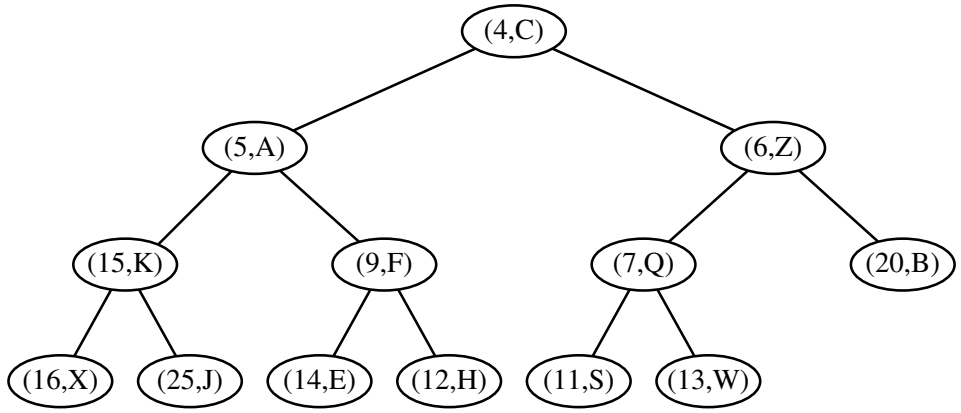



**Figure 9.1:** Example of a heap storing 13 entries with integer keys. The last position is the one storing entry $(13,W)$.

The tree in Figure 9.1 is complete because levels 0, 1, and 2 are full, and the six nodes in level 3 are in the six leftmost possible positions at that level. In formalizing what we mean by the leftmost possible positions, we refer to the discussion of ***level numbering*** from Section 8.3.2, in the context of an array-based representation of a binary tree. (In fact, in Section 9.3.3 we will discuss the use of an array to represent a heap.) A complete binary tree with $n$ elements is one that has positions with level numbering 0 through $n-1$. For example, in an array-based representation of the above tree, its 13 entries would be stored consecutively from $A[0]$ to $A[12]$.

## The Height of a Heap

Let $h$ denote the height of $T$. Insisting that $T$ be complete also has an important consequence, as shown in Proposition 9.2.

**Proposition 9.2:** *A heap $T$ storing $n$ entries has height $h = \lfloor \log n \rfloor$.*

**Justification:** From the fact that $T$ is complete, we know that the number of nodes in levels 0 through $h-1$ of $T$ is precisely $1+2+4+\cdots+2^{h-1} = 2^h - 1$, and that the number of nodes in level $h$ is at least 1 and at most $2^h$. Therefore

$$n \geq 2^h - 1 + 1 = 2^h \qquad \text{and} \qquad n \leq 2^h - 1 + 2^h = 2^{h+1} - 1.$$

By taking the logarithm of both sides of inequality $2^h \leq n$, we see that height $h \leq \log n$. By rearranging terms and taking the logarithm of both sides of inequality $n \leq 2^{h+1} - 1$, we see that $\log(n+1) - 1 \leq h$. Since $h$ is an integer, these two inequalities imply that $h = \lfloor \log n \rfloor$. ■

### 9.3.2 Implementing a Priority Queue with a Heap

Proposition 9.2 has an important consequence, for it implies that if we can perform update operations on a heap in time proportional to its height, then those operations will run in logarithmic time. Let us therefore turn to the problem of how to efficiently perform various priority queue methods using a heap.

We will use the composition pattern from Section 9.2.1 to store key-value pairs as items in the heap. The len and is_empty methods can be implemented based on examination of the tree, and the min operation is equally trivial because the heap property assures that the element at the root of the tree has a minimum key. The interesting algorithms are those for implementing the add and remove_min methods.

#### Adding an Item to the Heap

Let us consider how to perform add(k,v) on a priority queue implemented with a heap $T$. We store the pair $(k, v)$ as an item at a new node of the tree. To maintain the ***complete binary tree property***, that new node should be placed at a position $p$ just beyond the rightmost node at the bottom level of the tree, or as the leftmost position of a new level, if the bottom level is already full (or if the heap is empty).

#### Up-Heap Bubbling After an Insertion

After this action, the tree $T$ is complete, but it may violate the ***heap-order property***. Hence, unless position $p$ is the root of $T$ (that is, the priority queue was empty before the insertion), we compare the key at position $p$ to that of $p$'s parent, which we denote as q. If key $k_p \geq k_q$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p < k_q$, then we need to restore the heap-order property, which can be locally achieved by swapping the entries stored at positions $p$ and $q$. (See Figure 9.2c and d.) This swap causes the new item to move up one level. Again, the heap-order property may be violated, so we repeat the process, going up in $T$ until no violation of the heap-order property occurs. (See Figure 9.2e and h.)

The upward movement of the newly inserted entry by means of swaps is conventionally called ***up-heap bubbling***. A swap either resolves the violation of the heap-order property or propagates it one level up in the heap. In the worst case, up-heap bubbling causes the new entry to move all the way up to the root of heap $T$. Thus, in the worst case, the number of swaps performed in the execution of method add is equal to the height of $T$. By Proposition 9.2, that bound is $\lfloor \log n \rfloor$.

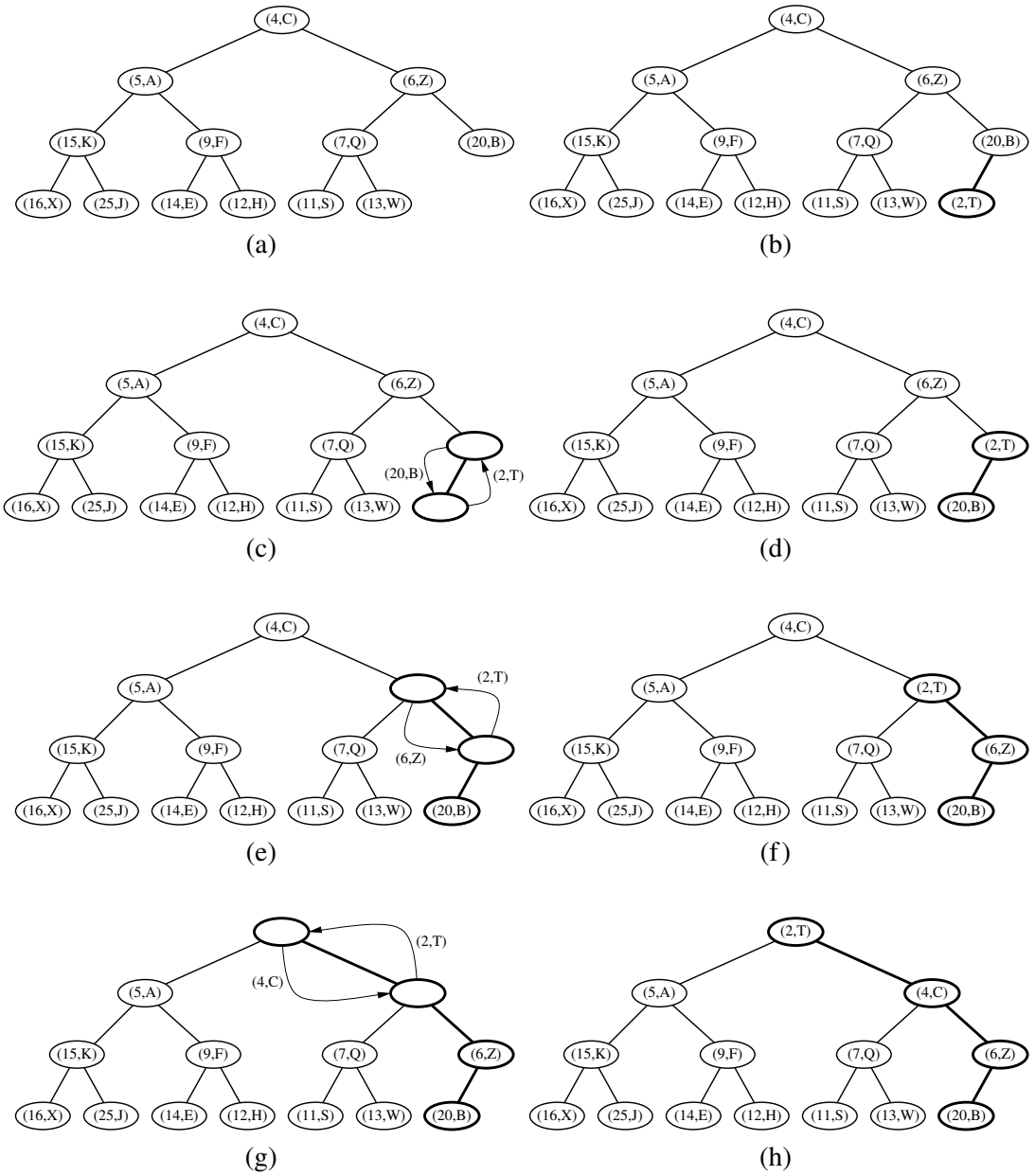



**Figure 9.2:** Insertion of a new entry with key 2 into the heap of Figure 9.1: (a) initial heap; (b) after performing operation add; (c and d) swap to locally restore the partial order property; (e and f) another swap; (g and h) final swap.

### Removing the Item with Minimum Key

Let us now turn to method remove_min of the priority queue ADT. We know that an entry with the smallest key is stored at the root $r$ of $T$ (even if there is more than one entry with smallest key). However, in general we cannot simply delete node $r$, because this would leave two disconnected subtrees.

Instead, we ensure that the shape of the heap respects the ***complete binary tree property*** by deleting the leaf at the *last* position $p$ of $T$, defined as the rightmost position at the bottommost level of the tree. To preserve the item from the last position $p$, we copy it to the root $r$ (in place of the item with minimum key that is being removed by the operation). Figure 9.3a and b illustrates an example of these steps, with minimal item $(4,C)$ being removed from the root and replaced by item $(13,W)$ from the last position. The node at the last position is removed from the tree.

### Down-Heap Bubbling After a Removal

We are not yet done, however, for even though $T$ is now complete, it likely violates the heap-order property. If $T$ has only one node (the root), then the heap-order property is trivially satisfied and the algorithm terminates. Otherwise, we distinguish two cases, where $p$ initially denotes the root of $T$:

- If $p$ has no right child, let $c$ be the left child of $p$.
- Otherwise ($p$ has both children), let $c$ be a child of $p$ with minimal key.

If key $k_p \leq k_c$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p > k_c$, then we need to restore the heap-order property. This can be locally achieved by swapping the entries stored at $p$ and $c$. (See Figure 9.3c and d.) It is worth noting that when $p$ has two children, we intentionally consider the *smaller* key of the two children. Not only is the key of $c$ smaller than that of $p$, it is at least as small as the key at $c$'s sibling. This ensures that the heap-order property is locally restored when that smaller key is promoted above the key that had been at $p$ and that at $c$'s sibling.

Having restored the heap-order property for node $p$ relative to its children, there may be a violation of this property at $c$; hence, we may have to continue swapping down $T$ until no violation of the heap-order property occurs. (See Figure 9.3e–h.) This downward swapping process is called ***down-heap bubbling***. A swap either resolves the violation of the heap-order property or propagates it one level down in the heap. In the worst case, an entry moves all the way down to the bottom level. (See Figure 9.3.) Thus, the number of swaps performed in the execution of method remove_min is, in the worst case, equal to the height of heap $T$, that is, it is $\lfloor \log n \rfloor$ by Proposition 9.2.

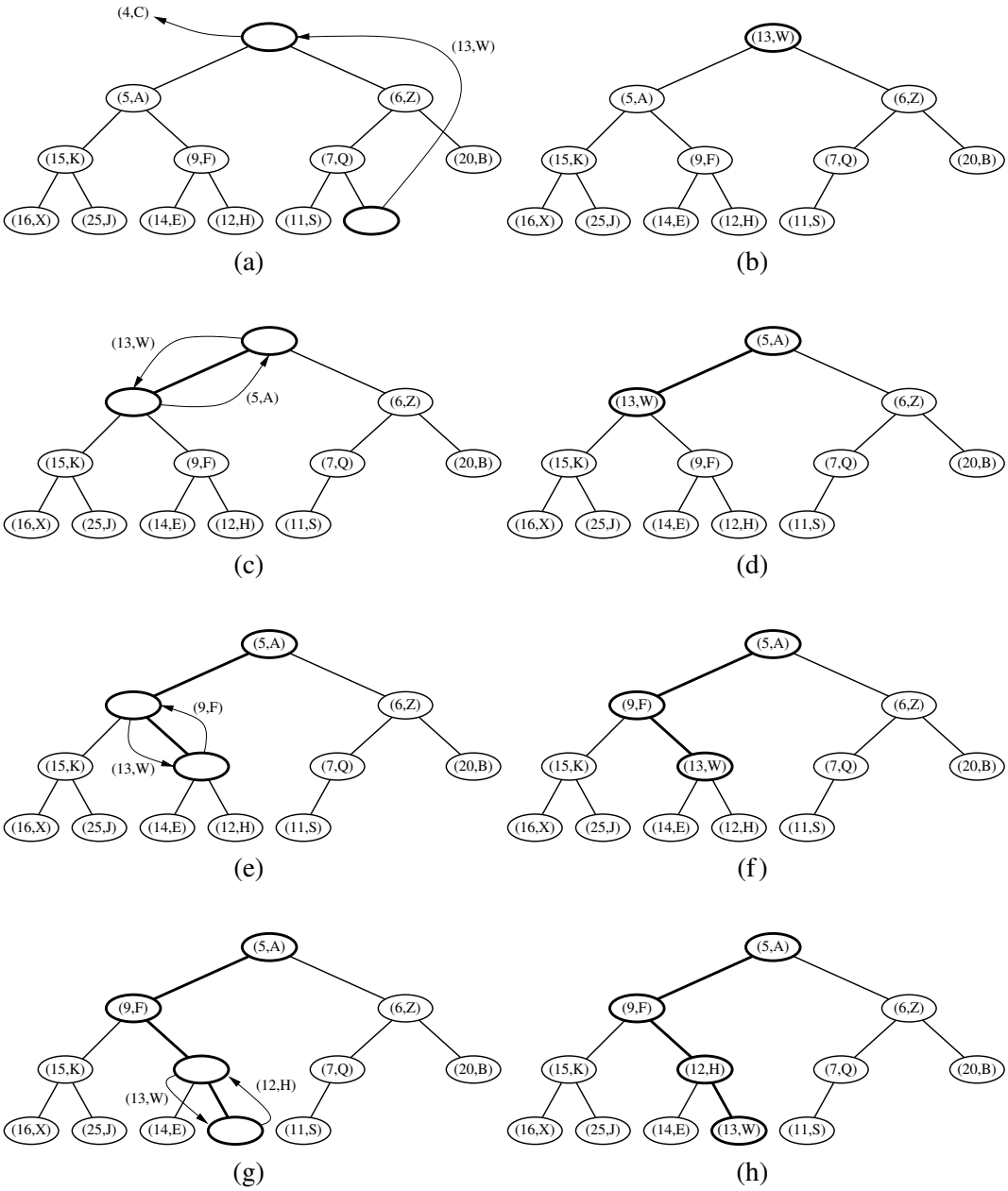



**Figure 9.3:** Removal of the entry with the smallest key from a heap: (a and b) deletion of the last node, whose entry gets stored into the root; (c and d) swap to locally restore the heap-order property; (e and f) another swap; (g and h) final swap.

### 9.3.3 Array-Based Representation of a Complete Binary Tree

The array-based representation of a binary tree (Section 8.3.2) is especially suitable for a complete binary tree $T$. We recall that in this implementation, the elements of $T$ are stored in an array-based list $A$ such that the element at position $p$ in $T$ is stored in $A$ with index equal to the level number $f(p)$ of $p$, defined as follows:

- If $p$ is the root of $T$, then $f(p) = 0$.
- If $p$ is the left child of position $q$, then $f(p) = 2f(q) + 1$.
- If $p$ is the right child of position $q$, then $f(p) = 2f(q) + 2$.

With this implementation, the elements of $T$ have contiguous indices in the range $[0, n-1]$ and the last position of $T$ is always at index $n-1$, where $n$ is the number of positions of $T$. For example, Figure 9.4 illustrates the array-based representation of the heap structure originally portrayed in Figure 9.1.

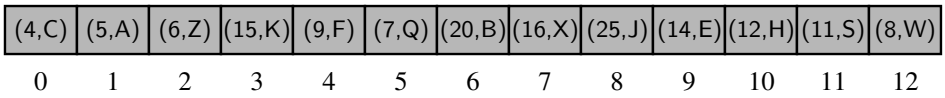



**Figure 9.4:** An array-based representation of the heap from Figure 9.1.

Implementing a priority queue using an array-based heap representation allows us to avoid some complexities of a node-based tree structure. In particular, the add and remove_min operations of a priority queue both depend on locating the last index of a heap of size $n$. With the array-based representation, the last position is at index $n-1$ of the array. Locating the last position of a complete binary tree implemented with a linked structure requires more effort. (See Exercise C-9.34.)

If the size of a priority queue is not known in advance, use of an array-based representation does introduce the need to dynamically resize the array on occasion, as is done with a Python list. The space usage of such an array-based representation of a complete binary tree with $n$ nodes is $O(n)$, and the time bounds of methods for adding or removing elements become ***amortized***. (See Section 5.3.1.)

### 9.3.4 Python Heap Implementation

We provide a Python implementation of a heap-based priority queue in Code Fragments 9.4 and 9.5. We use an array-based representation, maintaining a Python list of item composites. Although we do not formally use the binary tree ADT, Code Fragment 9.4 includes nonpublic utility functions that compute the level numbering of a parent or child of another. This allows us to describe the rest of our algorithms using tree-like terminology of *parent*, *left*, and *right*. However, the relevant variables are integer indexes (not "position" objects). We use recursion to implement the repetition in the _upheap and _downheap utilities.

```
1  class HeapPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with a binary heap."""
3    #------------------------------ nonpublic behaviors ------------------------------
4    def _parent(self, j):
5      return (j-1) // 2
6
7    def _left(self, j):
8      return 2*j + 1
9
10   def _right(self, j):
11     return 2*j + 2
12
13   def _has_left(self, j):
14     return self._left(j) < len(self._data)      # index beyond end of list?
15
16   def _has_right(self, j):
17     return self._right(j) < len(self._data)     # index beyond end of list?
18
19   def _swap(self, i, j):
20     """Swap the elements at indices i and j of array."""
21     self._data[i], self._data[j] = self._data[j], self._data[i]
22
23   def _upheap(self, j):
24     parent = self._parent(j)
25     if j > 0 and self._data[j] < self._data[parent]:
26       self._swap(j, parent)
27       self._upheap(parent)                      # recur at position of parent
28
29   def _downheap(self, j):
30     if self._has_left(j):
31       left = self._left(j)
32       small_child = left                        # although right may be smaller
33       if self._has_right(j):
34         right = self._right(j)
35         if self._data[right] < self._data[left]:
36           small_child = right
37       if self._data[small_child] < self._data[j]:
38         self._swap(j, small_child)
39         self._downheap(small_child)             # recur at position of small child
```

**Code Fragment 9.4:** An implementation of a priority queue using an array-based heap (continued in Code Fragment 9.5). The extends the PriorityQueueBase class from Code Fragment 9.1.

```
40    #------------------------------ public behaviors ------------------------------
41    def __init__(self):
42      """Create a new empty Priority Queue."""
43      self._data = [ ]
44
45    def __len__(self):
46      """Return the number of items in the priority queue."""
47      return len(self._data)
48
49    def add(self, key, value):
50      """Add a key-value pair to the priority queue."""
51      self._data.append(self._Item(key, value))
52      self._upheap(len(self._data) - 1)          # upheap newly added position
53
54    def min(self):
55      """Return but do not remove (k,v) tuple with minimum key.
56
57      Raise Empty exception if empty.
58      """
59      if self.is_empty( ):
60        raise Empty('Priority queue is empty.')
61      item = self._data[0]
62      return (item._key, item._value)
63
64    def remove_min(self):
65      """Remove and return (k,v) tuple with minimum key.
66
67      Raise Empty exception if empty.
68      """
69      if self.is_empty( ):
70        raise Empty('Priority queue is empty.')
71      self._swap(0, len(self._data) - 1)         # put minimum item at the end
72      item = self._data.pop( )                   # and remove it from the list;
73      self._downheap(0)                          # then fix new root
74      return (item._key, item._value)
```

**Code Fragment 9.5:** An implementation of a priority queue using an array-based heap (continued from Code Fragment 9.4).

### 9.3.5 Analysis of a Heap-Based Priority Queue

Table 9.3 shows the running time of the priority queue ADT methods for the heap implementation of a priority queue, assuming that two keys can be compared in $O(1)$ time and that the heap $T$ is implemented with an array-based or linked-based tree representation.

In short, each of the priority queue ADT methods can be performed in $O(1)$ or in $O(\log n)$ time, where $n$ is the number of entries at the time the method is executed. The analysis of the running time of the methods is based on the following:

- The heap $T$ has $n$ nodes, each storing a reference to a key-value pair.
- The height of heap $T$ is $O(\log n)$, since $T$ is complete (Proposition 9.2).
- The min operation runs in $O(1)$ because the root of the tree contains such an element.
- Locating the last position of a heap, as required for add and remove_min, can be performed in $O(1)$ time for an array-based representation, or $O(\log n)$ time for a linked-tree representation. (See Exercise C-9.34.)
- In the worst case, up-heap and down-heap bubbling perform a number of swaps equal to the height of $T$.

| Operation | Running Time |
|---:|---|
| len(P), P.is_empty( ) | $O(1)$ |
| P.min( ) | $O(1)$ |
| P.add( ) | $O(\log n)^*$ |
| P.remove_min( ) | $O(\log n)^*$ |

*amortized, if array-based

**Table 9.3:** Performance of a priority queue, $P$, realized by means of a heap. We let $n$ denote the number of entries in the priority queue at the time an operation is executed. The space requirement is $O(n)$. The running time of operations min and remove_min are amortized for an array-based representation, due to occasional resizing of a dynamic array; those bounds are worst case with a linked tree structure.

We conclude that the heap data structure is a very efficient realization of the priority queue ADT, independent of whether the heap is implemented with a linked structure or an array. The heap-based implementation achieves fast running times for both insertion and removal, unlike the implementations that were based on using an unsorted or sorted list.

### 9.3.6 Bottom-Up Heap Construction ⋆

If we start with an initially empty heap, $n$ successive calls to the add operation will run in $O(n \log n)$ time in the worst case. However, if all $n$ key-value pairs to be stored in the heap are given in advance, such as during the first phase of the heap-sort algorithm, there is an alternative ***bottom-up*** construction method that runs in $O(n)$ time. (Heap-sort, however, still requires $\Theta(n \log n)$ time because of the second phase in which we repeatedly remove the remaining element with smallest key.)

In this section, we describe the bottom-up heap construction, and provide an implementation that can be used by the constructor of a heap-based priority queue.

For simplicity of exposition, we describe this bottom-up heap construction assuming the number of keys, $n$, is an integer such that $n = 2^{h+1} - 1$. That is, the heap is a complete binary tree with every level being full, so the heap has height $h = \log(n+1) - 1$. Viewed nonrecursively, bottom-up heap construction consists of the following $h + 1 = \log(n+1)$ steps:

1. In the first step (see Figure 9.5b), we construct $(n+1)/2$ elementary heaps storing one entry each.
2. In the second step (see Figure 9.5c–d), we form $(n+1)/4$ heaps, each storing three entries, by joining pairs of elementary heaps and adding a new entry. The new entry is placed at the root and may have to be swapped with the entry stored at a child to preserve the heap-order property.
3. In the third step (see Figure 9.5e–f), we form $(n+1)/8$ heaps, each storing 7 entries, by joining pairs of 3-entry heaps (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

   ⋮

$i$. In the generic $i^{th}$ step, $2 \leq i \leq h$, we form $(n+1)/2^i$ heaps, each storing $2^i - 1$ entries, by joining pairs of heaps storing $(2^{i-1} - 1)$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

   ⋮

$h+1$. In the last step (see Figure 9.5g–h), we form the final heap, storing all the $n$ entries, by joining two heaps storing $(n-1)/2$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

We illustrate bottom-up heap construction in Figure 9.5 for $h = 3$.

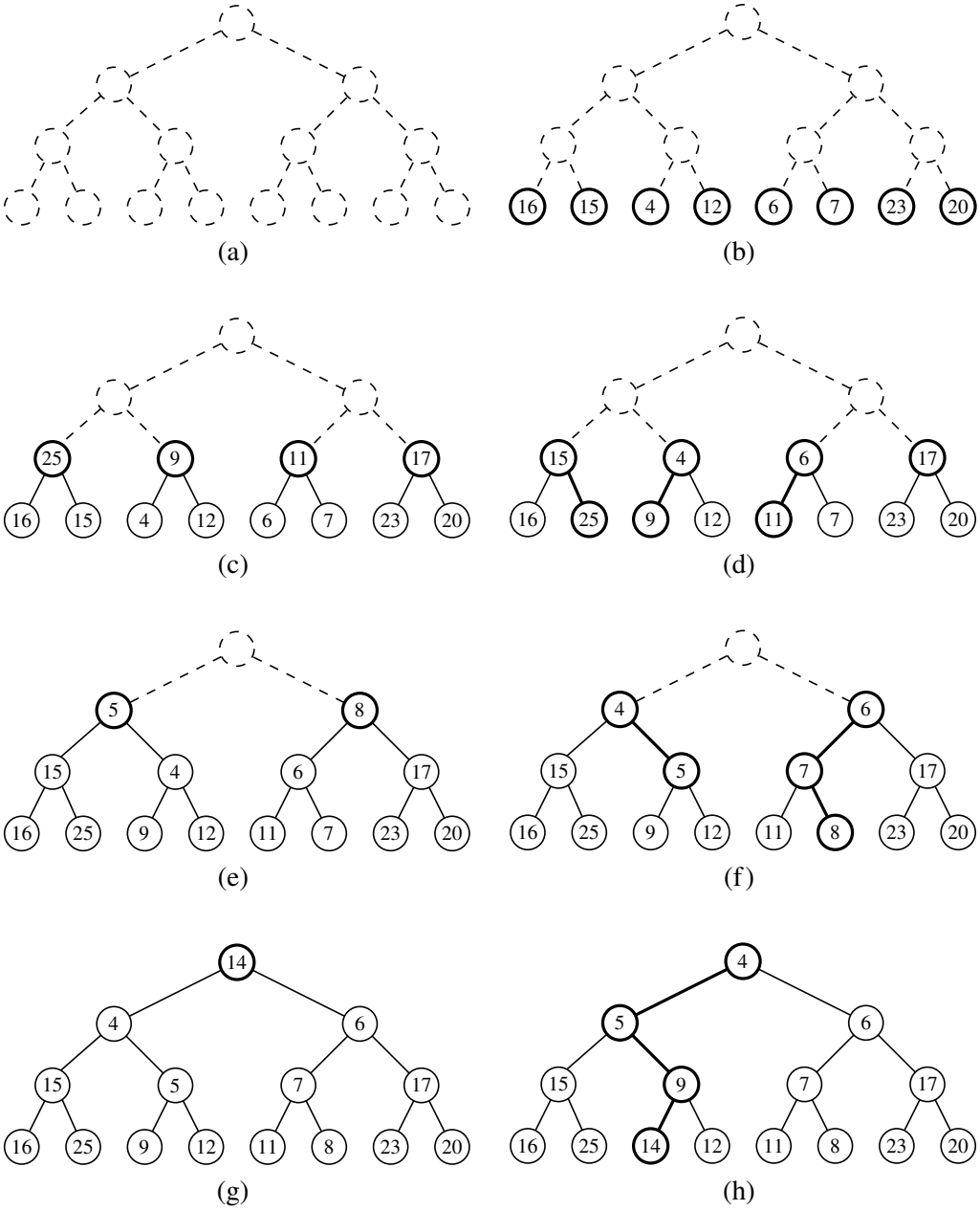



**Figure 9.5:** Bottom-up construction of a heap with 15 entries: (a and b) we begin by constructing 1-entry heaps on the bottom level; (c and d) we combine these heaps into 3-entry heaps, and then (e and f) 7-entry heaps, until (g and h) we create the final heap. The paths of the down-heap bubblings are highlighted in (d, f, and h). For simplicity, we only show the key within each node instead of the entire entry.

## Python Implementation of a Bottom-Up Heap Construction

Implementing a bottom-up heap construction is quite easy, given the existence of a "down-heap" utility function. The "merging" of two equally sized heaps that are subtrees of a common position $p$, as described in the opening of this section, can be accomplished simply by down-heaping $p$'s entry. For example, that is what happened to the key 14 in going from Figure 9.5(f) to (g).

With our array-based representation of a heap, if we initially store all $n$ items in arbitrary order within the array, we can implement the bottom-up heap construction process with a single loop that makes a call to _downheap from each position of the tree, as long as those calls are ordered starting with the *deepest* level and ending with the root of the tree. In fact, that loop can start with the deepest nonleaf, since there is no effect when down-heap is called at a leaf position.

In Code Fragment 9.6, we augment the original HeapPriorityQueue class from Section 9.3.4 to provide support for the bottom-up construction of an initial collection. We introduce a nonpublic utility method, _heapify, that calls _downheap on each nonleaf position, beginning with the deepest and concluding with a call at the root of the tree. We have redesigned the constructor of the class to accept an optional parameter that can be any sequence of (k,v) tuples. Rather than initializing self._data to an empty list, we use a list comprehension syntax (see Section 1.9.2) to create an initial list of item composites based on the given contents. We declare an empty sequence as the default parameter value so that the default syntax HeapPriorityQueue( ) continues to result in an empty priority queue.

```
def __init__(self, contents=( )):
  """Create a new priority queue.

  By default, queue will be empty. If contents is given, it should be as an
  iterable sequence of (k,v) tuples specifying the initial contents.
"""
  self._data = [ self._Item(k,v) for k,v in contents ]      # empty by default
  if len(self._data) > 1:
    self._heapify( )

def _heapify(self):
  start = self._parent(len(self) - 1)       # start at PARENT of last leaf
  for j in range(start, -1, -1):            # going to and including the root
    self._downheap(j)
```

**Code Fragment 9.6:** Revision to the HeapPriorityQueue class of Code Fragments 9.4 and 9.5 to support a linear-time construction given an initial sequence of entries.

## Asymptotic Analysis of Bottom-Up Heap Construction

Bottom-up heap construction is asymptotically faster than incrementally inserting $n$ keys into an initially empty heap. Intuitively, we are performing a single down-heap operation at each position in the tree, rather than a single up-heap operation from each. Since more nodes are closer to the bottom of a tree than the top, the sum of the downward paths is linear, as shown in the following proposition.

**Proposition 9.3:** *Bottom-up construction of a heap with $n$ entries takes $O(n)$ time, assuming two keys can be compared in $O(1)$ time.*

**Justification:** The primary cost of the construction is due to the down-heap steps performed at each nonleaf position. Let $\pi_v$ denote the path of $T$ from nonleaf node $v$ to its "inorder successor" leaf, that is, the path that starts at $v$, goes to the right child of $v$, and then goes down leftward until it reaches a leaf. Although, $\pi_v$ is not necessarily the path followed by the down-heap bubbling step from $v$, the length $\|\pi_v\|$ (its number of edges) is proportional to the height of the subtree rooted at $v$, and thus a bound on the complexity of the down-heap operation at $v$. We can bound the total running time of the bottom-up heap construction algorithm based on the sum of the sizes of paths, $\sum_v \|\pi_v\|$. For intuition, Figure 9.6 illustrates the justification "visually," marking each edge with the label of the nonleaf node $v$ whose path $\pi_v$ contains that edge.

We claim that the paths $\pi_v$ for all nonleaf $v$ are edge-disjoint, and thus the sum of the path lengths is bounded by the number of total edges in the tree, hence $O(n)$. To show this, we consider what we term "right-leaning" and "left-leaning" edges (i.e., those going from a parent to a right, respectively left, child). A particular right-leaning edge $e$ can only be part of the path $\pi_v$ for node $v$ that is the parent in the relationship represented by $e$. Left-leaning edges can be partitioned by considering the leaf that is reached if continuing down leftward until reaching a leaf. Each nonleaf node only uses left-leaning edges in the group leading to that nonleaf node's inorder successor. Since each nonleaf node must have a different inorder successor, no two such paths can contain the same left-leaning edge. We conclude that the bottom-up construction of heap $T$ takes $O(n)$ time. ■

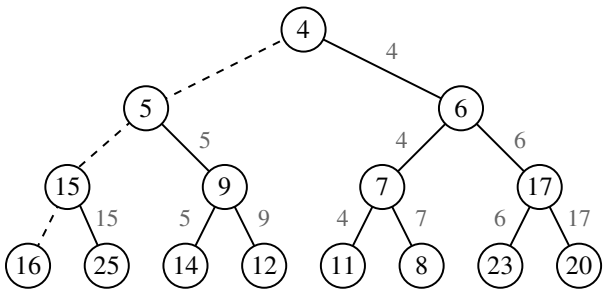



**Figure 9.6:** Visual justification of the linear running time of bottom-up heap construction. Each edge $e$ is labeled with a node $v$ for which $\pi_v$ contains $e$ (if any).

### 9.3.7 Python's heapq Module

Python's standard distribution includes a heapq module that provides support for heap-based priority queues. That module does not provide any priority queue class; instead it provides functions that allow a standard Python list to be managed as a heap. Its model is essentially the same as our own, with $n$ elements stored in list cells $L[0]$ through $L[n-1]$, based on the level-numbering indices with the smallest element at the root in $L[0]$. We note that heapq does not separately manage associated values; elements serve as their own key.

The heapq module supports the following functions, all of which presume that existing list $L$ satisfies the heap-order property prior to the call:

**heappush(L, e):** Push element e onto list L and restore the heap-order property. The function executes in $O(\log n)$ time.

**heappop(L):** Pop and return the element with smallest value from list L, and reestablish the heap-order property. The operation executes in $O(\log n)$ time.

**heappushpop(L, e):** Push element e on list L and then pop and return the smallest item. The time is $O(\log n)$, but it is slightly more efficient than separate calls to push and pop because the size of the list never changes. If the newly pushed element becomes the smallest, it is immediately returned. Otherwise, the new element takes the place of the popped element at the root and a down-heap is performed.

**heapreplace(L, e):** Similar to heappushpop, but equivalent to the pop being performed before the push (in other words, the new element cannot be returned as the smallest). Again, the time is $O(\log n)$, but it is more efficient that two separate operations.

The module supports additional functions that operate on sequences that do not previously satisfy the heap-order property.

**heapify(L):** Transform unordered list to satisfy the heap-order property. This executes in $O(n)$ time by using the bottom-up construction algorithm.

**nlargest(k, iterable):** Produce a list of the k largest values from a given iterable. This can be implemented to run in $O(n + k \log n)$ time, where we use $n$ to denote the length of the iterable (see Exercise C-9.42).

**nsmallest(k, iterable):** Produce a list of the k smallest values from a given iterable. This can be implemented to run in $O(n + k \log n)$ time, using similar technique as with nlargest.

## 9.4 Sorting with a Priority Queue

In defining the priority queue ADT, we noted that any type of object can be used as a key, but that any pair of keys must be comparable to each other, and that the set of keys be naturally ordered. In Python, it is common to rely on the $<$ operator to define such an order, in which case the following properties must be satisfied:

- **Irreflexive property**: $k \not< k$.
- **Transitive property**: if $k_1 < k_2$ and $k_2 < k_3$, then $k_1 < k_3$.

Formally, such a relationship defines what is known as a ***strict weak order***, as it allows for keys to be considered equal to each other, but the broader equivalence classes are ***totally ordered***, as they can be uniquely arranged from smallest to largest due to the transitive property.

As our first application of priority queues, we demonstrate how they can be used to sort a collection $C$ of comparable elements. That is, we can produce a sequence of elements of $C$ in increasing order (or at least in nondecreasing order if there are duplicates). The algorithm is quite simple—we insert all elements into an initially empty priority queue, and then we repeatedly call remove_min to retrieve the elements in nondecreasing order.

An implementation of this algorithm is given in Code Fragment 9.7, assuming that $C$ is a positional list. (See Chapter 7.4.) We use an original element of the collection as both a key and value when calling P.add(element, element).

```
1  def pq_sort(C):
2    """Sort a collection of elements stored in a positional list."""
3    n = len(C)
4    P = PriorityQueue( )
5    for j in range(n):
6      element = C.delete(C.first( ))
7      P.add(element, element)          # use element as key and value
8    for j in range(n):
9      (k,v) = P.remove_min( )
10     C.add_last(v)                    # store smallest remaining element in C
```

**Code Fragment 9.7:** An implementation of the pq_sort function, assuming an appropriate implementation of a PriorityQueue class. Note that each element of the input list $C$ serves as its own key in the priority queue $P$.

With a minor modification to this code, we can provide more general support, sorting elements according to an ordering other than the default. For example, when working with strings, the $<$ operator defines a ***lexicographic ordering***, which is an extension of the alphabetic ordering to Unicode. For example, we have that `'12'` $<$ `'4'` because of the order of the first character of each string, just as

'apple' < 'banana'. Suppose that we have an application in which we have a list of strings that are all known to represent integral values (e.g., '12'), and our goal is to sort the strings according to those integral values.

In Python, the standard approach for customizing the order for a sorting algorithm is to provide, as an optional parameter to the sorting function, an object that is itself a one-parameter function that computes a key for a given element. (See Sections 1.5 and 1.10 for a discussion of this approach in the context of the built-in max function.) For example, with a list of (numeric) strings, we might wish to use the value of int(s) as a key for a string s of the list. In this case, the constructor for the int class can serve as the one-parameter function for computing a key. In that way, the string '4' will be ordered before string '12' because its key int('4') < int('12'). We leave it as an exercise to support such an optional key parameter for the pq_sort function. (See Exercise C-9.46.)

## 9.4.1 Selection-Sort and Insertion-Sort

Our pq_sort function works correctly given any valid implementation of the priority queue class. However, the running time of the sorting algorithm depends on the running times of the operations add and remove_min for the given priority queue class. We next discuss a choice of priority queue implementations that in effect cause the pq_sort computation to behave as one of several classic sorting algorithms.

### Selection-Sort

If we implement $P$ with an unsorted list, then Phase 1 of pq_sort takes $O(n)$ time, for we can add each element in $O(1)$ time. In Phase 2, the running time of each remove_min operation is proportional to the size of $P$. Thus, the bottleneck computation is the repeated "selection" of the minimum element in Phase 2. For this reason, this algorithm is better known as ***selection-sort***. (See Figure 9.7.)

As noted above, the bottleneck is in Phase 2 where we repeatedly remove an entry with smallest key from the priority queue $P$. The size of $P$ starts at $n$ and incrementally decreases with each remove_min until it becomes 0. Thus, the first operation takes time $O(n)$, the second one takes time $O(n-1)$, and so on. Therefore, the total time needed for the second phase is

$$O(n+(n-1)+\cdots+2+1) = O\left(\textstyle\sum_{i=1}^{n} i\right).$$

By Proposition 3.3, we have $\sum_{i=1}^{n} i = n(n+1)/2$. Thus, Phase 2 takes time $O(n^2)$, as does the entire selection-sort algorithm.

| | | *Collection C* | *Priority Queue P* |
|---|---|---|---|
| Input | | $(7,4,8,2,5,3)$ | $()$ |
| Phase 1 | (a) | $(4,8,2,5,3)$ | $(7)$ |
| | (b) | $(8,2,5,3)$ | $(7,4)$ |
| | $\vdots$ | $\vdots$ | $\vdots$ |
| | (f) | $()$ | $(7,4,8,2,5,3)$ |
| Phase 2 | (a) | $(2)$ | $(7,4,8,5,3)$ |
| | (b) | $(2,3)$ | $(7,4,8,5)$ |
| | (c) | $(2,3,4)$ | $(7,8,5)$ |
| | (d) | $(2,3,4,5)$ | $(7,8)$ |
| | (e) | $(2,3,4,5,7)$ | $(8)$ |
| | (f) | $(2,3,4,5,7,8)$ | $()$ |

**Figure 9.7:** Execution of selection-sort on collection $C = (7,4,8,2,5,3)$.

## Insertion-Sort

If we implement the priority queue $P$ using a sorted list, then we improve the running time of Phase 2 to $O(n)$, for each remove_min operation on $P$ now takes $O(1)$ time. Unfortunately, Phase 1 becomes the bottleneck for the running time, since, in the worst case, each add operation takes time proportional to the current size of $P$. This sorting algorithm is better known as ***insertion-sort*** (see Figure 9.8); in fact, our implementation for adding an element to a priority queue is almost identical to a step of insertion-sort as presented in Section 7.5.

The worst-case running time of Phase 1 of insertion-sort is

$$O(1+2+\ldots+(n-1)+n) = O\left(\textstyle\sum_{i=1}^{n} i\right).$$

Again, by Proposition 3.3, this implies a worst-case $O(n^2)$ time for Phase 1, and thus, the entire insertion-sort algorithm. However, unlike selection-sort, insertion-sort has a *best-case* running time of $O(n)$.

| | | *Collection C* | *Priority Queue P* |
|---|---|---|---|
| Input | | $(7,4,8,2,5,3)$ | $()$ |
| Phase 1 | (a) | $(4,8,2,5,3)$ | $(7)$ |
| | (b) | $(8,2,5,3)$ | $(4,7)$ |
| | (c) | $(2,5,3)$ | $(4,7,8)$ |
| | (d) | $(5,3)$ | $(2,4,7,8)$ |
| | (e) | $(3)$ | $(2,4,5,7,8)$ |
| | (f) | $()$ | $(2,3,4,5,7,8)$ |
| Phase 2 | (a) | $(2)$ | $(3,4,5,7,8)$ |
| | (b) | $(2,3)$ | $(4,5,7,8)$ |
| | $\vdots$ | $\vdots$ | $\vdots$ |
| | (f) | $(2,3,4,5,7,8)$ | $()$ |

**Figure 9.8:** Execution of insertion-sort on collection $C = (7,4,8,2,5,3)$.

### 9.4.2 Heap-Sort

As we have previously observed, realizing a priority queue with a heap has the advantage that all the methods in the priority queue ADT run in logarithmic time or better. Hence, this realization is suitable for applications where fast running times are sought for all the priority queue methods. Therefore, let us again consider the pq_sort scheme, this time using a heap-based implementation of the priority queue.

During Phase 1, the $i^{th}$ add operation takes $O(\log i)$ time, since the heap has $i$ entries after the operation is performed. Therefore this phase takes $O(n \log n)$ time. (It could be improved to $O(n)$ with the bottom-up heap construction described in Section 9.3.6.)

During the second phase of pq_sort, the $j^{th}$ remove_min operation runs in $O(\log(n-j+1))$, since the heap has $n-j+1$ entries at the time the operation is performed. Summing over all $j$, this phase takes $O(n \log n)$ time, so the entire priority-queue sorting algorithm runs in $O(n \log n)$ time when we use a heap to implement the priority queue. This sorting algorithm is better known as ***heap-sort***, and its performance is summarized in the following proposition.

**Proposition 9.4:** *The heap-sort algorithm sorts a collection $C$ of $n$ elements in $O(n \log n)$ time, assuming two elements of $C$ can be compared in $O(1)$ time.*

Let us stress that the $O(n \log n)$ running time of heap-sort is considerably better than the $O(n^2)$ running time of selection-sort and insertion-sort (Section 9.4.1).

#### Implementing Heap-Sort In-Place

If the collection $C$ to be sorted is implemented by means of an array-based sequence, most notably as a Python list, we can speed up heap-sort and reduce its space requirement by a constant factor using a portion of the list itself to store the heap, thus avoiding the use of an auxiliary heap data structure. This is accomplished by modifying the algorithm as follows:

1. We redefine the heap operations to be a *maximum-oriented* heap, with each position's key being at least as *large* as its children. This can be done by recoding the algorithm, or by adjusting the notion of keys to be negatively oriented. At any time during the execution of the algorithm, we use the left portion of $C$, up to a certain index $i-1$, to store the entries of the heap, and the right portion of $C$, from index $i$ to $n-1$, to store the elements of the sequence. Thus, the first $i$ elements of $C$ (at indices $0, \ldots, i-1$) provide the array-list representation of the heap.
2. In the first phase of the algorithm, we start with an empty heap and move the boundary between the heap and the sequence from left to right, one step at a time. In step $i$, for $i = 1, \ldots, n$, we expand the heap by adding the element at index $i-1$.

3. In the second phase of the algorithm, we start with an empty sequence and move the boundary between the heap and the sequence from right to left, one step at a time. At step $i$, for $i = 1, \ldots, n$, we remove a maximum element from the heap and store it at index $n - i$.

In general, we say that a sorting algorithm is ***in-place*** if it uses only a small amount of memory in addition to the sequence storing the objects to be sorted. The variation of heap-sort above qualifies as in-place; instead of transferring elements out of the sequence and then back in, we simply rearrange them. We illustrate the second phase of in-place heap-sort in Figure 9.9.

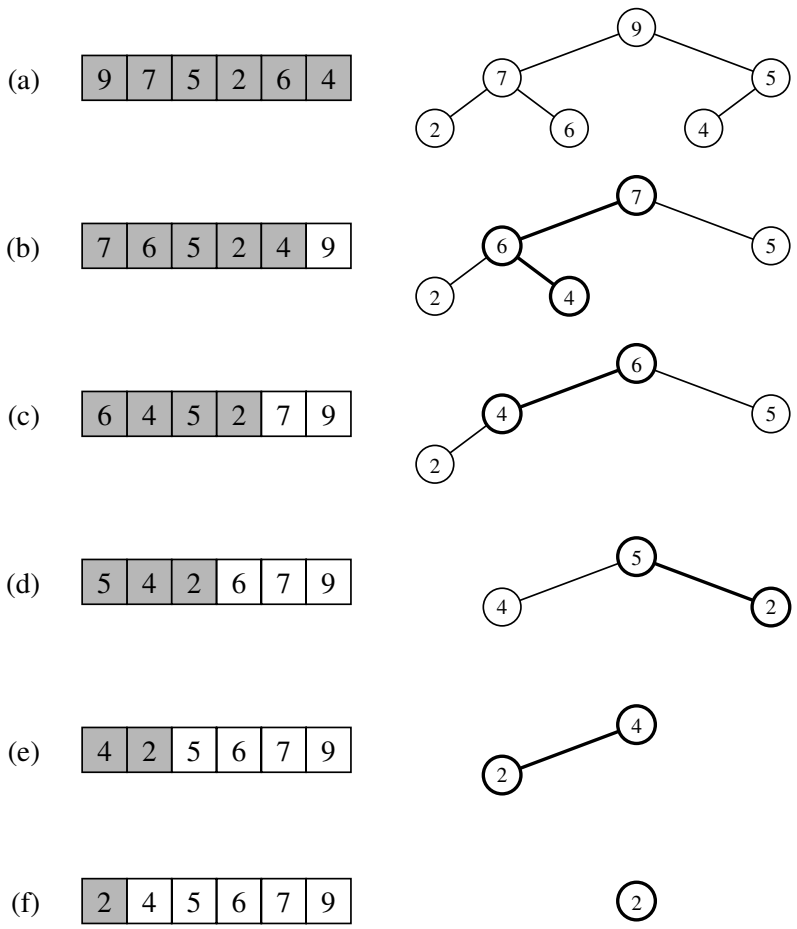



**Figure 9.9:** Phase 2 of an in-place heap-sort. The heap portion of each sequence representation is highlighted. The binary tree that each sequence (implicitly) represents is diagrammed with the most recent path of down-heap bubbling highlighted.

## 9.5 Adaptable Priority Queues

The methods of the priority queue ADT given in Section 9.1.2 are sufficient for most basic applications of priority queues, such as sorting. However, there are situations in which additional methods would be useful, as shown by the scenarios below involving the standby airline passenger application.

- A standby passenger with a pessimistic attitude may become tired of waiting and decide to leave ahead of the boarding time, requesting to be removed from the waiting list. Thus, we would like to remove from the priority queue the entry associated with this passenger. Operation remove_min does not suffice since the passenger leaving does not necessarily have first priority. Instead, we want a new operation, remove, that removes an arbitrary entry.
- Another standby passenger finds her gold frequent-flyer card and shows it to the agent. Thus, her priority has to be modified accordingly. To achieve this change of priority, we would like to have a new operation update allowing us to replace the key of an existing entry with a new key.

We will see another application of adaptable priority queues when implementing certain graph algorithms in Sections 14.6.2 and 14.7.1.

In this section, we develop an ***adaptable priority queue*** ADT and demonstrate how to implement this abstraction as an extension to our heap-based priority queue.

### 9.5.1 Locators

In order to implement methods update and remove efficiently, we need a mechanism for finding a user's element within a priority queue that avoids performing a linear search through the entire collection. To support our goal, when a new element is added to the priority queue, we return a special object known as a ***locator*** to the caller. We then require the user to provide an appropriate locator as a parameter when invoking the update or remove method, as follows, for a priority queue $P$:

**P.update(loc, k, v):** Replace key and value for the item identified by locator loc.

**P.remove(loc):** Remove the item identified by locator loc from the priority queue and return its (key,value) pair.

The locator abstraction is somewhat akin to the Position abstraction used in our positional list ADT from Section 7.4, and our tree ADT from Chapter 8. However, we differentiate between a locator and a position because a locator for a priority queue does not represent a tangible placement of an element within the structure. In our priority queue, an element may be relocated within our data structure during an operation that does not seem directly relevant to that element. A locator for an item will remain valid, as long as that item remains somewhere in the queue.

### 9.5.2 Implementing an Adaptable Priority Queue

In this section, we provide a Python implementation of an adaptable priority queue as an extension of our HeapPriorityQueue class from Section 9.3.4. To implement a Locator class, we will extend the existing _Item composite to add an additional field designating the current index of the element within the array-based representation of our heap, as shown in Figure 9.10.

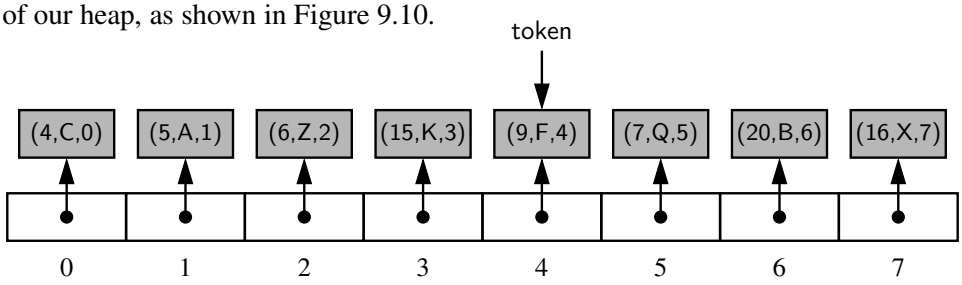



**Figure 9.10:** Representing a heap using a sequence of locators. The third element of each locator instance corresponds to the index of the item within the array. Identifier token is presumed to be a locator reference in the user's scope.

The list is a sequence of references to locator instances, each of which stores a key, value, and the current index of the item within the list. The user will be given a reference to the Locator instance for each inserted element, as portrayed by the token identifier in Figure 9.10.

When we perform priority queue operations on our heap, and items are relocated within our structure, we reposition the locator instances within the list and we update the third field of each locator to reflect its new index within the list. As an example, Figure 9.11 shows the state of the above heap after a call to remove_min( ). The heap operation caused the minimum entry, (4,C), to be removed, and the entry, (16,X), to be temporarily moved from the last position to the root, followed by a down-heap bubble phase. During the down-heap, element (16,X) was swapped

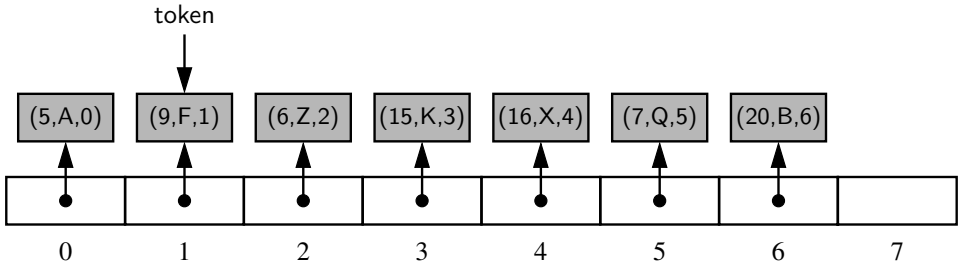



**Figure 9.11:** The result of a call to remove_min( ) on the heap originally portrayed in Figure 9.10. Identifier token continues to reference the same locator instance as in the original configuration, but the placement of that locator in the list has changed, as has the third field of the locator.

with its left child, (5,A), at index 1 of the list, then swapped with its right child, (9,F), at index 4 of the list. In the final configuration, the locator instances for all affected elements have been modified to reflect their new location.

It is important to emphasize that the locator instances have not changed identity. The user's token reference, portrayed in Figures 9.10 and 9.11, continues to reference the same instance; we have simply changed the third field of that instance, and we have changed where that instance is referenced within the list sequence.

With this new representation, providing the additional support for the adaptable priority queue ADT is rather straightforward. When a locator instance is sent as a parameter to update or remove, we may rely on the third field of that structure to designate where the element resides in the heap. With that knowledge, the update of a key may simply require an up-heap or down-heap bubbling step to reestablish the heap-order property. (The complete binary tree property remains intact.) To implement the removal of an arbitrary element, we move the element at the last position to the vacated location, and again perform an appropriate bubbling step to satisfy the heap-order property.

## Python Implementation

Code Fragments 9.8 and 9.9 present a Python implementation of an adaptable priority queue, as a subclass of the HeapPriorityQueue class from Section 9.3.4. Our modifications to the original class are relatively minor. We define a public Locator class that inherits from the nonpublic _Item class and augments it with an additional _index field. We make it a public class because we will be using locators as return values and parameters; however, the public interface for the locator class does not include any other functionality for the user.

To update locators during the flow of our heap operations, we rely on an intentional design decision that our original class uses a nonpublic _swap method for all data movement. We override that utility to execute the additional step of updating the stored indices within the two swapped locator instances.

We provide a new _bubble utility that manages the reinstatement of the heap-order property when a key has changed at an arbitrary position within the heap, either due to a key update, or the blind replacement of a removed element with the item from the last position of the tree. The _bubble utility determines whether to apply up-heap or down-heap bubbling, depending on whether the given location has a parent with a smaller key. (If an updated key coincidentally remains valid for its current location, we technically call _downheap but no swaps result.)

The public methods are provided in Code Fragment 9.9. The existing add method is overridden, both to make use of a Locator instance rather than an _Item instance for storage of the new element, and to return the locator to the caller. The remainder of that method is similar to the original, with the management of locator indices enacted by the use of the new version of _swap. There is no reason to over-

ride the remove_min method because the only change in behavior for the adaptable priority queue is again provided by the overridden _swap method.

The update and remove methods provide the core new functionality for the adaptable priority queue. We perform robust checking of the validity of a locator that is sent by a caller (although in the interest of space, our displayed code does not do preliminary type-checking to ensure that the parameter is indeed a Locator instance). To ensure that a locator is associated with a current element of the given priority queue, we examine the index that is encapsulated within the locator object, and then verify that the entry of the list at that index is the very same locator.

In conclusion, the adaptable priority queue provides the same asymptotic efficiency and space usage as the nonadaptive version, and provides logarithmic performance for the new locator-based update and remove methods. A summary of the performance is given in Table 9.4.

```
1  class AdaptableHeapPriorityQueue(HeapPriorityQueue):
2    """A locator-based priority queue implemented with a binary heap."""
3
4    #------------------------------ nested Locator class ------------------------------
5    class Locator(HeapPriorityQueue._Item):
6      """Token for locating an entry of the priority queue."""
7      __slots__ = '_index'                    # add index as additional field
8
9      def __init__(self, k, v, j):
10       super( ).__init__(k,v)
11       self._index = j
12
13   #------------------------------ nonpublic behaviors ------------------------------
14   # override swap to record new indices
15   def _swap(self, i, j):
16     super( )._swap(i,j)                     # perform the swap
17     self._data[i]._index = i                # reset locator index (post-swap)
18     self._data[j]._index = j                # reset locator index (post-swap)
19
20   def _bubble(self, j):
21     if j > 0 and self._data[j] < self._data[self._parent(j)]:
22       self._upheap(j)
23     else:
24       self._downheap(j)
```

**Code Fragment 9.8:** An implementation of an adaptable priority queue (continued in Code Fragment 9.9). This extends the HeapPriorityQueue class of Code Fragments 9.4 and 9.5

```
25    def add(self, key, value):
26      """Add a key-value pair."""
27      token = self.Locator(key, value, len(self._data)) # initiaize locator index
28      self._data.append(token)
29      self._upheap(len(self._data) - 1)
30      return token
31
32    def update(self, loc, newkey, newval):
33      """Update the key and value for the entry identified by Locator loc."""
34      j = loc._index
35      if not (0 <= j < len(self) and self._data[j] is loc):
36        raise ValueError('Invalid locator')
37      loc._key = newkey
38      loc._value = newval
39      self._bubble(j)
40
41    def remove(self, loc):
42      """Remove and return the (k,v) pair identified by Locator loc."""
43      j = loc._index
44      if not (0 <= j < len(self) and self._data[j] is loc):
45        raise ValueError('Invalid locator')
46      if j == len(self) - 1:                  # item at last position
47        self._data.pop( )                     # just remove it
48      else:
49        self._swap(j, len(self)-1)            # swap item to the last position
50        self._data.pop( )                     # remove it from the list
51        self._bubble(j)                       # fix item displaced by the swap
52      return (loc._key, loc._value)
```

**Code Fragment 9.9:** An implementation of an adaptable priority queue (continued from Code Fragment 9.8).

| Operation | Running Time |
|---|---|
| len(P), P.is_empty( ), P.min( ) | $O(1)$ |
| P.add(k,v) | $O(\log n)^*$ |
| P.update(loc, k, v) | $O(\log n)$ |
| P.remove(loc) | $O(\log n)^*$ |
| P.remove_min( ) | $O(\log n)^*$ |

*amortized with dynamic array

**Table 9.4:** Running times of the methods of an adaptable priority queue, $P$, of size $n$, realized by means of our array-based heap representation. The space requirement is $O(n)$.

## 9.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

### Reinforcement

**R-9.1** How long would it take to remove the $\lceil \log n \rceil$ smallest elements from a heap that contains $n$ entries, using the remove_min operation?

**R-9.2** Suppose you label each position $p$ of a binary tree $T$ with a key equal to its preorder rank. Under what circumstances is $T$ a heap?

**R-9.3** What does each remove_min call return within the following sequence of priority queue ADT methods: add(5,A), add(4,B), add(7,F), add(1,D), remove_min( ), add(3,J), add(6,L), remove_min( ), remove_min( ), add(8,G), remove_min( ), add(2,H), remove_min( ), remove_min( )?

**R-9.4** An airport is developing a computer simulation of air-traffic control that handles events such as landings and takeoffs. Each event has a *time stamp* that denotes the time when the event will occur. The simulation program needs to efficiently perform the following two fundamental operations:

- Insert an event with a given time stamp (that is, add a future event).
- Extract the event with smallest time stamp (that is, determine the next event to process).

Which data structure should be used for the above operations? Why?

**R-9.5** The min method for the UnsortedPriorityQueue class executes in $O(n)$ time, as analyzed in Table 9.2. Give a simple modification to the class so that min runs in $O(1)$ time. Explain any necessary modifications to other methods of the class.

**R-9.6** Can you adapt your solution to the previous problem to make remove_min run in $O(1)$ time for the UnsortedPriorityQueue class? Explain your answer.

**R-9.7** Illustrate the execution of the selection-sort algorithm on the following input sequence: $(22, 15, 36, 44, 10, 3, 9, 13, 29, 25)$.

**R-9.8** Illustrate the execution of the insertion-sort algorithm on the input sequence of the previous problem.

**R-9.9** Give an example of a worst-case sequence with $n$ elements for insertion-sort, and show that insertion-sort runs in $\Omega(n^2)$ time on such a sequence.

**R-9.10** At which positions of a heap might the third smallest key be stored?

**R-9.11** At which positions of a heap might the largest key be stored?

**R-9.12** Consider a situation in which a user has numeric keys and wishes to have a priority queue that is *maximum-oriented*. How could a standard (min-oriented) priority queue be used for such a purpose?

**R-9.13** Illustrate the execution of the in-place heap-sort algorithm on the following input sequence: $(2,5,16,4,10,23,39,18,26,15)$.

**R-9.14** Let $T$ be a complete binary tree such that position $p$ stores an element with key $f(p)$, where $f(p)$ is the level number of $p$ (see Section 8.3.2). Is tree $T$ a heap? Why or why not?

**R-9.15** Explain why the description of down-heap bubbling does not consider the case in which position $p$ has a right child but not a left child.

**R-9.16** Is there a heap $H$ storing seven entries with distinct keys such that a preorder traversal of $H$ yields the entries of $H$ in increasing or decreasing order by key? How about an inorder traversal? How about a postorder traversal? If so, give an example; if not, say why.

**R-9.17** Let $H$ be a heap storing 15 entries using the array-based representation of a complete binary tree. What is the sequence of indices of the array that are visited in a preorder traversal of $H$? What about an inorder traversal of $H$? What about a postorder traversal of $H$?

**R-9.18** Show that the sum

$$\sum_{i=1}^{n} \log i,$$

which appears in the analysis of heap-sort, is $\Omega(n \log n)$.

**R-9.19** Bill claims that a preorder traversal of a heap will list its keys in nondecreasing order. Draw an example of a heap that proves him wrong.

**R-9.20** Hillary claims that a postorder traversal of a heap will list its keys in nonincreasing order. Draw an example of a heap that proves her wrong.

**R-9.21** Show all the steps of the algorithm for removing the entry $(16,X)$ from the heap of Figure 9.1, assuming the entry had been identified with a locator.

**R-9.22** Show all the steps of the algorithm for replacing key of entry $(5,A)$ with 18 in the heap of Figure 9.1, assuming the entry had been identified with a locator.

**R-9.23** Draw an example of a heap whose keys are all the odd numbers from 1 to 59 (with no repeats), such that the insertion of an entry with key 32 would cause up-heap bubbling to proceed all the way up to a child of the root (replacing that child's key with 32).

**R-9.24** Describe a sequence of $n$ insertions in a heap that requires $\Omega(n \log n)$ time to process.

**R-9.25** Complete Figure 9.9 by showing all the steps of the in-place heap-sort algorithm. Show both the array and the associated heap at the end of each step.

## Creativity

**C-9.26** Show how to implement the stack ADT using only a priority queue and one additional integer instance variable.

**C-9.27** Show how to implement the FIFO queue ADT using only a priority queue and one additional integer instance variable.

**C-9.28** Professor Idle suggests the following solution to the previous problem. Whenever an item is inserted into the queue, it is assigned a key that is equal to the current size of the queue. Does such a strategy result in FIFO semantics? Prove that it is so or provide a counterexample.

**C-9.29** Reimplement the SortedPriorityQueue using a Python list. Make sure to maintain remove_min's $O(1)$ performance.

**C-9.30** Give a nonrecursive implementation of the _upheap method for the class HeapPriorityQueue.

**C-9.31** Give a nonrecursive implementation of the _downheap method for the class HeapPriorityQueue.

**C-9.32** Assume that we are using a linked representation of a complete binary tree $T$, and an extra reference to the last node of that tree. Show how to update the reference to the last node after operations add or remove_min in $O(\log n)$ time, where $n$ is the current number of nodes of $T$. Be sure and handle all possible cases, as illustrated in Figure 9.12.

**C-9.33** When using a linked-tree representation for a heap, an alternative method for finding the last node during an insertion in a heap $T$ is to store, in the last node and each leaf node of $T$, a reference to the leaf node immediately to its right (wrapping to the first node in the next lower level for the rightmost leaf node). Show how to maintain such references in $O(1)$ time per operation of the priority queue ADT assuming that $T$ is implemented with a linked structure.

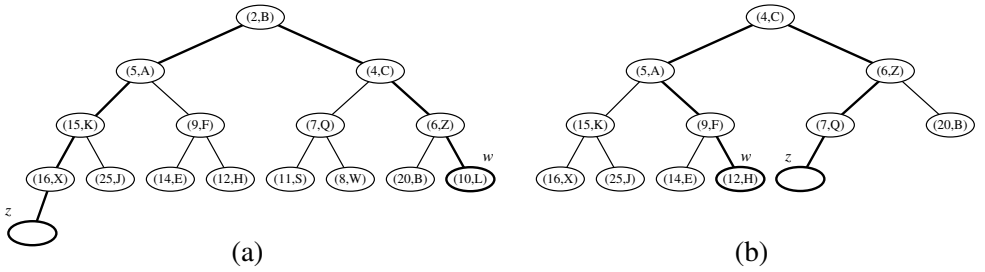



**Figure 9.12:** Updating the last node in a complete binary tree after operation add or remove. Node $w$ is the last node before operation add or after operation remove. Node $z$ is the last node after operation add or before operation remove.

**C-9.34** We can represent a path from the root to a given node of a binary tree by means of a binary string, where 0 means "go to the left child" and 1 means "go to the right child." For example, the path from the root to the node storing $(8,W)$ in the heap of Figure 9.12a is represented by "101." Design an $O(\log n)$-time algorithm for finding the last node of a complete binary tree with $n$ nodes, based on the above representation. Show how this algorithm can be used in the implementation of a complete binary tree by means of a linked structure that does not keep a reference to the last node.

**C-9.35** Given a heap $T$ and a key $k$, give an algorithm to compute all the entries in $T$ having a key less than or equal to $k$. For example, given the heap of Figure 9.12a and query $k = 7$, the algorithm should report the entries with keys 2, 4, 5, 6, and 7 (but not necessarily in this order). Your algorithm should run in time proportional to the number of entries returned, and should *not* modify the heap

**C-9.36** Provide a justification of the time bounds in Table 9.4.

**C-9.37** Give an alternative analysis of bottom-up heap construction by showing the following summation is $O(1)$, for any positive integer $h$:

$$\sum_{i=1}^{h} \left(i/2^i\right).$$

**C-9.38** Suppose two binary trees, $T_1$ and $T_2$, hold entries satisfying the heap-order property (but not necessarily the complete binary tree property). Describe a method for combining $T_1$ and $T_2$ into a binary tree $T$, whose nodes hold the union of the entries in $T_1$ and $T_2$ and also satisfy the heap-order property. Your algorithm should run in time $O(h_1 + h_2)$ where $h_1$ and $h_2$ are the respective heights of $T_1$ and $T_2$.

**C-9.39** Implement a heappushpop method for the HeapPriorityQueue class, with semantics akin to that described for the heapq module in Section 9.3.7.

**C-9.40** Implement a heapreplace method for the HeapPriorityQueue class, with semantics akin to that described for the heapq module in Section 9.3.7.

**C-9.41** Tamarindo Airlines wants to give a first-class upgrade coupon to their top $\log n$ frequent flyers, based on the number of miles accumulated, where $n$ is the total number of the airlines' frequent flyers. The algorithm they currently use, which runs in $O(n \log n)$ time, sorts the flyers by the number of miles flown and then scans the sorted list to pick the top $\log n$ flyers. Describe an algorithm that identifies the top $\log n$ flyers in $O(n)$ time.

**C-9.42** Explain how the $k$ largest elements from an unordered collection of size $n$ can be found in time $O(n + k \log n)$ using a maximum-oriented heap.

**C-9.43** Explain how the $k$ largest elements from an unordered collection of size $n$ can be found in time $O(n \log k)$ using $O(k)$ auxiliary space.

**C-9.44** Given a class, PriorityQueue, that implements the minimum-oriented priority queue ADT, provide an implementation of a MaxPriorityQueue class that adapts to provide a maximum-oriented abstraction with methods add, max, and remove_max. Your implementation should not make any assumption about the internal workings of the original PriorityQueue class, nor the type of keys that might be used.

**C-9.45** Write a key function for nonnegative integers that determines order based on the number of 1's in each integer's binary expansion.

**C-9.46** Give an alternative implementation of the pq_sort function, from Code Fragment 9.7, that accepts a key function as an optional parameter.

**C-9.47** Describe an in-place version of the selection-sort algorithm for an array that uses only $O(1)$ space for instance variables in addition to the array.

**C-9.48** Assuming the input to the sorting problem is given in an array $A$, describe how to implement the insertion-sort algorithm using only the array $A$ and at most a constant number of additional variables.

**C-9.49** Give an alternate description of the in-place heap-sort algorithm using the standard minimum-oriented priority queue (instead of a maximum-oriented one).

**C-9.50** An online computer system for trading stocks needs to process orders of the form "buy 100 shares at \$$x$ each" or "sell 100 shares at \$$y$ each." A buy order for \$$x$ can only be processed if there is an existing sell order with price \$$y$ such that $y \leq x$. Likewise, a sell order for \$$y$ can only be processed if there is an existing buy order with price \$$x$ such that $y \leq x$. If a buy or sell order is entered but cannot be processed, it must wait for a future order that allows it to be processed. Describe a scheme that allows buy and sell orders to be entered in $O(\log n)$ time, independent of whether or not they can be immediately processed.

**C-9.51** Extend a solution to the previous problem so that users are allowed to update the prices for their buy or sell orders that have yet to be processed.

**C-9.52** A group of children want to play a game, called ***Unmonopoly***, where in each turn the player with the most money must give half of his/her money to the player with the least amount of money. What data structure(s) should be used to play this game efficiently? Why?

## Projects

**P-9.53** Implement the in-place heap-sort algorithm. Experimentally compare its running time with that of the standard heap-sort that is not in-place.

**P-9.54** Use the approach of either Exercise C-9.42 or C-9.43 to reimplement the top method of the FavoritesListMTF class from Section 7.6.2. Make sure that results are generated from largest to smallest.

**P-9.55** Write a program that can process a sequence of stock buy and sell orders as described in Exercise C-9.50.

**P-9.56** Let $S$ be a set of $n$ points in the plane with distinct integer $x$- and $y$-coordinates. Let $T$ be a complete binary tree storing the points from $S$ at its external nodes, such that the points are ordered left to right by increasing $x$-coordinates. For each node $v$ in $T$, let $S(v)$ denote the subset of $S$ consisting of points stored in the subtree rooted at $v$. For the root $r$ of $T$, define $top(r)$ to be the point in $S = S(r)$ with maximum $y$-coordinate. For every other node $v$, define $top(r)$ to be the point in $S$ with highest $y$-coordinate in $S(v)$ that is not also the highest $y$-coordinate in $S(u)$, where $u$ is the parent of $v$ in $T$ (if such a point exists). Such labeling turns $T$ into a ***priority search tree***. Describe a linear-time algorithm for turning $T$ into a priority search tree. Implement this approach.

**P-9.57** One of the main applications of priority queues is in operating systems—for ***scheduling jobs*** on a CPU. In this project you are to build a program that schedules simulated CPU jobs. Your program should run in a loop, each iteration of which corresponds to a ***time slice*** for the CPU. Each job is assigned a priority, which is an integer between $-20$ (highest priority) and 19 (lowest priority), inclusive. From among all jobs waiting to be processed in a time slice, the CPU must work on a job with highest priority. In this simulation, each job will also come with a ***length*** value, which is an integer between 1 and 100, inclusive, indicating the number of time slices that are needed to process this job. For simplicity, you may assume jobs cannot be interrupted—once it is scheduled on the CPU, a job runs for a number of time slices equal to its length. Your simulator must output the name of the job running on the CPU in each time slice and must process a sequence of commands, one per time slice, each of which is of the form "add job *name* with length $n$ and priority $p$" or "no new job this slice".

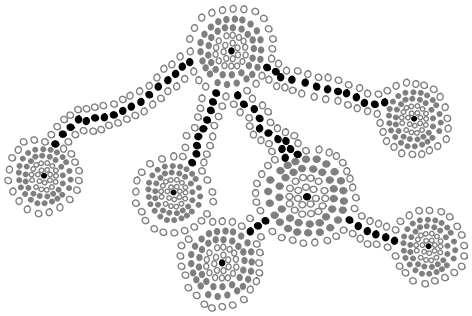
**P-9.58** Develop a Python implementation of an adaptable priority queue that is based on an unsorted list and supports location-aware entries.

# Chapter Notes

Knuth's book on sorting and searching [65] describes the motivation and history for the selection-sort, insertion-sort, and heap-sort algorithms. The heap-sort algorithm is due to Williams [103], and the linear-time heap construction algorithm is due to Floyd [39]. Additional algorithms and analyses for heaps and heap-sort variations can be found in papers by Bentley [15], Carlsson [24], Gonnet and Munro [45], McDiarmid and Reed [74], and Schaffer and Sedgewick [88].

Chapter

# 11 Search Trees



## Contents


**11.1 Binary Search Trees . . . . . 460**
11.1.1 Navigating a Binary Search Tree . . . . . 461
11.1.2 Searches . . . . . 463
11.1.3 Insertions and Deletions . . . . . 465
11.1.4 Python Implementation . . . . . 468
11.1.5 Performance of a Binary Search Tree . . . . . 473
**11.2 Balanced Search Trees . . . . . 475**
11.2.1 Python Framework for Balancing Search Trees . . . . . 478
**11.3 AVL Trees . . . . . 481**
11.3.1 Update Operations . . . . . 483
11.3.2 Python Implementation . . . . . 488
**11.4 Splay Trees . . . . . 490**
11.4.1 Splaying . . . . . 490
11.4.2 When to Splay . . . . . 494
11.4.3 Python Implementation . . . . . 496
11.4.4 Amortized Analysis of Splaying ★ . . . . . 497
**11.5 (2,4) Trees . . . . . 502**
11.5.1 Multiway Search Trees . . . . . 502
11.5.2 (2,4)-Tree Operations . . . . . 505
**11.6 Red-Black Trees . . . . . 512**
11.6.1 Red-Black Tree Operations . . . . . 514
11.6.2 Python Implementation . . . . . 525
**11.7 Exercises . . . . . 528**

## 11.1 Binary Search Trees

In Chapter 8 we introduced the tree data structure and demonstrated a variety of applications. One important use is as a ***search tree*** (as described on page 332). In this chapter, we use a search tree structure to efficiently implement a ***sorted map***. The three most fundamental methods of a map $M$ (see Section 10.1.1) are:

**M[k]:** Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError; implemented with __getitem__ method.

**M[k] = v:** Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k; implemented with __setitem__ method.

**del M[k]:** Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError; implemented with __delitem__ method.

The sorted map ADT includes additional functionality (see Section 10.3), guaranteeing that an iteration reports keys in sorted order, and supporting additional searches such as find_gt(k) and find_range(start, stop).

Binary trees are an excellent data structure for storing items of a map, assuming we have an order relation defined on the keys. In this context, a ***binary search tree*** is a binary tree $T$ with each position $p$ storing a key-value pair $(k, v)$ such that:

- Keys stored in the left subtree of $p$ are less than $k$.
- Keys stored in the right subtree of $p$ are greater than $k$.

An example of such a binary search tree is given in Figure 11.1. As a matter of convenience, we will not diagram the values associated with keys in this chapter, since those values do not affect the placement of items within a search tree.

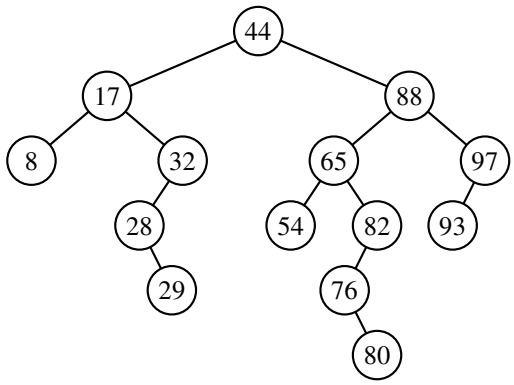



**Figure 11.1:** A binary search tree with integer keys. We omit the display of associated values in this chapter, since they are not relevant to the order of items within a search tree.

### 11.1.1 Navigating a Binary Search Tree

We begin by demonstrating that a binary search tree hierarchically represents the sorted order of its keys. In particular, the structural property regarding the placement of keys within a binary search tree assures the following important consequence regarding an ***inorder traversal*** (Section 8.4.3) of the tree.

**Proposition 11.1:** *An inorder traversal of a binary search tree visits positions in increasing order of their keys.*

**Justification:** We prove this by induction on the size of a subtree. If a subtree has at most one item, its keys are trivially visited in order. More generally, an inorder traversal of a (sub)tree consists of a recursive traversal of the (possibly empty) left subtree, followed by a visit of the root, and then a recursive traversal of the (possibly empty) right subtree. By induction, a recursive inorder traversal of the left subtree will produce an iteration of the keys in that subtree in increasing order. Furthermore, by the binary search tree property, all keys in the left subtree have keys strictly smaller than that of the root. Therefore, visiting the root just after that subtree extends the increasing order of keys. Finally, by the search tree property, all keys in the right subtree are strictly greater than the root, and by induction, an inorder traversal of that subtree will visit those keys in increasing order. ■

Since an inorder traversal can be executed in linear time, a consequence of this proposition is that we can produce a sorted iteration of the keys of a map in linear time, when represented as a binary search tree.

Although an inorder traversal is typically expressed using a top-down recursion, we can provide nonrecursive descriptions of operations that allow more fine-grained navigation among the positions of a binary search relative to the order of their keys. Our generic binary tree ADT from Chapter 8 is defined as a positional structure, allowing direct navigation using methods such as `parent(p)`, `left(p)`, and `right(p)`. With a binary *search* tree, we can provide additional navigation based on the natural order of the keys stored in the tree. In particular, we can support the following methods, akin to those provided by a `PositionalList` (Section 7.4.1).

**first( ):** Return the position containing the least key, or `None` if the tree is empty.

**last( ):** Return the position containing the greatest key, or `None` if empty tree.

**before(p):** Return the position containing the greatest key that is less than that of position `p` (i.e., the position that would be visited immediately before `p` in an inorder traversal), or `None` if `p` is the first position.

**after(p):** Return the position containing the least key that is greater than that of position `p` (i.e., the position that would be visited immediately after `p` in an inorder traversal), or `None` if `p` is the last position.

The "first" position of a binary search tree can be located by starting a walk at the root and continuing to the left child, as long as a left child exists. By symmetry, the last position is reached by repeated steps rightward starting at the root.

The successor of a position, after(p), is determined by the following algorithm.

```
Algorithm after(p):
    if right(p) is not None then {successor is leftmost position in p's right subtree}
      walk = right(p)
      while left(walk) is not None do
        walk = left(walk)
      return walk
    else {successor is nearest ancestor having p in its left subtree}
      walk = p
      ancestor = parent(walk)
      while ancestor is not None and walk == right(ancestor) do
        walk = ancestor
        ancestor = parent(walk)
      return ancestor
```

**Code Fragment 11.1:** Computing the successor of a position in a binary search tree.

The rationale for this process is based purely on the workings of an inorder traversal, given the correspondence of Proposition 11.1. If p has a right subtree, that right subtree is recursively traversed immediately after p is visited, and so the first position to be visited after p is the *leftmost* position within the right subtree. If p does not have a right subtree, then the flow of control of an inorder traversal returns to p's parent. If p were in the *right* subtree of that parent, then the parent's subtree traversal is complete and the flow of control progresses to its parent and so on. Once an ancestor is reached in which the recursion is returning from its *left* subtree, then that ancestor becomes the next position visited by the inorder traversal, and thus is the successor of p. Notice that the only case in which no such ancestor is found is when p was the rightmost (last) position of the full tree, in which case there is no successor.

A symmetric algorithm can be defined to determine the predecessor of a position, before(p). At this point, we note that the running time of single call to after(p) or before(p) is bounded by the height $h$ of the full tree, because it is found after either a single downward walk or a single upward walk. While the worst-case running time is $O(h)$, we note that either of these methods run in $O(1)$ ***amortized*** time, in that series of $n$ calls to after(p) starting at the first position will execute in a total of $O(n)$ time. We leave a formal justification of this fact to Exercise C-11.34, but intuitively the upward and downward paths mimic steps of the inorder traversal (a related argument was made in the justification of Proposition 9.3).

### 11.1.2 Searches

The most important consequence of the structural property of a binary search tree is its namesake search algorithm. We can attempt to locate a particular key in a binary search tree by viewing it as a decision tree (recall Figure 8.7). In this case, the question asked at each position p is whether the desired key k is less than, equal to, or greater than the key stored at position p, which we denote as p.key( ). If the answer is "less than," then the search continues in the left subtree. If the answer is "equal," then the search terminates successfully. If the answer is "greater than," then the search continues in the right subtree. Finally, if we reach an empty subtree, then the search terminates unsuccessfully. (See Figure 11.2.)

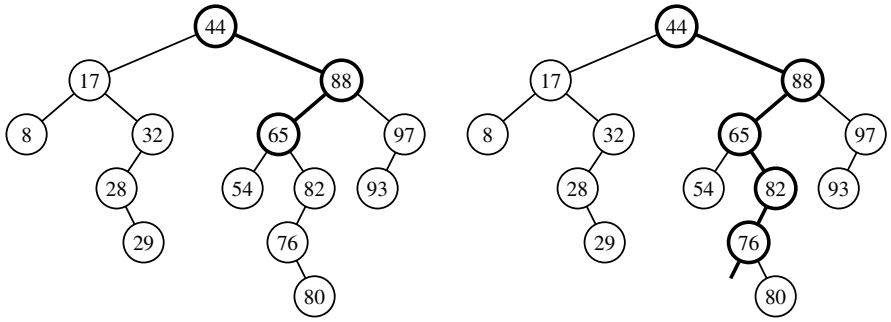



**Figure 11.2:** (a) A successful search for key 65 in a binary search tree; (b) an unsuccessful search for key 68 that terminates because there is no subtree to the left of the key 76.

We describe this approach in Code Fragment 11.2. If key k occurs in a subtree rooted at p, a call to TreeSearch(T, p, k) results in the position at which the key is found; in this case, the __getitem__ map operation would return the associated value at that position. In the event of an unsuccessful search, the TreeSearch algorithm returns the final position explored on the search path (which we will later make use of when determining where to insert a new item in a search tree).

```
Algorithm TreeSearch(T, p, k):
    if k == p.key() then
        return p                                        {successful search}
    else if k < p.key() and T.left(p) is not None then
        return TreeSearch(T, T.left(p), k)              {recur on left subtree}
    else if k > p.key() and T.right(p) is not None then
        return TreeSearch(T, T.right(p), k)             {recur on right subtree}
    return p                                            {unsuccessful search}
```

**Code Fragment 11.2:** Recursive search in a binary search tree.

## Analysis of Binary Tree Searching

The analysis of the worst-case running time of searching in a binary search tree $T$ is simple. Algorithm TreeSearch is recursive and executes a constant number of primitive operations for each recursive call. Each recursive call of TreeSearch is made on a child of the previous position. That is, TreeSearch is called on the positions of a path of $T$ that starts at the root and goes down one level at a time. Thus, the number of such positions is bounded by $h+1$, where $h$ is the height of $T$. In other words, since we spend $O(1)$ time per position encountered in the search, the overall search runs in $O(h)$ time, where $h$ is the height of the binary search tree $T$. (See Figure 11.3.)

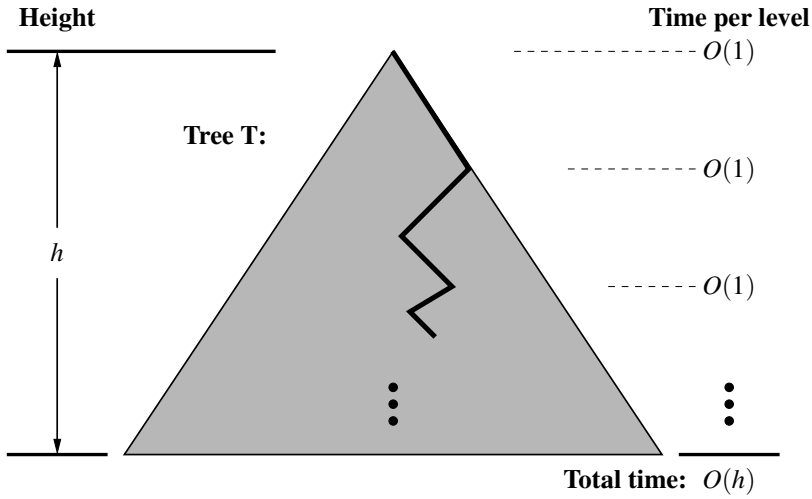



**Figure 11.3:** Illustrating the running time of searching in a binary search tree. The figure uses standard caricature of a binary search tree as a big triangle and a path from the root as a zig-zag line.

In the context of the sorted map ADT, the search will be used as a subroutine for implementing the __getitem__ method, as well as for the __setitem__ and __delitem__ methods, since each of these begins by trying to locate an existing item with a given key. To implement sorted map operations such as find_lt and find_gt, we will combine this search with traversal methods before and after. All of these operations will run in worst-case $O(h)$ time for a tree with height $h$. We can use a variation of this technique to implement the find_range method in time $O(s+h)$, where $s$ is the number of items reported (see Exercise C-11.34).

Admittedly, the height $h$ of $T$ can be as large as the number of entries, $n$, but we expect that it is usually much smaller. Indeed, later in this chapter we show various strategies to maintain an upper bound of $O(\log n)$ on the height of a search tree $T$.

### 11.1.3 Insertions and Deletions

Algorithms for inserting or deleting entries of a binary search tree are fairly straightforward, although not trivial.

#### Insertion

The map command M[k] = v, as supported by the __setitem__ method, begins with a search for key k (assuming the map is nonempty). If found, that item's existing value is reassigned. Otherwise, a node for the new item can be inserted into the underlying tree T in place of the empty subtree that was reached at the end of the failed search. The binary search tree property is sustained by that placement (note that it is placed exactly where a search would expect it). Pseudo-code for such a TreeInsert algorithm is given in in Code Fragment 11.3.

**Algorithm** TreeInsert(T, k, v):
*Input:* A search key k to be associated with value v
p = TreeSearch(T, T.root(), k)
**if** k == p.key() **then**
  Set p's value to v
**else if** k < p.key() **then**
  add node with item (k,v) as left child of p
**else**
  add node with item (k,v) as right child of p

**Code Fragment 11.3:** Algorithm for inserting a key-value pair into a map that is represented as a binary search tree.

An example of insertion into a binary search tree is shown in Figure 11.4.

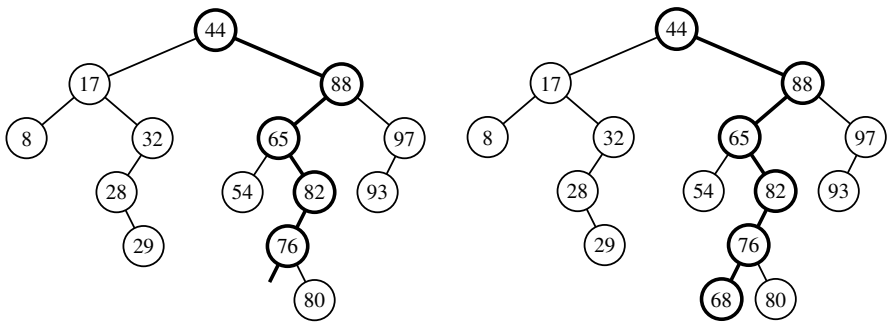



**Figure 11.4:** Insertion of an item with key 68 into the search tree of Figure 11.2. Finding the position to insert is shown in (a), and the resulting tree is shown in (b).

### Deletion

Deleting an item from a binary search tree T is a bit more complex than inserting a new item because the location of the deletion might be anywhere in the tree. (In contrast, insertions are always enacted at the bottom of a path.) To delete an item with key k, we begin by calling TreeSearch(T, T.root( ), k) to find the position p of T storing an item with key equal to k. If the search is successful, we distinguish between two cases (of increasing difficulty):

- If p has at most one child, the deletion of the node at position p is easily implemented. When introducing update methods for the LinkedBinaryTree class in Section 8.3.1, we declared a nonpublic utility, _delete(p), that deletes a node at position p and replaces it with its child (if any), presuming that p has at most one child. That is precisely the desired behavior. It removes the item with key k from the map while maintaining all other ancestor-descendant relationships in the tree, thereby assuring the upkeep of the binary search tree property. (See Figure 11.5.)
- If position p has two children, we cannot simply remove the node from T since this would create a "hole" and two orphaned children. Instead, we proceed as follows (see Figure 11.6):
  - We locate position r containing the item having the greatest key that is strictly less than that of position p, that is, r = before(p) by the notation of Section 11.1.1. Because p has two children, its predecessor is the rightmost position of the left subtree of p.
  - We use r's item as a replacement for the one being deleted at position p. Because r has the immediately preceding key in the map, any items in p's right subtree will have keys greater than r and any other items in p's left subtree will have keys less than r. Therefore, the binary search tree property is satisfied after the replacement.
  - Having used r's as a replacement for p, we instead delete the node at position r from the tree. Fortunately, since r was located as the rightmost position in a subtree, r does not have a right child. Therefore, its deletion can be performed using the first (and simpler) approach.

As with searching and insertion, this algorithm for a deletion involves the traversal of a single path downward from the root, possibly moving an item between two positions of this path, and removing a node from that path and promoting its child. Therefore, it executes in time $O(h)$ where $h$ is the height of the tree.

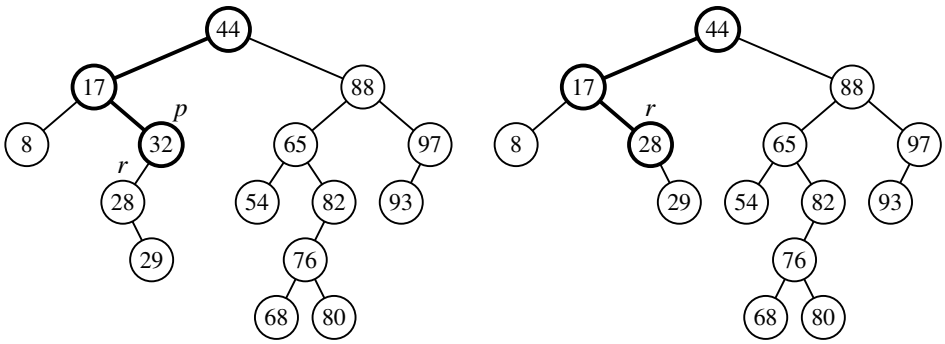



**Figure 11.5:** Deletion from the binary search tree of Figure 11.4b, where the item to delete (with key 32) is stored at a position $p$ with one child $r$: (a) before the deletion; (b) after the deletion.

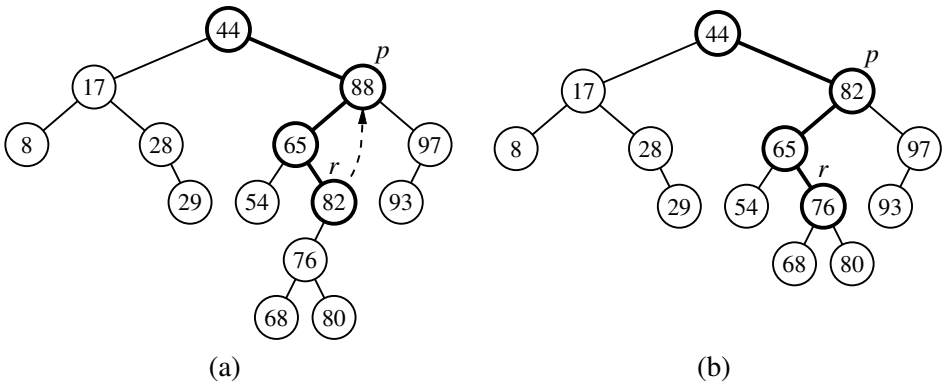



**Figure 11.6:** Deletion from the binary search tree of Figure 11.5b, where the item to delete (with key 88) is stored at a position $p$ with two children, and replaced by its predecessor $r$: (a) before the deletion; (b) after the deletion.

### 11.1.4 Python Implementation

In Code Fragments 11.4 through 11.8 we define a TreeMap class that implements the sorted map ADT using a binary search tree. In fact, our implementation is more general. We support all of the standard map operations (Section 10.1.1), all additional sorted map operations (Section 10.3), and positional operations including first( ), last( ), find_position(k), before(p), after(p), and delete(p).

Our TreeMap class takes advantage of ***multiple inheritance*** for code reuse, inheriting from the LinkedBinaryTree class of Section 8.3.1 for our representation as a positional binary tree, and from the MapBase class from Code Fragment 10.2 of Section 10.1.4 to provide us with the key-value composite item and the concrete behaviors from the collections.MutableMapping abstract base class. We subclass the nested Position class to support more specific p.key( ) and p.value( ) accessors for our map, rather than the p.element( ) syntax inherited from the tree ADT.

We define several nonpublic utilities, most notably a _subtree_search(p, k) method that corresponds to the TreeSearch algorithm of Code Fragment 11.2. That returns a position, ideally one that contains the key k, or otherwise the last position that is visited on the search path. We rely on the fact that the final position during an unsuccessful search is either the nearest key less than k or the nearest key greater than k. This search utility becomes the basis for the public find_position(k) method, and also for internal use when searching, inserting, or deleting items from a map, as well as for the robust searches of the sorted map ADT.

When making structural modifications to the tree, we rely on nonpublic update methods, such as _add_right, that are inherited from the LinkedBinaryTree class (see Section 8.3.1). It is important that these inherited methods remain nonpublic, as the search tree property could be violated through misuse of such operations.

Finally, we note that our code is peppered with calls to presumed methods named _rebalance_insert, _rebalance_delete, and _rebalance_access. These methods serve as ***hooks*** for future use when balancing search trees; we discuss them in Section 11.2. We conclude with a brief guide to the organization of our code.

Code Fragment 11.4: Beginning of TreeMap class including redefined Position class and nonpublic search utilities.

Code Fragment 11.5: Positional methods first( ), last( ), before(p), after(p), and find_position(p) accessor.

Code Fragment 11.6: Selected methods of the sorted map ADT: find_min( ), find_ge(k), and find_range(start, stop); related methods are omitted for the sake of brevity.

Code Fragment 11.7: __getitem__(k), __setitem__(k, v), and __iter__( ).

Code Fragment 11.8: Deletion either by position, as delete(p), or by key, as __delitem__(k).

```
class TreeMap(LinkedBinaryTree, MapBase):
  """Sorted map implementation using a binary search tree."""

  #---------------------------- override Position class ----------------------------
  class Position(LinkedBinaryTree.Position):
    def key(self):
      """Return key of map's key-value pair."""
      return self.element()._key

    def value(self):
      """Return value of map's key-value pair."""
      return self.element()._value

  #------------------------------ nonpublic utilities ------------------------------
  def _subtree_search(self, p, k):
    """Return Position of p's subtree having key k, or last node searched."""
    if k == p.key():                                  # found match
      return p
    elif k < p.key():                                 # search left subtree
      if self.left(p) is not None:
        return self._subtree_search(self.left(p), k)
    else:                                             # search right subtree
      if self.right(p) is not None:
        return self._subtree_search(self.right(p), k)
    return p                                          # unsucessful search

  def _subtree_first_position(self, p):
    """Return Position of first item in subtree rooted at p."""
    walk = p
    while self.left(walk) is not None:                # keep walking left
      walk = self.left(walk)
    return walk

  def _subtree_last_position(self, p):
    """Return Position of last item in subtree rooted at p."""
    walk = p
    while self.right(walk) is not None:               # keep walking right
      walk = self.right(walk)
    return walk
```

**Code Fragment 11.4:** Beginning of a TreeMap class based on a binary search tree.

```
40    def first(self):
41      """Return the first Position in the tree (or None if empty)."""
42      return self._subtree_first_position(self.root( )) if len(self) > 0 else None
43
44    def last(self):
45      """Return the last Position in the tree (or None if empty)."""
46      return self._subtree_last_position(self.root( )) if len(self) > 0 else None
47
48    def before(self, p):
49      """Return the Position just before p in the natural order.
50
51      Return None if p is the first position.
52      """
53      self._validate(p)                       # inherited from LinkedBinaryTree
54      if self.left(p):
55        return self._subtree_last_position(self.left(p))
56      else:
57        # walk upward
58        walk = p
59        above = self.parent(walk)
60        while above is not None and walk == self.left(above):
61          walk = above
62          above = self.parent(walk)
63        return above
64
65    def after(self, p):
66      """Return the Position just after p in the natural order.
67
68      Return None if p is the last position.
69      """
70      # symmetric to before(p)
71
72    def find_position(self, k):
73      """Return position with key k, or else neighbor (or None if empty)."""
74      if self.is_empty( ):
75        return None
76      else:
77        p = self._subtree_search(self.root( ), k)
78        self._rebalance_access(p)             # hook for balanced tree subclasses
79        return p
```

**Code Fragment 11.5:** Navigational methods of the TreeMap class.

```
80    def find_min(self):
81      """Return (key,value) pair with minimum key (or None if empty)."""
82      if self.is_empty( ):
83        return None
84      else:
85        p = self.first( )
86        return (p.key( ), p.value( ))
87
88    def find_ge(self, k):
89      """Return (key,value) pair with least key greater than or equal to k.
90
91      Return None if there does not exist such a key.
92      """
93      if self.is_empty( ):
94        return None
95      else:
96        p = self.find_position(k)                    # may not find exact match
97        if p.key( ) < k:                             # p's key is too small
98          p = self.after(p)
99        return (p.key( ), p.value( )) if p is not None else None
100
101   def find_range(self, start, stop):
102     """Iterate all (key,value) pairs such that start <= key < stop.
103
104     If start is None, iteration begins with minimum key of map.
105     If stop is None, iteration continues through the maximum key of map.
106     """
107     if not self.is_empty( ):
108       if start is None:
109         p = self.first( )
110       else:
111         # we initialize p with logic similar to find_ge
112         p = self.find_position(start)
113         if p.key( ) < start:
114           p = self.after(p)
115       while p is not None and (stop is None or p.key( ) < stop):
116         yield (p.key( ), p.value( ))
117         p = self.after(p)
```

**Code Fragment 11.6:** Some of the sorted map operations for the TreeMap class.

```
118   def __getitem__(self, k):
119     """Return value associated with key k (raise KeyError if not found)."""
120     if self.is_empty( ):
121       raise KeyError('Key Error: ' + repr(k))
122     else:
123       p = self._subtree_search(self.root( ), k)
124       self._rebalance_access(p)            # hook for balanced tree subclasses
125       if k != p.key( ):
126         raise KeyError('Key Error: ' + repr(k))
127       return p.value( )
128
129   def __setitem__(self, k, v):
130     """Assign value v to key k, overwriting existing value if present."""
131     if self.is_empty( ):
132       leaf = self._add_root(self._Item(k,v))          # from LinkedBinaryTree
133     else:
134       p = self._subtree_search(self.root( ), k)
135       if p.key( ) == k:
136         p.element( )._value = v              # replace existing item's value
137         self._rebalance_access(p)            # hook for balanced tree subclasses
138         return
139       else:
140         item = self._Item(k,v)
141         if p.key( ) < k:
142           leaf = self._add_right(p, item)  # inherited from LinkedBinaryTree
143         else:
144           leaf = self._add_left(p, item)   # inherited from LinkedBinaryTree
145     self._rebalance_insert(leaf)             # hook for balanced tree subclasses
146
147   def __iter__(self):
148     """Generate an iteration of all keys in the map in order."""
149     p = self.first( )
150     while p is not None:
151       yield p.key( )
152       p = self.after(p)
```

**Code Fragment 11.7:** Map operations for accessing and inserting items in the TreeMap class. Reverse iteration can be implemented with __reverse__, using symmetric approach to __iter__.

```
153   def delete(self, p):
154     """Remove the item at given Position."""
155     self._validate(p)                       # inherited from LinkedBinaryTree
156     if self.left(p) and self.right(p):      # p has two children
157       replacement = self._subtree_last_position(self.left(p))
158       self._replace(p, replacement.element())       # from LinkedBinaryTree
159       p =  replacement
160     # now p has at most one child
161     parent = self.parent(p)
162     self._delete(p)                         # inherited from LinkedBinaryTree
163     self._rebalance_delete(parent)          # if root deleted, parent is None
164
165   def __delitem__(self, k):
166     """Remove item associated with key k (raise KeyError if not found)."""
167     if not self.is_empty():
168       p = self._subtree_search(self.root(), k)
169       if k == p.key():
170         self.delete(p)                      # rely on positional version
171         return                              # successful deletion complete
172       self._rebalance_access(p)             # hook for balanced tree subclasses
173     raise KeyError('Key Error: ' + repr(k))
```

**Code Fragment 11.8:** Support for deleting an item from a TreeMap, located either by position or by key.

### 11.1.5 Performance of a Binary Search Tree

An analysis of the operations of our TreeMap class is given in Table 11.1. Almost all operations have a worst-case running time that depends on $h$, where $h$ is the height of the current tree. This is because most operations rely on a constant amount of work for each node along a particular path of the tree, and the maximum path length within a tree is proportional to the height of the tree. Most notably, our implementations of map operations __getitem__, __setitem__, and __delitem__ each begin with a call to the _subtree_search utility which traces a path downward from the root of the tree, using $O(1)$ time at each node to determine how to continue the search. Similar paths are traced when looking for a replacement during a deletion, or when computing a position's inorder predecessor or successor. We note that although a single call to the after method has worst-case running time of $O(h)$, the $n$ successive calls made during a call to __iter__ require a total of $O(n)$ time, since each edge is traced at most twice; in a sense, those calls have $O(1)$ amortized time bounds. A similar argument can be used to prove the $O(s+h)$ worst-case bound for a call to find_range that reports $s$ results (see Exercise C-11.34).

| Operation | Running Time |
|---:|---|
| k in T | $O(h)$ |
| T[k], T[k] = v | $O(h)$ |
| T.delete(p), del T[k] | $O(h)$ |
| T.find_position(k) | $O(h)$ |
| T.first( ), T.last( ), T.find_min( ), T.find_max( ) | $O(h)$ |
| T.before(p), T.after(p) | $O(h)$ |
| T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k) | $O(h)$ |
| T.find_range(start, stop) | $O(s+h)$ |
| iter(T), reversed(T) | $O(n)$ |

**Table 11.1:** Worst-case running times of the operations for a TreeMap T. We denote the current height of the tree with $h$, and the number of items reported by find_range as $s$. The space usage is $O(n)$, where $n$ is the number of items stored in the map.

A binary search tree T is therefore an efficient implementation of a map with $n$ entries only if its height is small. In the best case, T has height $h = \lceil \log(n+1) \rceil - 1$, which yields logarithmic-time performance for all the map operations. In the worst case, however, T has height $n$, in which case it would look and feel like an ordered list implementation of a map. Such a worst-case configuration arises, for example, if we insert items with keys in increasing or decreasing order. (See Figure 11.7.)

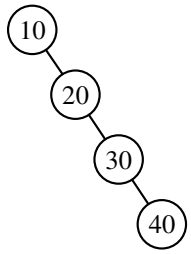



**Figure 11.7:** Example of a binary search tree with linear height, obtained by inserting entries with keys in increasing order.

We can nevertheless take comfort that, on average, a binary search tree with $n$ keys generated from a random series of insertions and removals of keys has expected height $O(\log n)$; the justification of this statement is beyond the scope of the book, requiring careful mathematical language to precisely define what we mean by a random series of insertions and removals, and sophisticated probability theory.

In applications where one cannot guarantee the random nature of updates, it is better to rely on variations of search trees, presented in the remainder of this chapter, that guarantee a ***worst-case*** height of $O(\log n)$, and thus $O(\log n)$ worst-case time for searches, insertions, and deletions.

## 11.2 Balanced Search Trees

In the closing of the previous section, we noted that if we could assume a random series of insertions and removals, the standard binary search tree supports $O(\log n)$ expected running times for the basic map operations. However, we may only claim $O(n)$ worst-case time, because some sequences of operations may lead to an unbalanced tree with height proportional to $n$.

In the remainder of this chapter, we explore four search tree algorithms that provide stronger performance guarantees. Three of the four data structures (AVL trees, splay trees, and red-black trees) are based on augmenting a standard binary search tree with occasional operations to reshape the tree and reduce its height.

The primary operation to rebalance a binary search tree is known as a ***rotation***. During a rotation, we "rotate" a child to be above its parent, as diagrammed in Figure 11.8.

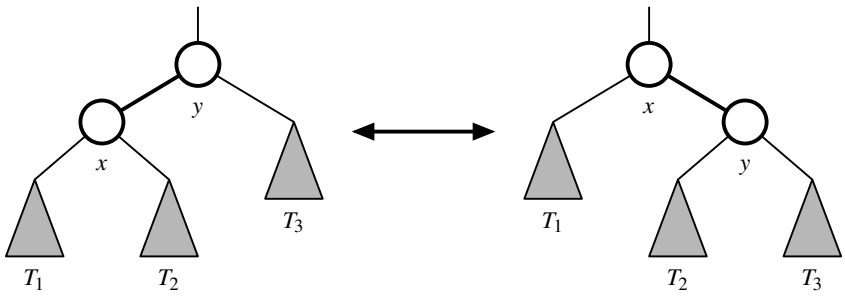



**Figure 11.8:** A rotation operation in a binary search tree. A rotation can be performed to transform the left formation into the right, or the right formation into the left. Note that all keys in subtree $T_1$ have keys less than that of position $x$, all keys in subtree $T_2$ have keys that are between those of positions $x$ and $y$, and all keys in subtree $T_3$ have keys that are greater than that of position $y$.

To maintain the binary search tree property through a rotation, we note that if position $x$ was a left child of position $y$ prior to a rotation (and therefore the key of $x$ is less than the key of $y$), then $y$ becomes the *right* child of $x$ after the rotation, and vice versa. Furthermore, we must relink the subtree of items with keys that lie between the keys of the two positions that are being rotated. For example, in Figure 11.8 the subtree labeled $T_2$ represents items with keys that are known to be greater than that of position $x$ and less than that of position $y$. In the first configuration of that figure, $T_2$ is the right subtree of position $x$; in the second configuration, it is the left subtree of position $y$.

Because a single rotation modifies a constant number of parent-child relationships, it can be implemented in $O(1)$ time with a linked binary tree representation.

In the context of a tree-balancing algorithm, a rotation allows the shape of a tree to be modified while maintaining the search tree property. If used wisely, this operation can be performed to avoid highly unbalanced tree configurations. For example, a rightward rotation from the first formation of Figure 11.8 to the second reduces the depth of each node in subtree $T_1$ by one, while increasing the depth of each node in subtree $T_3$ by one. (Note that the depth of nodes in subtree $T_2$ are unaffected by the rotation.)

One or more rotations can be combined to provide broader rebalancing within a tree. One such compound operation we consider is a ***trinode restructuring***. For this manipulation, we consider a position $x$, its parent $y$, and its grandparent $z$. The goal is to restructure the subtree rooted at $z$ in order to reduce the overall path length to $x$ and its subtrees. Pseudo-code for a restructure($x$) method is given in Code Fragment 11.9 and illustrated in Figure 11.9. In describing a trinode restructuring, we temporarily rename the positions $x$, $y$, and $z$ as $a$, $b$, and $c$, so that $a$ precedes $b$ and $b$ precedes $c$ in an inorder traversal of $T$. There are four possible orientations mapping $x$, $y$, and $z$ to $a$, $b$, and $c$, as shown in Figure 11.9, which are unified into one case by our relabeling. The trinode restructuring replaces $z$ with the node identified as $b$, makes the children of this node be $a$ and $c$, and makes the children of $a$ and $c$ be the four previous children of $x$, $y$, and $z$ (other than $x$ and $y$), while maintaining the inorder relationships of all the nodes in $T$.

**Algorithm** restructure(x):

***Input:*** A position x of a binary search tree T that has both a parent y and a grandparent z

***Output:*** Tree T after a trinode restructuring (which corresponds to a single or double rotation) involving positions x, y, and z

1: Let (a, b, c) be a left-to-right (inorder) listing of the positions x, y, and z, and let $(T_1, T_2, T_3, T_4)$ be a left-to-right (inorder) listing of the four subtrees of x, y, and z not rooted at x, y, or z.
2: Replace the subtree rooted at z with a new subtree rooted at b.
3: Let a be the left child of b and let $T_1$ and $T_2$ be the left and right subtrees of a, respectively.
4: Let c be the right child of b and let $T_3$ and $T_4$ be the left and right subtrees of c, respectively.

**Code Fragment 11.9:** The trinode restructuring operation in a binary search tree.

In practice, the modification of a tree $T$ caused by a trinode restructuring operation can be implemented through case analysis either as a single rotation (as in Figure 11.9a and b) or as a double rotation (as in Figure 11.9c and d). The double rotation arises when position $x$ has the middle of the three relevant keys and is first rotated above its parent, and then above what was originally its grandparent. In any of the cases, the trinode restructuring is completed with $O(1)$ running time.

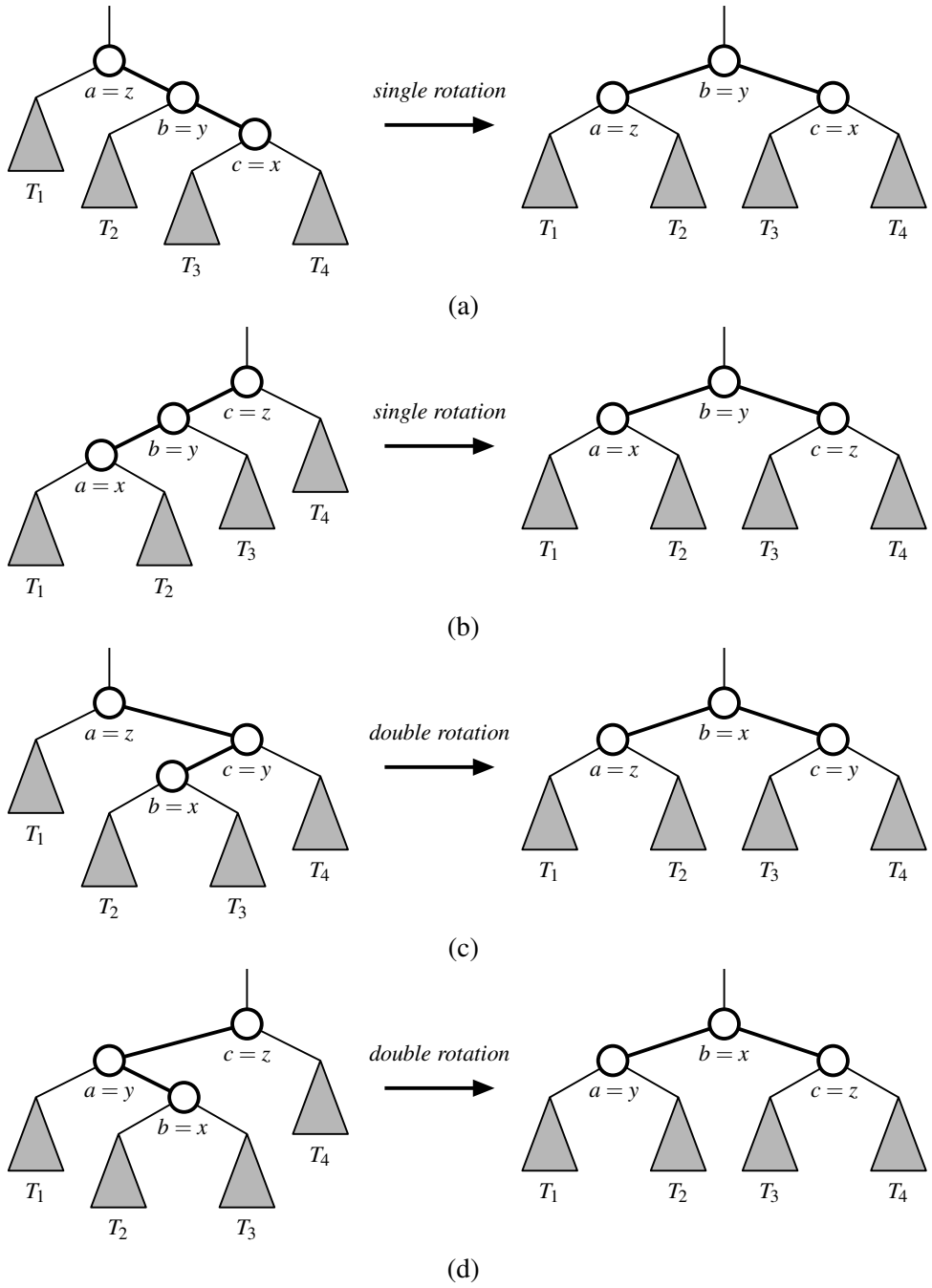



**Figure 11.9:** Schematic illustration of a trinode restructuring operation: (a and b) require a single rotation; (c and d) require a double rotation.

### 11.2.1 Python Framework for Balancing Search Trees

Our TreeMap class, introduced in Section 11.1.4, is a concrete map implementation that does not perform any explicit balancing operations. However, we designed that class to also serve as a base class for other subclasses that implement more advanced tree-balancing algorithms. A summary of our inheritance hierarchy is shown in Figure 11.10.

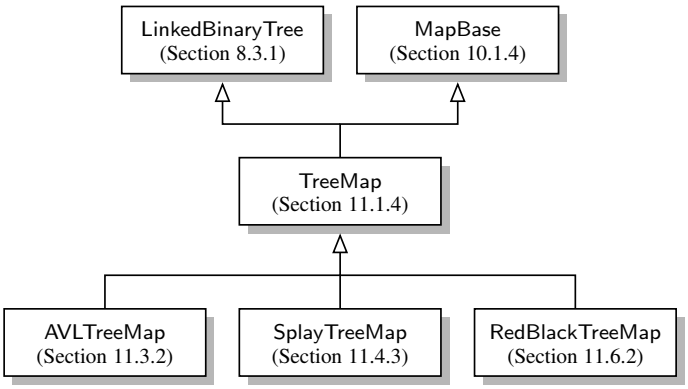



**Figure 11.10:** Our hierarchy of balanced search trees (with references to where they are defined). Recall that TreeMap inherits multiply from LinkedBinaryTree and MapBase.

#### Hooks for Rebalancing Operations

Our implementation of the basic map operations in Section 11.1.4 includes strategic calls to three nonpublic methods that serve as ***hooks*** for rebalancing algorithms:

- A call to _rebalance_insert(p) is made from within the __setitem__ method immediately after a new node is added to the tree at position *p*.
- A call to _rebalance_delete(p) is made each time a node has been deleted from the tree, with position *p* identifying the *parent* of the node that has just been removed. Formally, this hook is called from within the public delete(p) method, which is indirectly invoked by the public __delitem__(k) behavior.
- We also provide a hook, _rebalance_access(p), that is called when an item at position *p* of a tree is accessed through a public method such as __getitem__. This hook is used by the ***splay tree*** structure (see Section 11.4) to restructure a tree so that more frequently accessed items are brought closer to the root.

We provide trivial declarations of these three methods, in Code Fragment 11.10, having bodies that do nothing (using the **pass** statement). A subclass of TreeMap may override any of these methods to implement a nontrivial action to rebalance a tree. This is another example of the ***template method design pattern***, as seen in Section 8.4.6.

```
174    def _rebalance_insert(self, p): pass
175    def _rebalance_delete(self, p): pass
176    def _rebalance_access(self, p): pass
```

**Code Fragment 11.10:** Additional code for the TreeMap class (continued from Code Fragment 11.8), providing stubs for the rebalancing hooks.

### Nonpublic Methods for Rotating and Restructuring

A second form of support for balanced search trees is our inclusion of nonpublic utility methods _rotate and _restructure that, respectively, implement a single rotation and a trinode restructuring (described at the beginning of Section 11.2). Although these methods are not invoked by the public TreeMap operations, we promote code reuse by providing these implementation in this class so that they are inherited by all balanced-tree subclasses.

Our implementations are provided in Code Fragment 11.11. To simplify the code, we define an additional _relink utility that properly links parent and child nodes to each other, including the special case in which a "child" is a None reference. The focus of the _rotate method then becomes redefining the relationship between the parent and child, relinking a rotated node directly to its original grandparent, and shifting the "middle" subtree (that labeled as $T_2$ in Figure 11.8) between the rotated nodes. For the trinode restructuring, we determine whether to perform a single or double rotation, as originally described in Figure 11.9.

### Factory for Creating Tree Nodes

We draw attention to an important subtlety in the design of both our TreeMap class and the original LinkedBinaryTree subclass. The low-level definition of a node is provided by the nested _Node class within LinkedBinaryTree. Yet, several of our tree-balancing strategies require that auxiliary information be stored at each node to guide the balancing process. Those classes will override the nested _Node class to provide storage for an additional field.

Whenever we add a new node to the tree, as within the _add_right method of the LinkedBinaryTree (originally given in Code Fragment 8.10), we intentionally instantiate the node using the syntax self._Node, rather than the qualified name LinkedBinaryTree._Node. This is vital to our framework! When the expression self._Node is applied to an instance of a tree (sub)class, Python's name resolution follows the inheritance structure (as described in Section 2.5.2). If a subclass has overridden the definition for the _Node class, instantiation of self._Node relies on the newly defined node class. This technique is an example of the ***factory method design pattern***, as we provide a *subclass* the means to control the type of node that is created within methods of the *parent* class.

```
177   def _relink(self, parent, child, make_left_child):
178     """Relink parent node with child node (we allow child to be None)."""
179     if make_left_child:                          # make it a left child
180       parent._left = child
181     else:                                        # make it a right child
182       parent._right = child
183     if child is not None:                        # make child point to parent
184       child._parent = parent
185
186   def _rotate(self, p):
187     """Rotate Position p above its parent."""
188     x = p._node
189     y = x._parent                                # we assume this exists
190     z = y._parent                                # grandparent (possibly None)
191     if z is None:
192       self._root = x                             # x becomes root
193       x._parent = None
194     else:
195       self._relink(z, x, y == z._left)           # x becomes a direct child of z
196     # now rotate x and y, including transfer of middle subtree
197     if x == y._left:
198       self._relink(y, x._right, True)            # x._right becomes left child of y
199       self._relink(x, y, False)                  # y becomes right child of x
200     else:
201       self._relink(y, x._left, False)            # x._left becomes right child of y
202       self._relink(x, y, True)                   # y becomes left child of x
203
204   def _restructure(self, x):
205     """Perform trinode restructure of Position x with parent/grandparent."""
206     y = self.parent(x)
207     z = self.parent(y)
208     if (x == self.right(y)) == (y == self.right(z)):  # matching alignments
209       self._rotate(y)                            # single rotation (of y)
210       return y                                   # y is new subtree root
211     else:                                        # opposite alignments
212       self._rotate(x)                            # double rotation (of x)
213       self._rotate(x)
214       return x                                   # x is new subtree root
```

**Code Fragment 11.11:** Additional code for the TreeMap class (continued from Code Fragment 11.10), to provide nonpublic utilities for balanced search tree subclasses.

# 11.3 AVL Trees

The TreeMap class, which uses a standard binary search tree as its data structure, should be an efficient map data structure, but its worst-case performance for the various operations is linear time, because it is possible that a series of operations results in a tree with linear height. In this section, we describe a simple balancing strategy that guarantees worst-case logarithmic running time for all the fundamental map operations.

## Definition of an AVL Tree

The simple correction is to add a rule to the binary search tree definition that will maintain a logarithmic height for the tree. Although we originally defined the height of a subtree rooted at position $p$ of a tree to be the number of *edges* on the longest path from $p$ to a leaf (see Section 8.1.3), it is easier for explanation in this section to consider the height to be the number of *nodes* on such a longest path. By this definition, a leaf position has height 1, while we trivially define the height of a "null" child to be 0.

In this section, we consider the following ***height-balance property***, which characterizes the structure of a binary search tree $T$ in terms of the heights of its nodes.

***Height-Balance Property*:** For every position $p$ of $T$, the heights of the children of $p$ differ by at most 1.

Any binary search tree $T$ that satisfies the height-balance property is said to be an ***AVL tree***, named after the initials of its inventors: Adel'son-Vel'skii and Landis. An example of an AVL tree is shown in Figure 11.11.

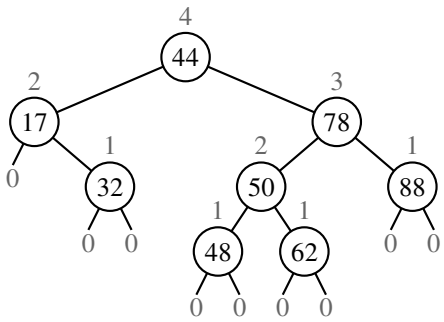



**Figure 11.11:** An example of an AVL tree. The keys of the items are shown inside the nodes, and the heights of the nodes are shown above the nodes (with empty subtrees having height 0).

An immediate consequence of the height-balance property is that a subtree of an AVL tree is itself an AVL tree. The height-balance property has also the important consequence of keeping the height small, as shown in the following proposition.

**Proposition 11.2:** *The height of an AVL tree storing $n$ entries is $O(\log n)$.*

**Justification:** Instead of trying to find an upper bound on the height of an AVL tree directly, it turns out to be easier to work on the "inverse problem" of finding a lower bound on the minimum number of nodes $n(h)$ of an AVL tree with height $h$. We will show that $n(h)$ grows at least exponentially. From this, it will be an easy step to derive that the height of an AVL tree storing $n$ entries is $O(\log n)$.

We begin by noting that $n(1) = 1$ and $n(2) = 2$, because an AVL tree of height 1 must have exactly one node and an AVL tree of height 2 must have at least two nodes. Now, an AVL tree with the minimum number of nodes having height $h$ for $h \geq 3$, is such that both its subtrees are AVL trees with the minimum number of nodes: one with height $h-1$ and the other with height $h-2$. Taking the root into account, we obtain the following formula that relates $n(h)$ to $n(h-1)$ and $n(h-2)$, for $h \geq 3$:

$$n(h) = 1 + n(h-1) + n(h-2). \tag{11.1}$$

At this point, the reader familiar with the properties of Fibonacci progressions (Section 1.8 and Exercise C-3.49) will already see that $n(h)$ is a function exponential in $h$. To formalize that observation, we proceed as follows.

Formula 11.1 implies that $n(h)$ is a strictly increasing function of $h$. Thus, we know that $n(h-1) > n(h-2)$. Replacing $n(h-1)$ with $n(h-2)$ in Formula 11.1 and dropping the 1, we get, for $h \geq 3$,

$$n(h) \; > \; 2 \cdot n(h-2). \tag{11.2}$$

Formula 11.2 indicates that $n(h)$ at least doubles each time $h$ increases by 2, which intuitively means that $n(h)$ grows exponentially. To show this fact in a formal way, we apply Formula 11.2 repeatedly, yielding the following series of inequalities:

$$\begin{aligned} n(h) &> 2 \cdot n(h-2) \\ &> 4 \cdot n(h-4) \\ &> 8 \cdot n(h-6) \\ &\;\;\vdots \\ &> 2^i \cdot n(h-2i). \end{aligned} \tag{11.3}$$

That is, $n(h) > 2^i \cdot n(h-2i)$, for any integer $i$, such that $h-2i \geq 1$. Since we already know the values of $n(1)$ and $n(2)$, we pick $i$ so that $h-2i$ is equal to either 1 or 2. That is, we pick

$$i = \left\lceil \frac{h}{2} \right\rceil - 1.$$

By substituting the above value of $i$ in Formula 11.3, we obtain, for $h \geq 3$,

$$\begin{aligned} n(h) &> 2^{\lceil \frac{h}{2} \rceil - 1} \cdot n\left(h - 2\left\lceil \frac{h}{2} \right\rceil + 2\right) \\ &\geq 2^{\lceil \frac{h}{2} \rceil - 1} n(1) \\ &\geq 2^{\frac{h}{2}-1}. \end{aligned} \tag{11.4}$$

By taking logarithms of both sides of Formula 11.4, we obtain

$$\log(n(h)) > \frac{h}{2} - 1,$$

from which we get

$$h < 2\log(n(h)) + 2, \tag{11.5}$$

which implies that an AVL tree storing $n$ entries has height at most $2\log n + 2$. ■

By Proposition 11.2 and the analysis of binary search trees given in Section 11.1, the operation `__getitem__`, in a map implemented with an AVL tree, runs in time $O(\log n)$, where $n$ is the number of items in the map. Of course, we still have to show how to maintain the height-balance property after an insertion or deletion.

## 11.3.1 Update Operations

Given a binary search tree $T$, we say that a position is ***balanced*** if the absolute value of the difference between the heights of its children is at most 1, and we say that it is ***unbalanced*** otherwise. Thus, the height-balance property characterizing AVL trees is equivalent to saying that every position is balanced.

The insertion and deletion operations for AVL trees begin similarly to the corresponding operations for (standard) binary search trees, but with post-processing for each operation to restore the balance of any portions of the tree that are adversely affected by the change.

### Insertion

Suppose that tree $T$ satisfies the height-balance property, and hence is an AVL tree, prior to the insertion of a new item. An insertion of a new item in a binary search tree, as described in Section 11.1.3, results in a new node at a leaf position $p$. This action may violate the height-balance property (see, for example, Figure 11.12a), yet the only positions that may become unbalanced are ancestors of $p$, because those are the only positions whose subtrees have changed. Therefore, let us describe how to restructure $T$ to fix any unbalance that may have occurred.

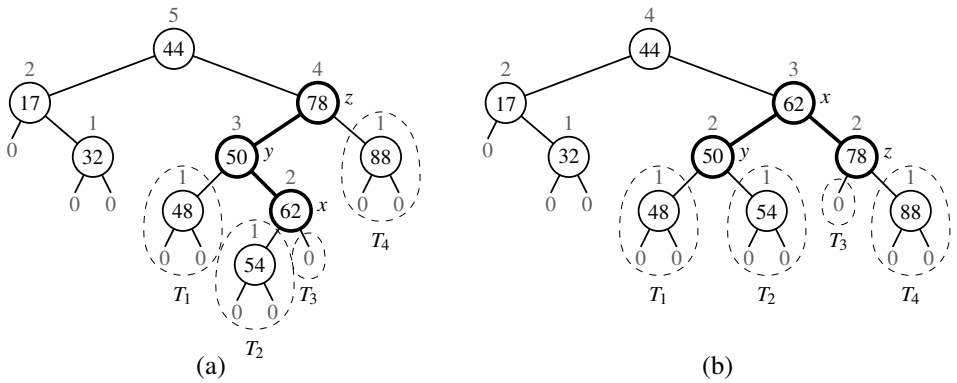



**Figure 11.12:** An example insertion of an item with key 54 in the AVL tree of Figure 11.11: (a) after adding a new node for key 54, the nodes storing keys 78 and 44 become unbalanced; (b) a trinode restructuring restores the height-balance property. We show the heights of nodes above them, and we identify the nodes $x$, $y$, and $z$ and subtrees $T_1$, $T_2$, $T_3$, and $T_4$ participating in the trinode restructuring.

We restore the balance of the nodes in the binary search tree $T$ by a simple "search-and-repair" strategy. In particular, let $z$ be the first position we encounter in going up from $p$ toward the root of $T$ such that $z$ is unbalanced (see Figure 11.12a.) Also, let $y$ denote the child of $z$ with higher height (and note that $y$ must be an ancestor of $p$). Finally, let $x$ be the child of $y$ with higher height (there cannot be a tie and position $x$ must also be an ancestor of $p$, possibly $p$ itself). We rebalance the subtree rooted at $z$ by calling the ***trinode restructuring*** method, `restructure`$(x)$, originally described in Section 11.2. An example of such a restructuring in the context of an AVL insertion is portrayed in Figure 11.12.

To formally argue the correctness of this process in reestablishing the AVL height-balance property, we consider the implication of $z$ being the nearest ancestor of $p$ that became unbalanced after the insertion of $p$. It must be that the height of $y$ increased by one due to the insertion and that it is now 2 greater than its sibling. Since $y$ remains balanced, it must be that it formerly had subtrees with equal heights, and that the subtree containing $x$ has increased its height by one. That subtree increased either because $x = p$, and thus its height changed from 0 to 1, or because $x$ previously had equal-height subtrees and the height of the one containing $p$ has increased by 1. Letting $h \geq 0$ denote the height of the tallest child of $x$, this scenario might be portrayed as in Figure 11.13.

After the trinode restructuring, we see that each of $x$, $y$, and $z$ has become balanced. Furthermore, the node that becomes the root of the subtree after the restructuring has height $h+2$, which is precisely the height that $z$ had before the insertion of the new item. Therefore, any ancestor of $z$ that became temporarily unbalanced becomes balanced again, and this one restructuring restores the height-balance property ***globally***.

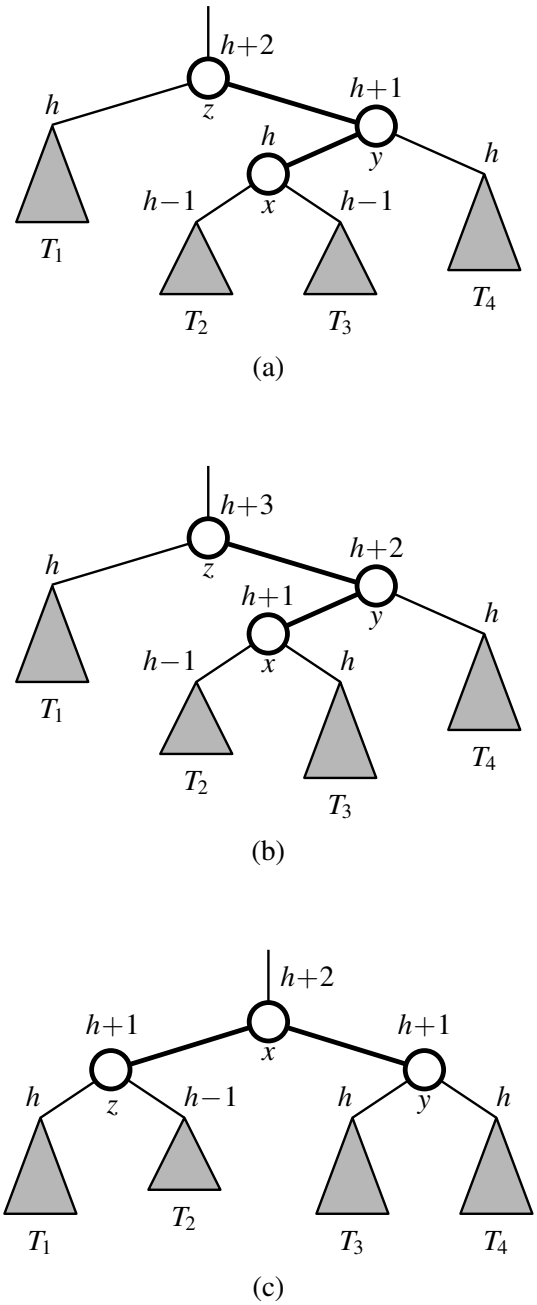



**Figure 11.13:** Rebalancing of a subtree during a typical insertion into an AVL tree: (a) before the insertion; (b) after an insertion in subtree $T_3$ causes imbalance at $z$; (c) after restoring balance with trinode restructuring. Notice that the overall height of the subtree after the insertion is the same as before the insertion.

## Deletion

Recall that a deletion from a regular binary search tree results in the structural removal of a node having either zero or one children. Such a change may violate the height-balance property in an AVL tree. In particular, if position $p$ represents the parent of the removed node in tree $T$, there may be an unbalanced node on the path from $p$ to the root of $T$. (See Figure 11.14a.) In fact, there can be at most one such unbalanced node. (The justification of this fact is left as Exercise C-11.49.)

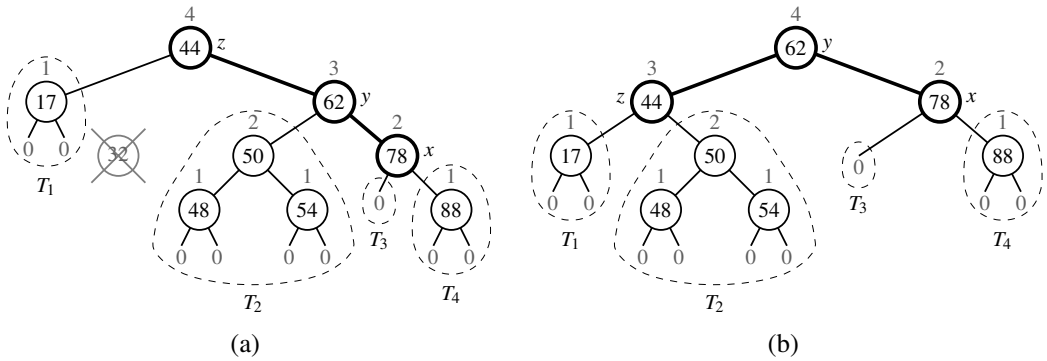



**Figure 11.14:** Deletion of the item with key 32 from the AVL tree of Figure 11.12b: (a) after removing the node storing key 32, the root becomes unbalanced; (b) a (single) rotation restores the height-balance property.

As with insertion, we use trinode restructuring to restore balance in the tree $T$. In particular, let $z$ be the first unbalanced position encountered going up from $p$ toward the root of $T$. Also, let $y$ be the child of $z$ with larger height (note that position $y$ is the child of $z$ that is not an ancestor of $p$), and let $x$ be the child of $y$ defined as follows: If one of the children of $y$ is taller than the other, let $x$ be the taller child of $y$; else (both children of $y$ have the same height), let $x$ be the child of $y$ on the same side as $y$ (that is, if $y$ is the left child of $z$, let $x$ be the left child of $y$, else let $x$ be the right child of $y$). In any case, we then perform a $\mathsf{restructure}(x)$ operation. (See Figure 11.14b.)

The restructured subtree is rooted at the middle position denoted as $b$ in the description of the trinode restructuring operation. The height-balance property is guaranteed to be ***locally*** restored within the subtree of $b$. (See Exercises R-11.11 and R-11.12). Unfortunately, this trinode restructuring may reduce the height of the subtree rooted at $b$ by 1, which may cause an ancestor of $b$ to become unbalanced. So, after rebalancing $z$, we continue walking up $T$ looking for unbalanced positions. If we find another, we perform a restructure operation to restore its balance, and continue marching up $T$ looking for more, all the way to the root. Still, since the height of $T$ is $O(\log n)$, where $n$ is the number of entries, by Proposition 11.2, $O(\log n)$ trinode restructurings are sufficient to restore the height-balance property.

## Performance of AVL Trees

By Proposition 11.2, the height of an AVL tree with $n$ items is guaranteed to be $O(\log n)$. Because the standard binary search tree operation had running times bounded by the height (see Table 11.1), and because the additional work in maintaining balance factors and restructuring an AVL tree can be bounded by the length of a path in the tree, the traditional map operations run in worst-case logarithmic time with an AVL tree. We summarize these results in Table 11.2, and illustrate this performance in Figure 11.15.

| Operation | Running Time |
|---:|---|
| k in T | $O(\log n)$ |
| T[k] = v | $O(\log n)$ |
| T.delete(p), del T[k] | $O(\log n)$ |
| T.find_position(k) | $O(\log n)$ |
| T.first( ), T.last( ), T.find_min( ), T.find_max( ) | $O(\log n)$ |
| T.before(p), T.after(p) | $O(\log n)$ |
| T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k) | $O(\log n)$ |
| T.find_range(start, stop) | $O(s+\log n)$ |
| iter(T), reversed(T) | $O(n)$ |

**Table 11.2:** Worst-case running times of operations for an $n$-item sorted map realized as an AVL tree T, with $s$ denoting the number of items reported by find_range.

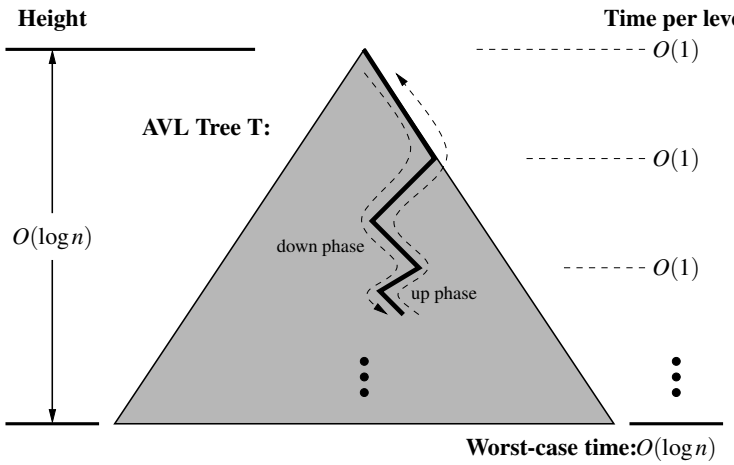



**Figure 11.15:** Illustrating the running time of searches and updates in an AVL tree. The time performance is $O(1)$ per level, broken into a down phase, which typically involves searching, and an up phase, which typically involves updating height values and performing local trinode restructurings (rotations).

### 11.3.2 Python Implementation

A complete implementation of an AVLTreeMap class is provided in Code Fragments 11.12 and 11.13. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1. We highlight two important aspects of our implementation. First, the AVLTreeMap overrides the definition of the nested _Node class, as shown in Code Fragment 11.12, in order to provide support for storing the height of the subtree stored at a node. We also provide several utilities involving heights of nodes, and the corresponding positions.

To implement the core logic of the AVL balancing strategy, we define a utility, named _rebalance, that suffices as a hook for restoring the height-balance property after an insertion or a deletion. Although the inherited behaviors for insertion and deletion are quite different, the necessary post-processing for an AVL tree can be unified. In both cases, we trace an upward path from the position $p$ at which the change took place, recalculating the height of each position based on the (updated) heights of its children, and using a trinode restructuring operation if an imbalanced position is reached. If we reach an ancestor with height that is unchanged by the overall map operation, or if we perform a trinode restructuring that results in the subtree having the same height it had before the map operation, we stop the process; no further ancestor's height will change. To detect the stopping condition, we record the "old" height of each node and compare it to the newly calculated height.

```
1  class AVLTreeMap(TreeMap):
2    """Sorted map implementation using an AVL tree."""
3
4    #------------------------- nested _Node class -------------------------
5    class _Node(TreeMap._Node):
6      """Node class for AVL maintains height value for balancing."""
7      __slots__ = '_height'          # additional data member to store height
8
9      def __init__(self, element, parent=None, left=None, right=None):
10       super().__init__(element, parent, left, right)
11       self._height = 0               # will be recomputed during balancing
12
13     def left_height(self):
14       return self._left._height if self._left is not None else 0
15
16     def right_height(self):
17       return self._right._height if self._right is not None else 0
```

**Code Fragment 11.12:** AVLTreeMap class (continued in Code Fragment 11.13).

```
18    #------------------------ positional-based utility methods ------------------------
19    def _recompute_height(self, p):
20      p._node._height = 1 + max(p._node.left_height( ), p._node.right_height( ))
21
22    def _isbalanced(self, p):
23      return abs(p._node.left_height( ) − p._node.right_height( )) <= 1
24
25    def _tall_child(self, p, favorleft=False): # parameter controls tiebreaker
26      if p._node.left_height( ) + (1 if favorleft else 0) > p._node.right_height( ):
27        return self.left(p)
28      else:
29        return self.right(p)
30
31    def _tall_grandchild(self, p):
32      child = self._tall_child(p)
33      # if child is on left, favor left grandchild; else favor right grandchild
34      alignment = (child == self.left(p))
35      return self._tall_child(child, alignment)
36
37    def _rebalance(self, p):
38      while p is not None:
39        old_height = p._node._height          # trivially 0 if new node
40        if not self._isbalanced(p):           # imbalance detected!
41          # perform trinode restructuring, setting p to resulting root,
42          # and recompute new local heights after the restructuring
43          p = self._restructure(self._tall_grandchild(p))
44          self._recompute_height(self.left(p))
45          self._recompute_height(self.right(p))
46        self._recompute_height(p)             # adjust for recent changes
47        if p._node._height == old_height:     # has height changed?
48          p = None                            # no further changes needed
49        else:
50          p = self.parent(p)                  # repeat with parent
51
52    #------------------------- override balancing hooks -------------------------
53    def _rebalance_insert(self, p):
54      self._rebalance(p)
55
56    def _rebalance_delete(self, p):
57      self._rebalance(p)
```

**Code Fragment 11.13:** AVLTreeMap class (continued from Code Fragment 11.12).

# 11.4 Splay Trees

The next search tree structure we study is known as a a ***splay tree***. This structure is conceptually quite different from the other balanced search trees we discuss in this chapter, for a splay tree does not strictly enforce a logarithmic upper bound on the height of the tree. In fact, there are no additional height, balance, or other auxiliary data associated with the nodes of this tree.

The efficiency of splay trees is due to a certain move-to-root operation, called ***splaying***, that is performed at the bottommost position $p$ reached during every insertion, deletion, or even a search. (In essence, this is a tree variant of the move-to-front heuristic that we explored for lists in Section 7.6.2.) Intuitively, a splay operation causes more frequently accessed elements to remain nearer to the root, thereby reducing the typical search times. The surprising thing about splaying is that it allows us to guarantee a logarithmic amortized running time, for insertions, deletions, and searches.

## 11.4.1 Splaying

Given a node $x$ of a binary search tree $T$, we ***splay*** $x$ by moving $x$ to the root of $T$ through a sequence of restructurings. The particular restructurings we perform are important, for it is not sufficient to move $x$ to the root of $T$ by just any sequence of restructurings. The specific operation we perform to move $x$ up depends upon the relative positions of $x$, its parent $y$, and (if it exists) $x$'s grandparent $z$. There are three cases that we consider.

***zig-zig*:** The node $x$ and its parent $y$ are both left children or both right children. (See Figure 11.16.) We promote $x$, making $y$ a child of $x$ and $z$ a child of $y$, while maintaining the inorder relationships of the nodes in $T$.

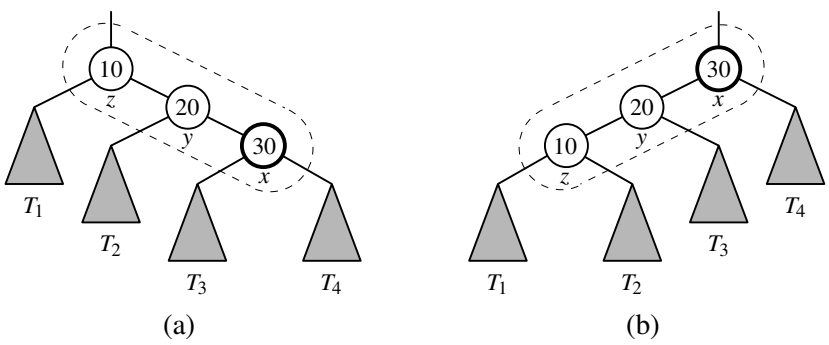



**Figure 11.16:** Zig-zig: (a) before; (b) after. There is another symmetric configuration where $x$ and $y$ are left children.

***zig-zag*:** One of $x$ and $y$ is a left child and the other is a right child. (See Figure 11.17.) In this case, we promote $x$ by making $x$ have $y$ and $z$ as its children, while maintaining the inorder relationships of the nodes in $T$.

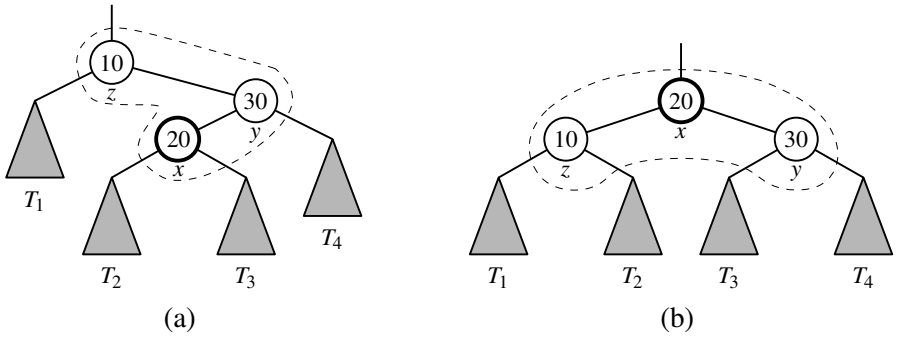



**Figure 11.17:** Zig-zag: (a) before; (b) after. There is another symmetric configuration where $x$ is a right child and $y$ is a left child.

***zig*:** $x$ does not have a grandparent. (See Figure 11.18.) In this case, we perform a single rotation to promote $x$ over $y$, making $y$ a child of $x$, while maintaining the relative inorder relationships of the nodes in $T$.

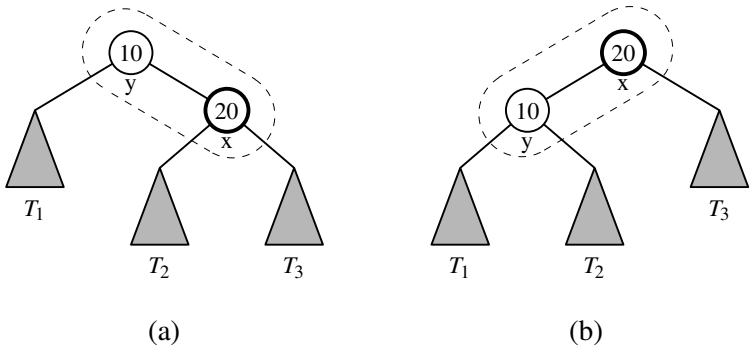



**Figure 11.18:** Zig: (a) before; (b) after. There is another symmetric configuration where $x$ is originally a left child of $y$.

We perform a zig-zig or a zig-zag when $x$ has a grandparent, and we perform a zig when $x$ has a parent but not a grandparent. A ***splaying*** step consists of repeating these restructurings at $x$ until $x$ becomes the root of $T$. An example of the splaying of a node is shown in Figures 11.19 and 11.20.

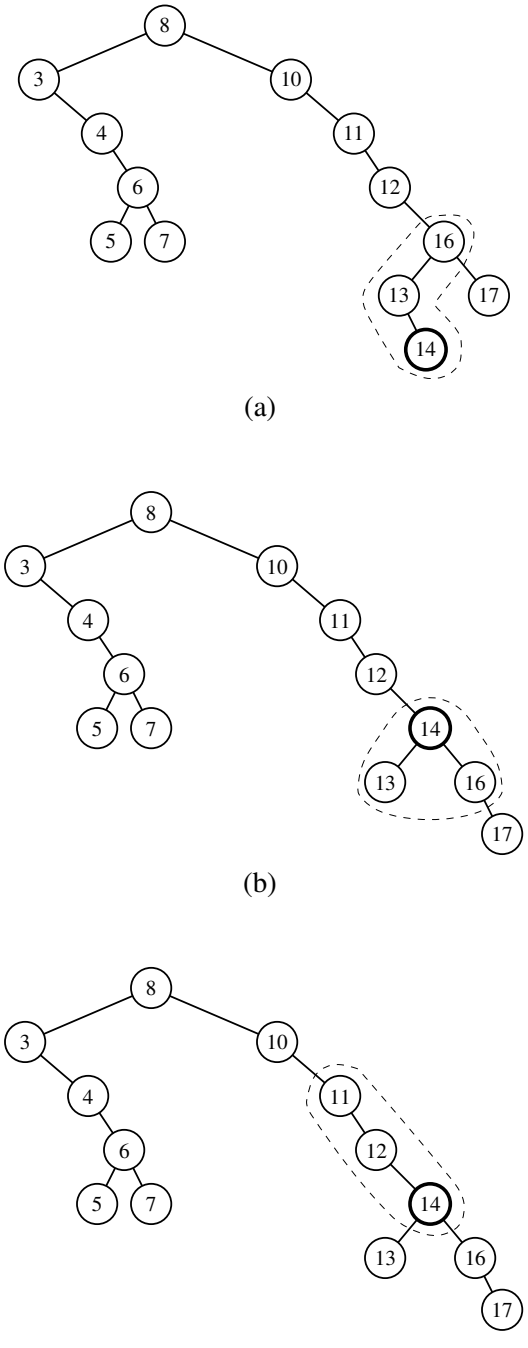
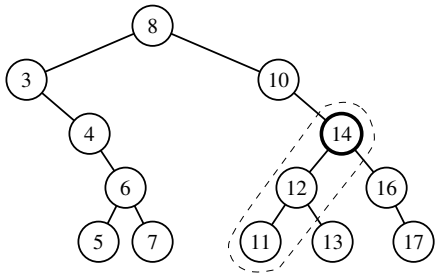
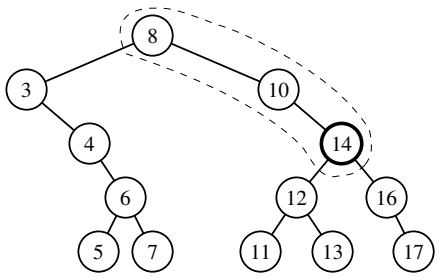


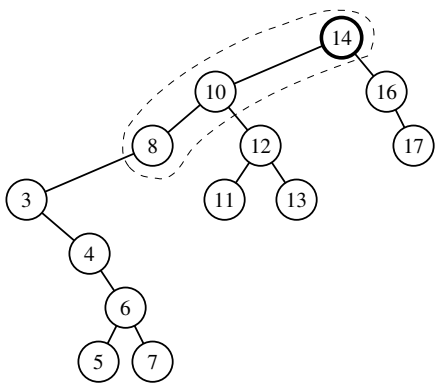

**Figure 11.19:** Example of splaying a node: (a) splaying the node storing 14 starts with a zig-zag; (b) after the zig-zag; (c) the next step will be a zig-zig. (Continues in Figure 11.20.)




(d)




(e)




(f)

**Figure 11.20:** Example of splaying a node:(d) after the zig-zig; (e) the next step is again a zig-zig; (f) after the zig-zig. (Continued from Figure 11.19.)

### 11.4.2 When to Splay

The rules that dictate when splaying is performed are as follows:

- When searching for key $k$, if $k$ is found at position $p$, we splay $p$, else we splay the leaf position at which the search terminates unsuccessfully. For example, the splaying in Figures 11.19 and 11.20 would be performed after searching successfully for key 14 or unsuccessfully for key 15.
- When inserting key $k$, we splay the newly created internal node where $k$ gets inserted. For example, the splaying in Figures 11.19 and 11.20 would be performed if 14 were the newly inserted key. We show a sequence of insertions in a splay tree in Figure 11.21.

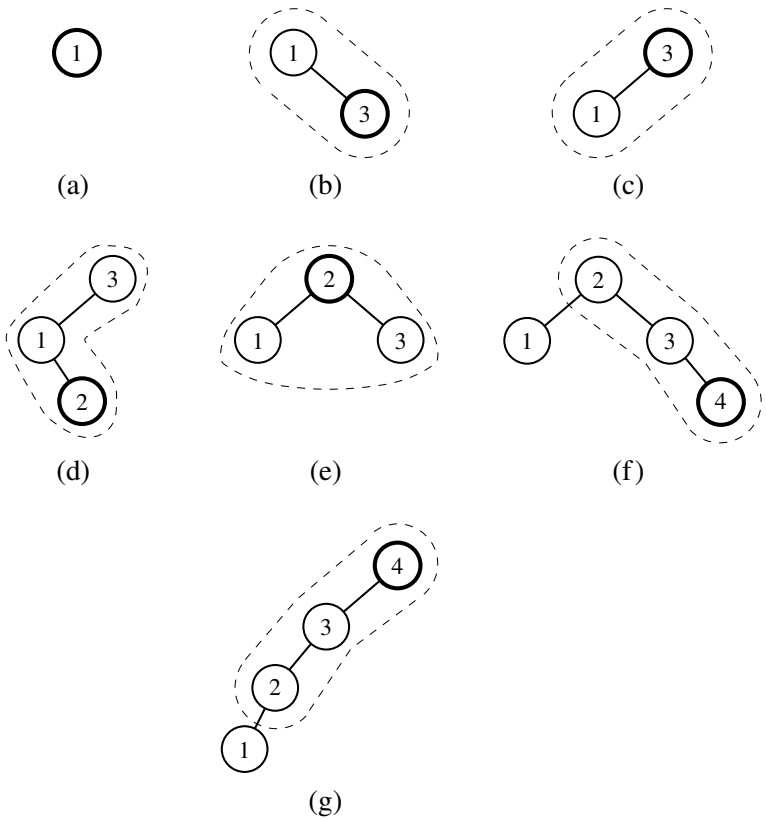



**Figure 11.21:** A sequence of insertions in a splay tree: (a) initial tree; (b) after inserting 3, but before a zig step; (c) after splaying; (d) after inserting 2, but before a zig-zag step; (e) after splaying; (f) after inserting 4, but before a zig-zig step; (g) after splaying.

- When deleting a key $k$, we splay the position $p$ that is the parent of the removed node; recall that by the removal algorithm for binary search trees, the removed node may be that originally containing $k$, or a descendant node with a replacement key. An example of splaying following a deletion is shown in Figure 11.22.

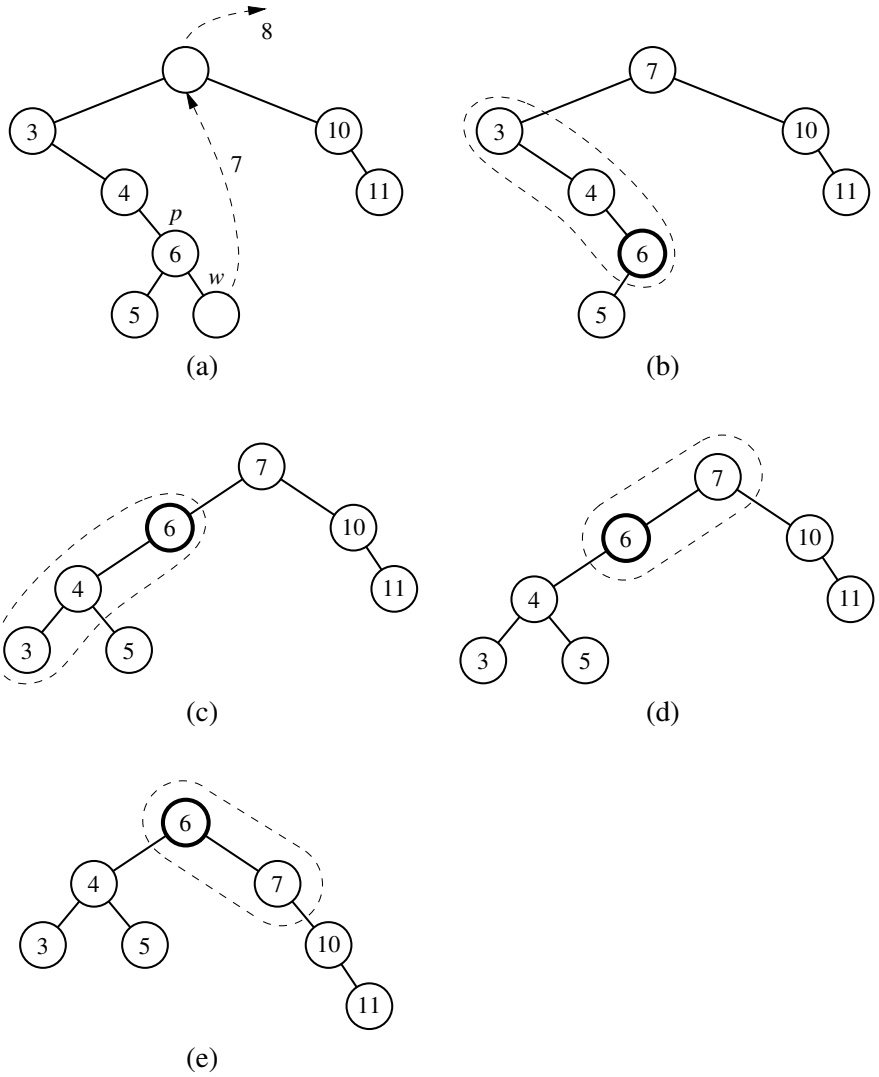



**Figure 11.22:** Deletion from a splay tree: (a) the deletion of 8 from the root node is performed by moving to the root the key of its inorder predecessor $w$, deleting $w$, and splaying the parent $p$ of $w$; (b) splaying $p$ starts with a zig-zig; (c) after the zig-zig; (d) the next step is a zig; (e) after the zig.

### 11.4.3 Python Implementation

Although the mathematical analysis of a splay tree's performance is complex (see Section 11.4.4), the *implementation* of splay trees is a rather simple adaptation to a standard binary search tree. Code Fragment 11.14 provides a complete implementation of a SplayTreeMap class, based upon the underlying TreeMap class and use of the balancing framework described in Section 11.2.1. It is important to note that our original TreeMap class makes calls to the _rebalance_access method, not just from within the __getitem__ method, but also during __setitem__ when modifying the value associated with an existing key, and after any map operations that result in a failed search.

```
1  class SplayTreeMap(TreeMap):
2    """Sorted map implementation using a splay tree."""
3    #------------------------------ splay operation ------------------------------
4    def _splay(self, p):
5      while p != self.root( ):
6        parent = self.parent(p)
7        grand = self.parent(parent)
8        if grand is None:
9          # zig case
10         self._rotate(p)
11       elif (parent == self.left(grand)) == (p == self.left(parent)):
12         # zig-zig case
13         self._rotate(parent)                    # move PARENT up
14         self._rotate(p)                         # then move p up
15       else:
16         # zig-zag case
17         self._rotate(p)                         # move p up
18         self._rotate(p)                         # move p up again
19
20   #---------------------------- override balancing hooks ----------------------------
21   def _rebalance_insert(self, p):
22     self._splay(p)
23
24   def _rebalance_delete(self, p):
25     if p is not None:
26       self._splay(p)
27
28   def _rebalance_access(self, p):
29     self._splay(p)
```

**Code Fragment 11.14:** A complete implementation of the SplayTreeMap class.

### 11.4.4 Amortized Analysis of Splaying ⋆

After a zig-zig or zig-zag, the depth of position $p$ decreases by two, and after a zig the depth of $p$ decreases by one. Thus, if $p$ has depth $d$, splaying $p$ consists of a sequence of $\lfloor d/2 \rfloor$ zig-zigs and/or zig-zags, plus one final zig if $d$ is odd. Since a single zig-zig, zig-zag, or zig affects a constant number of nodes, it can be done in $O(1)$ time. Thus, splaying a position $p$ in a binary search tree $T$ takes time $O(d)$, where $d$ is the depth of $p$ in $T$. In other words, the time for performing a splaying step for a position $p$ is asymptotically the same as the time needed just to reach that position in a top-down search from the root of $T$.

#### Worst-Case Time

In the worst case, the overall running time of a search, insertion, or deletion in a splay tree of height $h$ is $O(h)$, since the position we splay might be the deepest position in the tree. Moreover, it is possible for $h$ to be as large as $n$, as shown in Figure 11.21. Thus, from a worst-case point of view, a splay tree is not an attractive data structure.

In spite of its poor worst-case performance, a splay tree performs well in an amortized sense. That is, in a sequence of intermixed searches, insertions, and deletions, each operation takes on average logarithmic time. We perform the amortized analysis of splay trees using the accounting method.

#### Amortized Performance of Splay Trees

For our analysis, we note that the time for performing a search, insertion, or deletion is proportional to the time for the associated splaying. So let us consider only splaying time.

Let $T$ be a splay tree with $n$ keys, and let $w$ be a node of $T$. We define the ***size*** $n(w)$ of $w$ as the number of nodes in the subtree rooted at $w$. Note that this definition implies that the size of a nonleaf node is one more than the sum of the sizes of its children. We define the ***rank*** $r(w)$ of a node $w$ as the logarithm in base 2 of the size of $w$, that is, $r(w) = \log(n(w))$. Clearly, the root of $T$ has the maximum size, $n$, and the maximum rank, $\log n$, while each leaf has size 1 and rank 0.

We use cyber-dollars to pay for the work we perform in splaying a position $p$ in $T$, and we assume that one cyber-dollar pays for a zig, while two cyber-dollars pay for a zig-zig or a zig-zag. Hence, the cost of splaying a position at depth $d$ is $d$ cyber-dollars. We keep a virtual account storing cyber-dollars at each position of $T$. Note that this account exists only for the purpose of our amortized analysis, and does not need to be included in a data structure implementing the splay tree $T$.

## An Accounting Analysis of Splaying

When we perform a splaying, we pay a certain number of cyber-dollars (the exact value of the payment will be determined at the end of our analysis). We distinguish three cases:

- If the payment is equal to the splaying work, then we use it all to pay for the splaying.
- If the payment is greater than the splaying work, we deposit the excess in the accounts of several nodes.
- If the payment is less than the splaying work, we make withdrawals from the accounts of several nodes to cover the deficiency.

We show below that a payment of $O(\log n)$ cyber-dollars per operation is sufficient to keep the system working, that is, to ensure that each node keeps a nonnegative account balance.

## An Accounting Invariant for Splaying

We use a scheme in which transfers are made between the accounts of the nodes to ensure that there will always be enough cyber-dollars to withdraw for paying for splaying work when needed.

In order to use the accounting method to perform our analysis of splaying, we maintain the following invariant:

> ***Before and after a splaying, each node*** $w$ ***of*** $T$ ***has*** $r(w)$ ***cyber-dollars in its account.***

Note that the invariant is "financially sound," since it does not require us to make a preliminary deposit to endow a tree with zero keys.

Let $r(T)$ be the sum of the ranks of all the nodes of $T$. To preserve the invariant after a splaying, we must make a payment equal to the splaying work plus the total change in $r(T)$. We refer to a single zig, zig-zig, or zig-zag operation in a splaying as a splaying ***substep***. Also, we denote the rank of a node $w$ of $T$ before and after a splaying substep with $r(w)$ and $r'(w)$, respectively. The following proposition gives an upper bound on the change of $r(T)$ caused by a single splaying substep. We will repeatedly use this lemma in our analysis of a full splaying of a node to the root.

**Proposition 11.3:** *Let $\delta$ be the variation of $r(T)$ caused by a single splaying substep (a zig, zig-zig, or zig-zag) for a node $x$ in $T$. We have the following:*

- $\delta \leq 3(r'(x) - r(x)) - 2$ *if the substep is a zig-zig or zig-zag.*
- $\delta \leq 3(r'(x) - r(x))$ *if the substep is a zig.*

**Justification:** We use the fact (see Proposition B.1, Appendix A) that, if $a > 0$, $b > 0$, and $c > a + b$,

$$\log a + \log b < 2\log c - 2. \tag{11.6}$$

Let us consider the change in $r(T)$ caused by each type of splaying substep.

***zig-zig*:** (Recall Figure 11.16.) Since the size of each node is one more than the size of its two children, note that only the ranks of $x$, $y$, and $z$ change in a zig-zig operation, where $y$ is the parent of $x$ and $z$ is the parent of $y$. Also, $r'(x) = r(z)$, $r'(y) \leq r'(x)$, and $r(x) \leq r(y)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(x) + r'(z) - 2r(x). \end{aligned} \tag{11.7}$$

Note that $n(x) + n'(z) < n'(x)$. Thus, $r(x) + r'(z) < 2r'(x) - 2$, as per Formula 11.6; that is,

$$r'(z) < 2r'(x) - r(x) - 2.$$

This inequality and Formula 11.7 imply

$$\begin{aligned} \delta &\leq r'(x) + (2r'(x) - r(x) - 2) - 2r(x) \\ &\leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig-zag*:** (Recall Figure 11.17.) Again, by the definition of size and rank, only the ranks of $x$, $y$, and $z$ change, where $y$ denotes the parent of $x$ and $z$ denotes the parent of $y$. Also, $r(x) < r(y) < r(z) = r'(x)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(y) + r'(z) - 2r(x). \end{aligned} \tag{11.8}$$

Note that $n'(y) + n'(z) < n'(x)$; hence, $r'(y) + r'(z) < 2r'(x) - 2$, as per Formula 11.6. Thus,

$$\begin{aligned} \delta &\leq 2r'(x) - 2 - 2r(x) \\ &= 2(r'(x) - r(x)) - 2 \leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig*:** (Recall Figure 11.18.) In this case, only the ranks of $x$ and $y$ change, where $y$ denotes the parent of $x$. Also, $r'(y) \leq r(y)$ and $r'(x) \geq r(x)$. Thus,

$$\begin{aligned} \delta &= r'(y) + r'(x) - r(y) - r(x) \\ &\leq r'(x) - r(x) \\ &\leq 3(r'(x) - r(x)). \end{aligned}$$

■

**Proposition 11.4:** *Let $T$ be a splay tree with root $t$, and let $\Delta$ be the total variation of $r(T)$ caused by splaying a node $x$ at depth $d$. We have*

$$\Delta \leq 3(r(t) - r(x)) - d + 2.$$

**Justification:** Splaying node $x$ consists of $c = \lceil d/2 \rceil$ splaying substeps, each of which is a zig-zig or a zig-zag, except possibly the last one, which is a zig if $d$ is odd. Let $r_0(x) = r(x)$ be the initial rank of $x$, and for $i = 1, \ldots, c$, let $r_i(x)$ be the rank of $x$ after the $i^{th}$ substep and $\delta_i$ be the variation of $r(T)$ caused by the $i^{th}$ substep. By Proposition 11.3, the total variation $\Delta$ of $r(T)$ caused by splaying $x$ is

$$\begin{aligned}
\Delta &= \sum_{i=1}^{c} \delta_i \\
&\leq 2 + \sum_{i=1}^{c} 3(r_i(x) - r_{i-1}(x)) - 2 \\
&= 3(r_c(x) - r_0(x)) - 2c + 2 \\
&\leq 3(r(t) - r(x)) - d + 2.
\end{aligned}$$

■

By Proposition 11.4, if we make a payment of $3(r(t) - r(x)) + 2$ cyber-dollars towards the splaying of node $x$, we have enough cyber-dollars to maintain the invariant, keeping $r(w)$ cyber-dollars at each node $w$ in $T$, and pay for the entire splaying work, which costs $d$ cyber-dollars. Since the size of the root $t$ is $n$, its rank $r(t) = \log n$. Given that $r(x) \geq 0$, the payment to be made for splaying is $O(\log n)$ cyber-dollars. To complete our analysis, we have to compute the cost for maintaining the invariant when a node is inserted or deleted.

When inserting a new node $w$ into a splay tree with $n$ keys, the ranks of all the ancestors of $w$ are increased. Namely, let $w_0, w_i, \ldots, w_d$ be the ancestors of $w$, where $w_0 = w$, $w_i$ is the parent of $w_{i-1}$, and $w_d$ is the root. For $i = 1, \ldots, d$, let $n'(w_i)$ and $n(w_i)$ be the size of $w_i$ before and after the insertion, respectively, and let $r'(w_i)$ and $r(w_i)$ be the rank of $w_i$ before and after the insertion. We have

$$n'(w_i) = n(w_i) + 1.$$

Also, since $n(w_i) + 1 \leq n(w_{i+1})$, for $i = 0, 1, \ldots, d-1$, we have the following for each $i$ in this range:

$$r'(w_i) = \log(n'(w_i)) = \log(n(w_i) + 1) \leq \log(n(w_{i+1})) = r(w_{i+1}).$$

Thus, the total variation of $r(T)$ caused by the insertion is

$$\begin{aligned}
\sum_{i=1}^{d} \big(r'(w_i) - r(w_i)\big) &\leq r'(w_d) + \sum_{i=1}^{d-1} (r(w_{i+1}) - r(w_i)) \\
&= r'(w_d) - r(w_0) \\
&\leq \log n.
\end{aligned}$$

Therefore, a payment of $O(\log n)$ cyber-dollars is sufficient to maintain the invariant when a new node is inserted.

When deleting a node $w$ from a splay tree with $n$ keys, the ranks of all the ancestors of $w$ are decreased. Thus, the total variation of $r(T)$ caused by the deletion is negative, and we do not need to make any payment to maintain the invariant when a node is deleted. Therefore, we may summarize our amortized analysis in the following proposition (which is sometimes called the "balance proposition" for splay trees):

**Proposition 11.5:** *Consider a sequence of $m$ operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree with zero keys. Also, let $n_i$ be the number of keys in the tree after operation $i$, and $n$ be the total number of insertions. The total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{m} \log n_i\right),$$

*which is $O(m \log n)$.*

In other words, the amortized running time of performing a search, insertion, or deletion in a splay tree is $O(\log n)$, where $n$ is the size of the splay tree at the time. Thus, a splay tree can achieve logarithmic-time amortized performance for implementing a sorted map ADT. This amortized performance matches the worst-case performance of AVL trees, $(2,4)$ trees, and red-black trees, but it does so using a simple binary tree that does not need any extra balance information stored at each of its nodes. In addition, splay trees have a number of other interesting properties that are not shared by these other balanced search trees. We explore one such additional property in the following proposition (which is sometimes called the "Static Optimality" proposition for splay trees):

**Proposition 11.6:** *Consider a sequence of $m$ operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree $T$ with zero keys. Also, let $f(i)$ denote the number of times the entry $i$ is accessed in the splay tree, that is, its frequency, and let $n$ denote the total number of entries. Assuming that each entry is accessed at least once, then the total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{n} f(i) \log(m/f(i))\right).$$

We omit the proof of this proposition, but it is not as hard to justify as one might imagine. The remarkable thing is that this proposition states that the amortized running time of accessing an entry $i$ is $O(\log(m/f(i)))$.

# 11.5 (2,4) Trees

In this section, we consider a data structure known as a ***(2,4) tree***. It is a particular example of a more general structure known as a ***multiway search tree***, in which internal nodes may have more than two children. Other forms of multiway search trees will be discussed in Section 15.3.

## 11.5.1 Multiway Search Trees

Recall that general trees are defined so that internal nodes may have many children. In this section, we discuss how general trees can be used as multiway search trees. Map items stored in a search tree are pairs of the form $(k, v)$, where $k$ is the ***key*** and $v$ is the ***value*** associated with the key.

### Definition of a Multiway Search Tree

Let $w$ be a node of an ordered tree. We say that $w$ is a ***d-node*** if $w$ has $d$ children. We define a multiway search tree to be an ordered tree $T$ that has the following properties, which are illustrated in Figure 11.23a:

- Each internal node of $T$ has at least two children. That is, each internal node is a $d$-node such that $d \geq 2$.
- Each internal $d$-node $w$ of $T$ with children $c_1, \ldots, c_d$ stores an ordered set of $d-1$ key-value pairs $(k_1, v_1), \ldots, (k_{d-1}, v_{d-1})$, where $k_1 \leq \cdots \leq k_{d-1}$.
- Let us conventionally define $k_0 = -\infty$ and $k_d = +\infty$. For each item $(k, v)$ stored at a node in the subtree of $w$ rooted at $c_i$, $i = 1, \ldots, d$, we have that $k_{i-1} \leq k \leq k_i$.

That is, if we think of the set of keys stored at $w$ as including the special fictitious keys $k_0 = -\infty$ and $k_d = +\infty$, then a key $k$ stored in the subtree of $T$ rooted at a child node $c_i$ must be "in between" two keys stored at $w$. This simple viewpoint gives rise to the rule that a $d$-node stores $d-1$ regular keys, and it also forms the basis of the algorithm for searching in a multiway search tree.

By the above definition, the external nodes of a multiway search do not store any data and serve only as "placeholders." These external nodes can be efficiently represented by None references, as has been our convention with binary search trees (Section 11.1). However, for the sake of exposition, we will discuss these as actual nodes that do not store anything. Based on this definition, there is an interesting relationship between the number of key-value pairs and the number of external nodes in a multiway search tree.

**Proposition 11.7:** *An $n$-item multiway search tree has $n+1$ external nodes.*

We leave the justification of this proposition as an exercise (C-11.52).

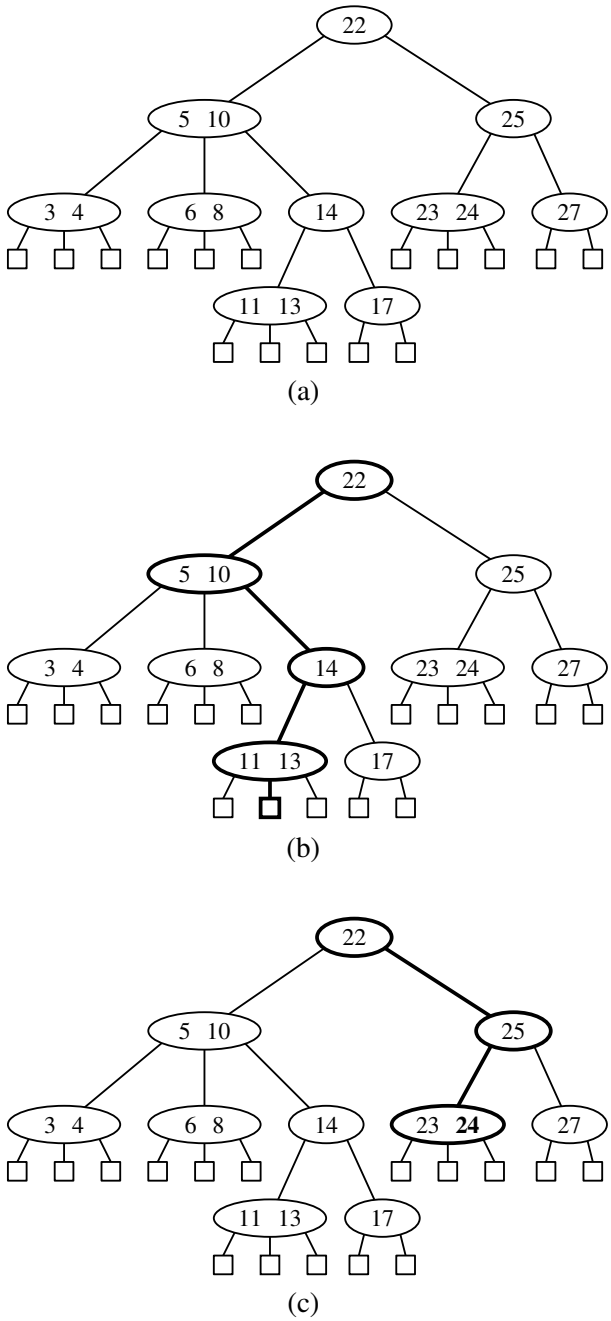



**Figure 11.23:** (a) A multiway search tree $T$; (b) search path in $T$ for key 12 (unsuccessful search); (c) search path in $T$ for key 24 (successful search).

## Searching in a Multiway Tree

Searching for an item with key $k$ in a multiway search tree $T$ is simple. We perform such a search by tracing a path in $T$ starting at the root. (See Figure 11.23b and c.) When we are at a $d$-node $w$ during this search, we compare the key $k$ with the keys $k_1, \ldots, k_{d-1}$ stored at $w$. If $k = k_i$ for some $i$, the search is successfully completed. Otherwise, we continue the search in the child $c_i$ of $w$ such that $k_{i-1} < k < k_i$. (Recall that we conventionally define $k_0 = -\infty$ and $k_d = +\infty$.) If we reach an external node, then we know that there is no item with key $k$ in $T$, and the search terminates unsuccessfully.

## Data Structures for Representing Multiway Search Trees

In Section 8.3.3, we discuss a linked data structure for representing a general tree. This representation can also be used for a multiway search tree. When using a general tree to implement a multiway search tree, we must store at each node one or more key-value pairs associated with that node. That is, we need to store with $w$ a reference to some collection that stores the items for $w$.

During a search for key $k$ in a multiway search tree, the primary operation needed when navigating a node is finding the smallest key at that node that is greater than or equal to $k$. For this reason, it is natural to model the information at a node itself as a sorted map, allowing use of the find_ge(k) method. We say such a map serves as a ***secondary*** data structure to support the ***primary*** data structure represented by the entire multiway search tree. This reasoning may at first seem like a circular argument, since we need a representation of a (secondary) ordered map to represent a (primary) ordered map. We can avoid any circular dependence, however, by using the ***bootstrapping*** technique, where we use a simple solution to a problem to create a new, more advanced solution.

In the context of a multiway search tree, a natural choice for the secondary structure at each node is the SortedTableMap of Section 10.3.1. Because we want to determine the associated value in case of a match for key $k$, and otherwise the corresponding child $c_i$ such that $k_{i-1} < k < k_i$, we recommend having each key $k_i$ in the secondary structure map to the pair $(v_i, c_i)$. With such a realization of a multiway search tree $T$, processing a $d$-node $w$ while searching for an item of $T$ with key $k$ can be performed using a binary search operation in $O(\log d)$ time. Let $d_{\max}$ denote the maximum number of children of any node of $T$, and let $h$ denote the height of $T$. The search time in a multiway search tree is therefore $O(h \log d_{\max})$. If $d_{\max}$ is a constant, the running time for performing a search is $O(h)$.

The primary efficiency goal for a multiway search tree is to keep the height as small as possible. We next discuss a strategy that caps $d_{\max}$ at 4 while guaranteeing a height $h$ that is logarithmic in $n$, the total number of items stored in the map.

## 11.5.2 (2,4)-Tree Operations

A multiway search tree that keeps the secondary data structures stored at each node small and also keeps the primary multiway tree balanced is the $(\mathbf{2},\mathbf{4})$ ***tree***, which is sometimes called a 2-4 tree or 2-3-4 tree. This data structure achieves these goals by maintaining two simple properties (see Figure 11.24):

***Size Property*:** Every internal node has at most four children.

***Depth Property*:** All the external nodes have the same depth.

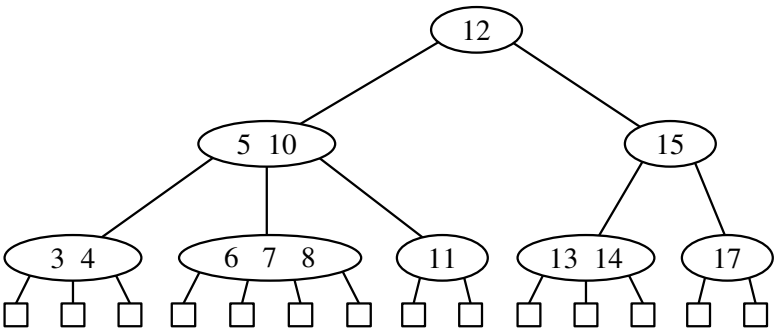



**Figure 11.24:** A $(2,4)$ tree.

Again, we assume that external nodes are empty and, for the sake of simplicity, we describe our search and update methods assuming that external nodes are real nodes, although this latter requirement is not strictly needed.

Enforcing the size property for $(2,4)$ trees keeps the nodes in the multiway search tree simple. It also gives rise to the alternative name "2-3-4 tree," since it implies that each internal node in the tree has 2, 3, or 4 children. Another implication of this rule is that we can represent the secondary map stored at each internal node using an unordered list or an ordered array, and still achieve $O(1)$-time performance for all operations (since $d_{\max} = 4$). The depth property, on the other hand, enforces an important bound on the height of a $(2,4)$ tree.

**Proposition 11.8:** *The height of a $(2,4)$ tree storing $n$ items is $O(\log n)$.*

**Justification:** Let $h$ be the height of a $(2,4)$ tree $T$ storing $n$ items. We justify the proposition by showing the claim

$$\frac{1}{2}\log(n+1) \leq h \leq \log(n+1). \tag{11.9}$$

To justify this claim note first that, by the size property, we can have at most 4 nodes at depth 1, at most $4^2$ nodes at depth 2, and so on. Thus, the number of external nodes in $T$ is at most $4^h$. Likewise, by the depth property and the definition

of a $(2,4)$ tree, we must have at least 2 nodes at depth 1, at least $2^2$ nodes at depth 2, and so on. Thus, the number of external nodes in $T$ is at least $2^h$. In addition, by Proposition 11.7, the number of external nodes in $T$ is $n+1$. Therefore, we obtain

$$2^h \leq n+1 \leq 4^h.$$

Taking the logarithm in base 2 of the terms for the above inequalities, we get that

$$h \leq \log(n+1) \leq 2h,$$

which justifies our claim (Formula 11.9) when terms are rearranged. ■

Proposition 11.8 states that the size and depth properties are sufficient for keeping a multiway tree balanced. Moreover, this proposition implies that performing a search in a $(2,4)$ tree takes $O(\log n)$ time and that the specific realization of the secondary structures at the nodes is not a crucial design choice, since the maximum number of children $d_{\max}$ is a constant.

Maintaining the size and depth properties requires some effort after performing insertions and deletions in a $(2,4)$ tree, however. We discuss these operations next.

## Insertion

To insert a new item $(k,v)$, with key $k$, into a $(2,4)$ tree $T$, we first perform a search for $k$. Assuming that $T$ has no item with key $k$, this search terminates unsuccessfully at an external node $z$. Let $w$ be the parent of $z$. We insert the new item into node $w$ and add a new child $y$ (an external node) to $w$ on the left of $z$.

Our insertion method preserves the depth property, since we add a new external node at the same level as existing external nodes. Nevertheless, it may violate the size property. Indeed, if a node $w$ was previously a 4-node, then it would become a 5-node after the insertion, which causes the tree $T$ to no longer be a $(2,4)$ tree. This type of violation of the size property is called an ***overflow*** at node $w$, and it must be resolved in order to restore the properties of a $(2,4)$ tree. Let $c_1,\ldots,c_5$ be the children of $w$, and let $k_1,\ldots,k_4$ be the keys stored at $w$. To remedy the overflow at node $w$, we perform a ***split*** operation on $w$ as follows (see Figure 11.25):

- Replace $w$ with two nodes $w'$ and $w''$, where
  - $w'$ is a 3-node with children $c_1,c_2,c_3$ storing keys $k_1$ and $k_2$
  - $w''$ is a 2-node with children $c_4,c_5$ storing key $k_4$.
- If $w$ is the root of $T$, create a new root node $u$; else, let $u$ be the parent of $w$.
- Insert key $k_3$ into $u$ and make $w'$ and $w''$ children of $u$, so that if $w$ was child $i$ of $u$, then $w'$ and $w''$ become children $i$ and $i+1$ of $u$, respectively.

As a consequence of a split operation on node $w$, a new overflow may occur at the parent $u$ of $w$. If such an overflow occurs, it triggers in turn a split at node $u$. (See Figure 11.26.) A split operation either eliminates the overflow or propagates it into the parent of the current node. We show a sequence of insertions in a $(2,4)$ tree in Figure 11.27.

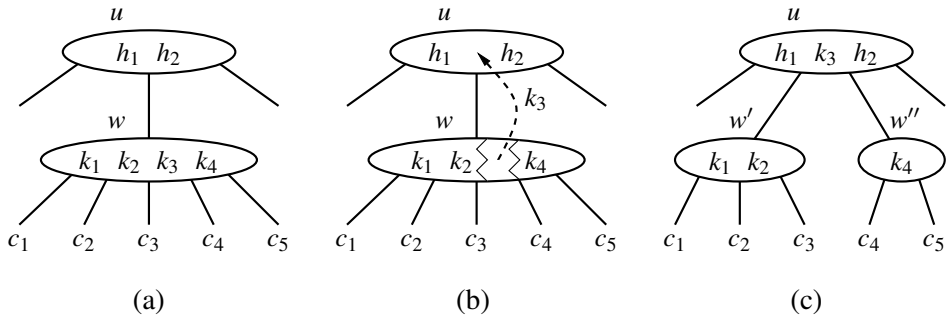



**Figure 11.25:** A node split: (a) overflow at a 5-node $w$; (b) the third key of $w$ inserted into the parent $u$ of $w$; (c) node $w$ replaced with a 3-node $w'$ and a 2-node $w''$.

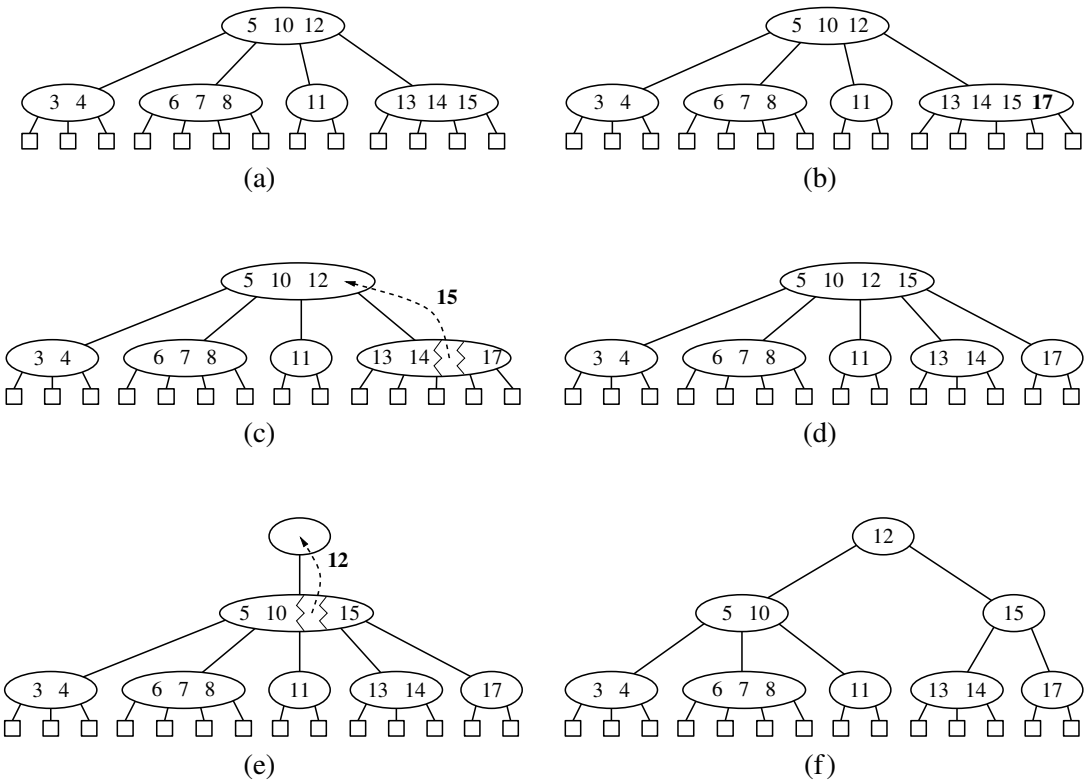



**Figure 11.26:** An insertion in a $(2,4)$ tree that causes a cascading split: (a) before the insertion; (b) insertion of 17, causing an overflow; (c) a split; (d) after the split a new overflow occurs; (e) another split, creating a new root node; (f) final tree.

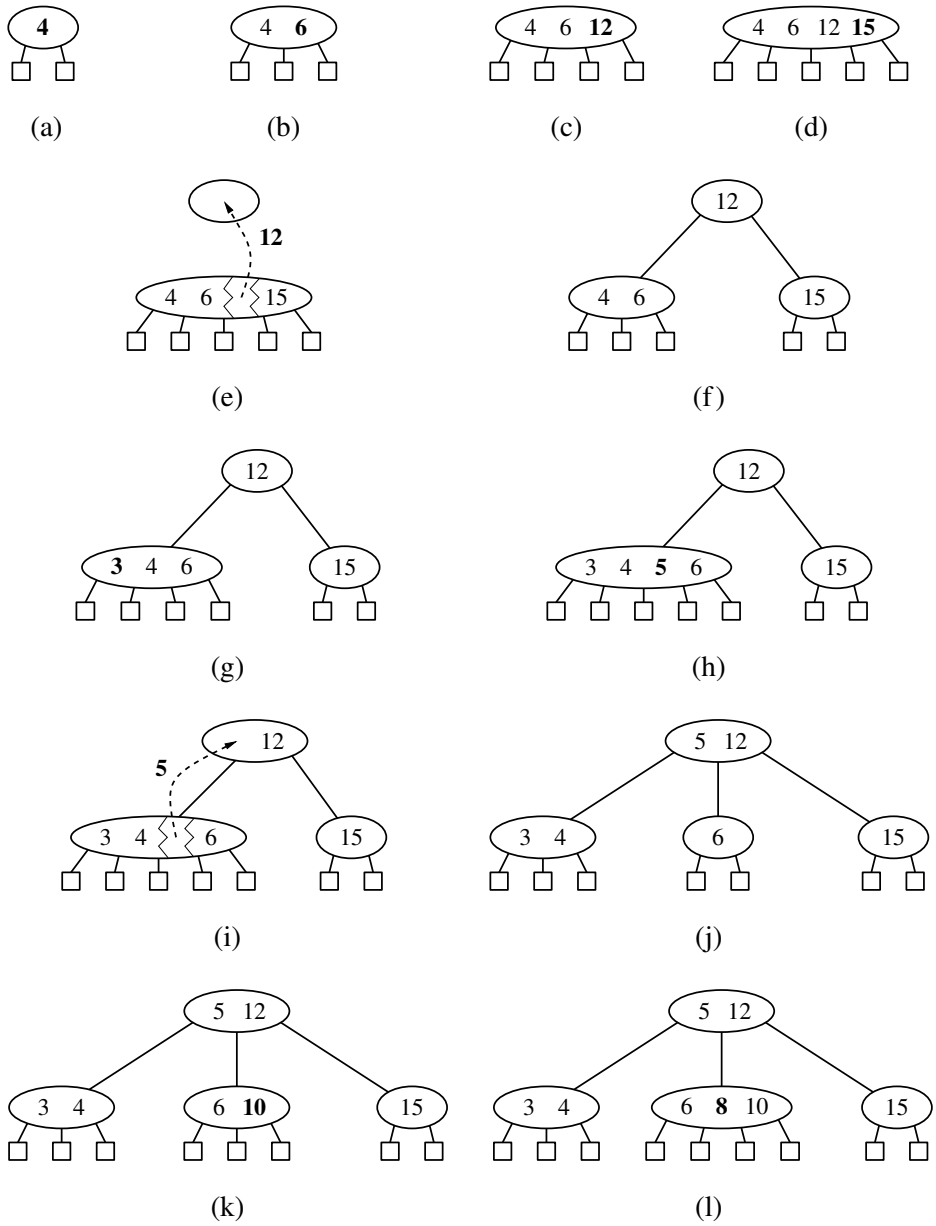



**Figure 11.27:** A sequence of insertions into a $(2,4)$ tree: (a) initial tree with one item; (b) insertion of 6; (c) insertion of 12; (d) insertion of 15, which causes an overflow; (e) split, which causes the creation of a new root node; (f) after the split; (g) insertion of 3; (h) insertion of 5, which causes an overflow; (i) split; (j) after the split; (k) insertion of 10; (l) insertion of 8.

### Analysis of Insertion in a (2,4) Tree

Because $d_{\max}$ is at most 4, the original search for the placement of new key $k$ uses $O(1)$ time at each level, and thus $O(\log n)$ time overall, since the height of the tree is $O(\log n)$ by Proposition 11.8.

The modifications to a single node to insert a new key and child can be implemented to run in $O(1)$ time, as can a single split operation. The number of cascading split operations is bounded by the height of the tree, and so that phase of the insertion process also runs in $O(\log n)$ time. Therefore, the total time to perform an insertion in a $(2,4)$ tree is $O(\log n)$.

## Deletion

Let us now consider the removal of an item with key $k$ from a $(2,4)$ tree $T$. We begin such an operation by performing a search in $T$ for an item with key $k$. Removing an item from a $(2,4)$ tree can always be reduced to the case where the item to be removed is stored at a node $w$ whose children are external nodes. Suppose, for instance, that the item with key $k$ that we wish to remove is stored in the $i^{th}$ item $(k_i, v_i)$ at a node $z$ that has only internal-node children. In this case, we swap the item $(k_i, v_i)$ with an appropriate item that is stored at a node $w$ with external-node children as follows (see Figure 11.28d):

1. We find the rightmost internal node $w$ in the subtree rooted at the $i^{th}$ child of $z$, noting that the children of node $w$ are all external nodes.
2. We swap the item $(k_i, v_i)$ at $z$ with the last item of $w$.

Once we ensure that the item to remove is stored at a node $w$ with only external-node children (because either it was already at $w$ or we swapped it into $w$), we simply remove the item from $w$ and remove the $i^{th}$ external node of $w$.

Removing an item (and a child) from a node $w$ as described above preserves the depth property, for we always remove an external child from a node $w$ with only external children. However, in removing such an external node, we may violate the size property at $w$. Indeed, if $w$ was previously a 2-node, then it becomes a 1-node with no items after the removal (Figure 11.28a and d), which is not allowed in a $(2,4)$ tree. This type of violation of the size property is called an ***underflow*** at node $w$. To remedy an underflow, we check whether an immediate sibling of $w$ is a 3-node or a 4-node. If we find such a sibling $s$, then we perform a ***transfer*** operation, in which we move a child of $s$ to $w$, a key of $s$ to the parent $u$ of $w$ and $s$, and a key of $u$ to $w$. (See Figure 11.28b and c.) If $w$ has only one sibling, or if both immediate siblings of $w$ are 2-nodes, then we perform a ***fusion*** operation, in which we merge $w$ with a sibling, creating a new node $w'$, and move a key from the parent $u$ of $w$ to $w'$. (See Figure 11.28e and f.)

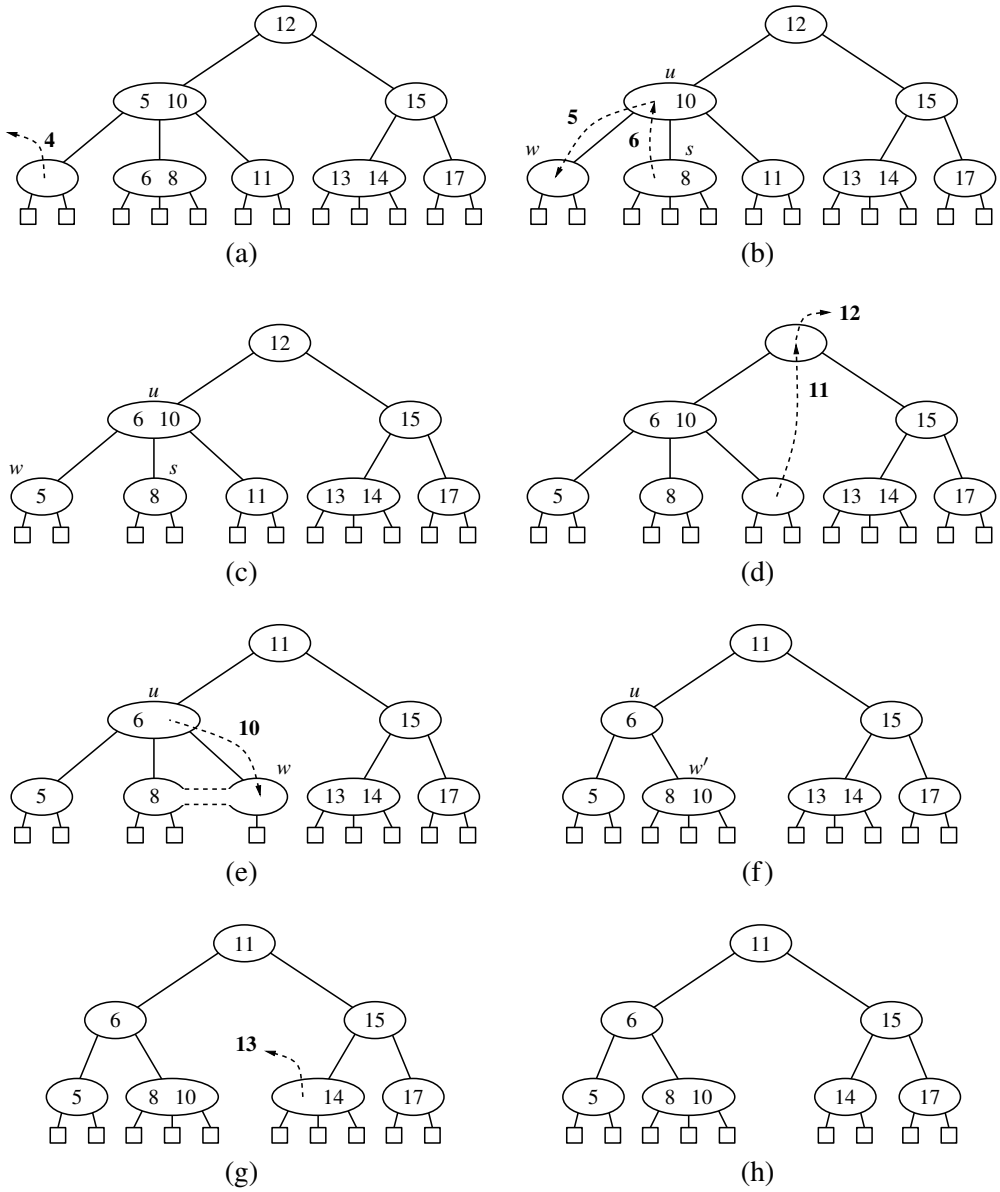



**Figure 11.28:** A sequence of removals from a $(2,4)$ tree: (a) removal of 4, causing an underflow; (b) a transfer operation; (c) after the transfer operation; (d) removal of 12, causing an underflow; (e) a fusion operation; (f) after the fusion operation; (g) removal of 13; (h) after removing 13.

A fusion operation at node $w$ may cause a new underflow to occur at the parent $u$ of $w$, which in turn triggers a transfer or fusion at $u$. (See Figure 11.29.) Hence, the number of fusion operations is bounded by the height of the tree, which is $O(\log n)$ by Proposition 11.8. If an underflow propagates all the way up to the root, then the root is simply deleted. (See Figure 11.29c and d.)

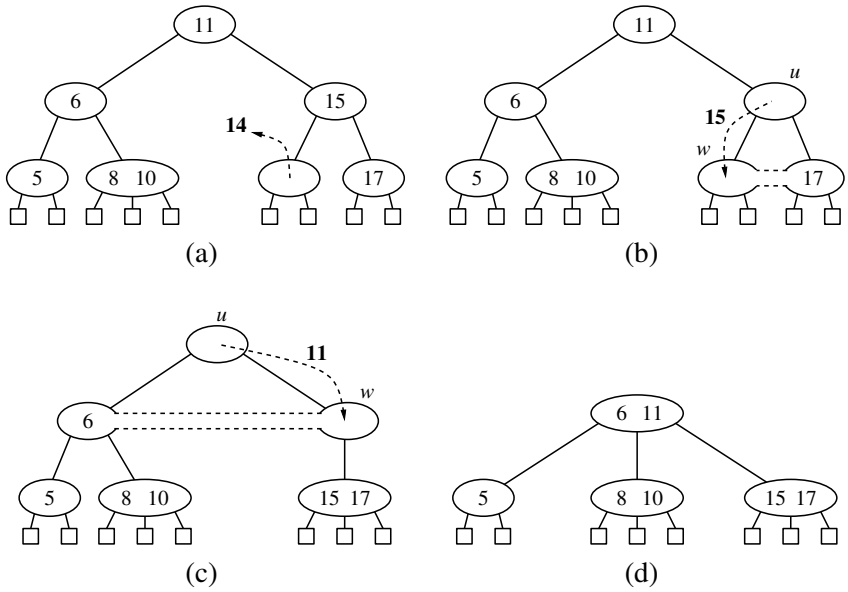



**Figure 11.29:** A propagating sequence of fusions in a $(2,4)$ tree: (a) removal of 14, which causes an underflow; (b) fusion, which causes another underflow; (c) second fusion operation, which causes the root to be removed; (d) final tree.

## Performance of (2,4) Trees

The asymptotic performance of a $(2,4)$ tree is identical to that of an AVL tree (see Table 11.2) in terms of the sorted map ADT, with guaranteed logarithmic bounds for most operations. The time complexity analysis for a $(2,4)$ tree having $n$ key-value pairs is based on the following:

- The height of a $(2,4)$ tree storing $n$ entries is $O(\log n)$, by Proposition 11.8.
- A split, transfer, or fusion operation takes $O(1)$ time.
- A search, insertion, or removal of an entry visits $O(\log n)$ nodes.

Thus, $(2,4)$ trees provide for fast map search and update operations. $(2,4)$ trees also have an interesting relationship to the data structure we discuss next.

## 11.6 Red-Black Trees

Although AVL trees and $(2,4)$ trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion, and $(2,4)$ trees may require many split or fusing operations to be performed after an insertion or removal. The data structure we discuss in this section, the red-black tree, does not have these drawbacks; it uses $O(1)$ structural changes after an update in order to stay balanced.

Formally, a ***red-black tree*** is a binary search tree (see Section 11.1) with nodes colored red and black in a way that satisfies the following properties:

***Root Property*:** The root is black.

***Red Property*:** The children of a red node (if any) are black.

***Depth Property*:** All nodes with zero or one children have the same ***black depth***, defined as the number of black ancestors. (Recall that a node is its own ancestor).

An example of a red-black tree is shown in Figure 11.30.

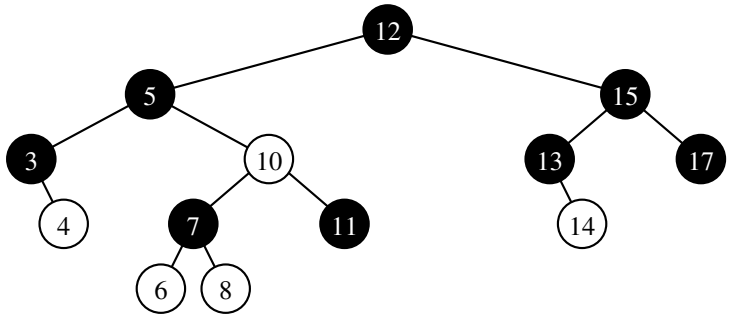



**Figure 11.30:** An example of a red-black tree, with "red" nodes drawn in white. The common black depth for this tree is 3.

We can make the red-black tree definition more intuitive by noting an interesting correspondence between red-black trees and $(2,4)$ trees (excluding their trivial external nodes). Namely, given a red-black tree, we can construct a corresponding $(2,4)$ tree by merging every red node $w$ into its parent, storing the entry from $w$ at its parent, and with the children of $w$ becoming ordered children of the parent. For example, the red-black tree in Figure 11.30 corresponds to the $(2,4)$ tree from Figure 11.24, as illustrated in Figure 11.31. The depth property of the red-black tree corresponds to the depth property of the $(2,4)$ tree since exactly one black node of the red-black tree contributes to each node of the corresponding $(2,4)$ tree.

Conversely, we can transform any $(2,4)$ tree into a corresponding red-black tree by coloring each node $w$ black and then performing the following transformations, as illustrated in Figure 11.32.

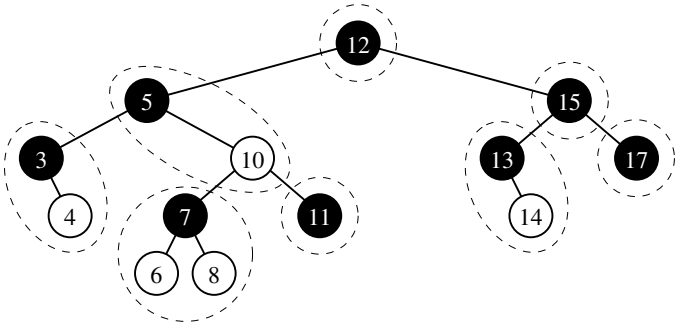



**Figure 11.31:** An illustration that the red-black tree of Figure 11.30 corresponds to the $(2,4)$ tree of Figure 11.24, based on the highlighted grouping of red nodes with their black parents.

- If $w$ is a 2-node, then keep the (black) children of $w$ as is.
- If $w$ is a 3-node, then create a new red node $y$, give $w$'s last two (black) children to $y$, and make the first child of $w$ and $y$ be the two children of $w$.
- If $w$ is a 4-node, then create two new red nodes $y$ and $z$, give $w$'s first two (black) children to $y$, give $w$'s last two (black) children to $z$, and make $y$ and $z$ be the two children of $w$.

Notice that a red node always has a black parent in this construction.

**Proposition 11.9:** *The height of a red-black tree storing $n$ entries is $O(\log n)$.*

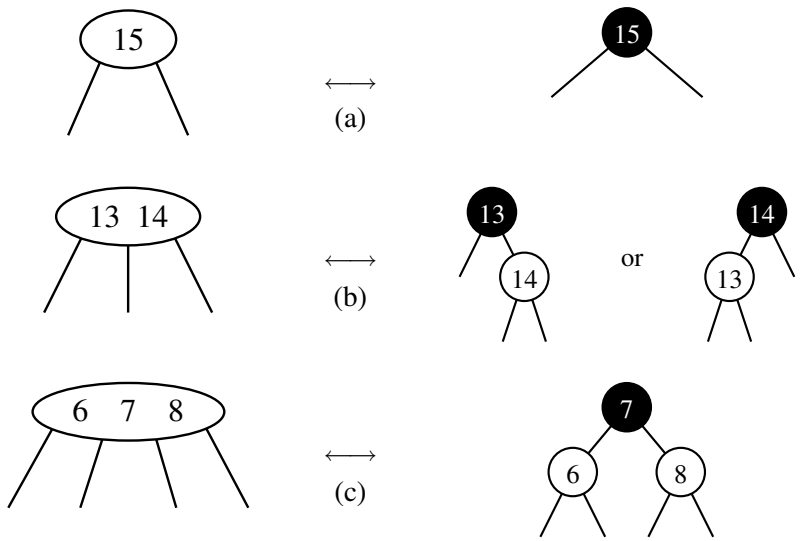



**Figure 11.32:** Correspondence between nodes of a $(2,4)$ tree and a red-black tree: (a) 2-node; (b) 3-node; (c) 4-node.

**Justification:** Let $T$ be a red-black tree storing $n$ entries, and let $h$ be the height of $T$. We justify this proposition by establishing the following fact:

$$\log(n+1) - 1 \leq h \leq 2\log(n+1) - 2.$$

Let $d$ be the common black depth of all nodes of $T$ having zero or one children. Let $T'$ be the $(2,4)$ tree associated with $T$, and let $h'$ be the height of $T'$ (excluding trivial leaves). Because of the correspondence between red-black trees and $(2,4)$ trees, we know that $h' = d$. Hence, by Proposition 11.8, $d = h' \leq \log(n+1) - 1$. By the red property, $h \leq 2d$. Thus, we obtain $h \leq 2\log(n+1) - 2$. The other inequality, $\log(n+1) - 1 \leq h$, follows from Proposition 8.8 and the fact that $T$ has $n$ nodes. ■

## 11.6.1 Red-Black Tree Operations

The algorithm for searching in a red-black tree $T$ is the same as that for a standard binary search tree (Section 11.1). Thus, searching in a red-black tree takes time proportional to the height of the tree, which is $O(\log n)$ by Proposition 11.9.

The correspondence between $(2,4)$ trees and red-black trees provides important intuition that we will use in our discussion of how to perform updates in red-black trees; in fact, the update algorithms for red-black trees can seem mysteriously complex without this intuition. Split and fuse operations of a $(2,4)$ tree will be effectively mimicked by recoloring neighboring red-black tree nodes. A rotation within a red-black tree will be used to change orientations of a 3-node between the two forms shown in Figure 11.32(b).

### Insertion

Now consider the insertion of a key-value pair $(k,v)$ into a red-black tree $T$. The algorithm initially proceeds as in a standard binary search tree (Section 11.1.3). Namely, we search for $k$ in $T$ until we reach a null subtree, and we introduce a new leaf $x$ at that position, storing the item. In the special case that $x$ is the only node of $T$, and thus the root, we color it black. In all other cases, we color $x$ red. This action corresponds to inserting $(k,v)$ into a node of the $(2,4)$ tree $T'$ with external children. The insertion preserves the root and depth properties of $T$, but it may violate the red property. Indeed, if $x$ is not the root of $T$ and the parent $y$ of $x$ is red, then we have a parent and a child (namely, $y$ and $x$) that are both red. Note that by the root property, $y$ cannot be the root of $T$, and by the red property (which was previously satisfied), the parent $z$ of $y$ must be black. Since $x$ and its parent are red, but $x$'s grandparent $z$ is black, we call this violation of the red property a ***double red*** at node $x$. To remedy a double red, we consider two cases.

**Case 1:** ***The Sibling s of y Is Black (or None).*** (See Figure 11.33.) In this case, the double red denotes the fact that we have added the new node to a corresponding 3-node of the $(2,4)$ tree $T'$, effectively creating a malformed 4-node. This formation has one red node ($y$) that is the parent of another red node ($x$), while we want it to have the two red nodes as siblings instead. To fix this problem, we perform a ***trinode restructuring*** of $T$. The trinode restructuring is done by the operation $\mathsf{restructure}(x)$, which consists of the following steps (see again Figure 11.33; this operation is also discussed in Section 11.2):

- Take node $x$, its parent $y$, and grandparent $z$, and temporarily relabel them as $a$, $b$, and $c$, in left-to-right order, so that $a$, $b$, and $c$ will be visited in this order by an inorder tree traversal.
- Replace the grandparent $z$ with the node labeled $b$, and make nodes $a$ and $c$ the children of $b$, keeping inorder relationships unchanged.

After performing the $\mathsf{restructure}(x)$ operation, we color $b$ black and we color $a$ and $c$ red. Thus, the restructuring eliminates the double-red problem. Notice that the portion of any path through the restructured part of the tree is incident to exactly one black node, both before and after the trinode restructuring. Therefore, the black depth of the tree is unaffected.

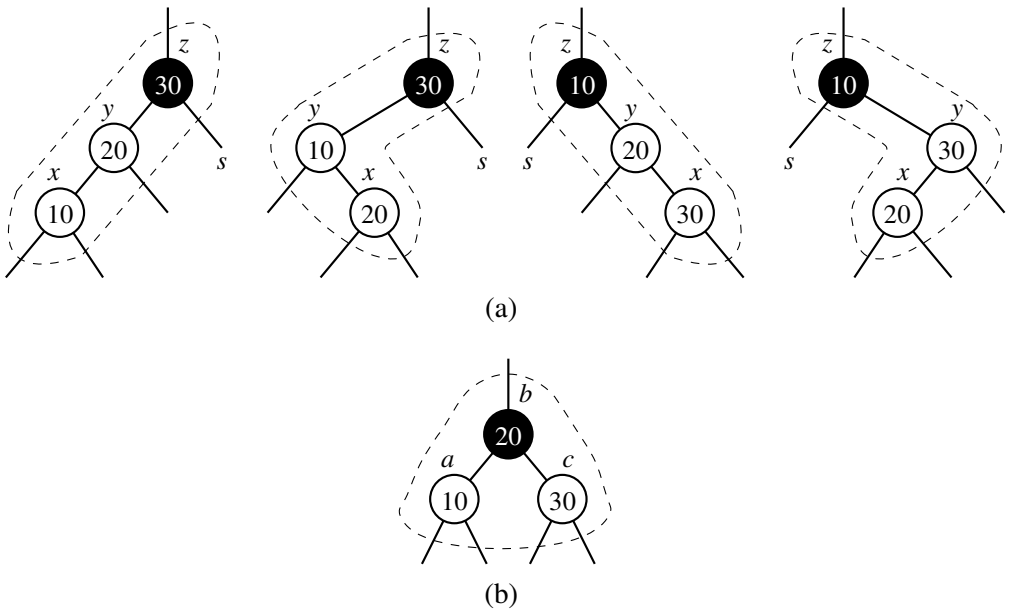



**Figure 11.33:** Restructuring a red-black tree to remedy a double red: (a) the four configurations for $x$, $y$, and $z$ before restructuring; (b) after restructuring.

**Case 2:** ***The Sibling s of y Is Red.*** (See Figure 11.34.) In this case, the double red denotes an overflow in the corresponding $(2,4)$ tree $T'$. To fix the problem, we perform the equivalent of a split operation. Namely, we do a ***recoloring***: we color $y$ and $s$ black and their parent $z$ red (unless $z$ is the root, in which case, it remains black). Notice that unless $z$ is the root, the portion of any path through the affected part of the tree is incident to exactly one black node, both before and after the recoloring. Therefore, the black depth of the tree is unaffected by the recoloring unless $z$ is the root, in which case it is increased by one.

However, it is possible that the double-red problem reappears after such a recoloring, albeit higher up in the tree $T$, since $z$ may have a red parent. If the double-red problem reappears at $z$, then we repeat the consideration of the two cases at $z$. Thus, a recoloring either eliminates the double-red problem at node $x$, or propagates it to the grandparent $z$ of $x$. We continue going up $T$ performing recolorings until we finally resolve the double-red problem (with either a final recoloring or a trinode restructuring). Thus, the number of recolorings caused by an insertion is no more than half the height of tree $T$, that is, $O(\log n)$ by Proposition 11.9.

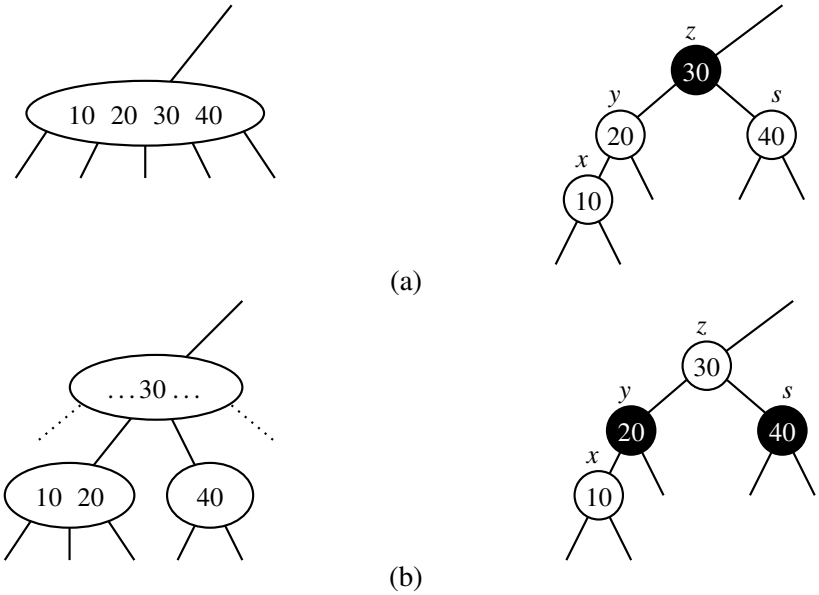



**Figure 11.34:** Recoloring to remedy the double-red problem: (a) before recoloring and the corresponding 5-node in the associated $(2,4)$ tree before the split; (b) after recoloring and the corresponding nodes in the associated $(2,4)$ tree after the split.

As further examples, Figures 11.35 and 11.36 show a sequence of insertion operations in a red-black tree.

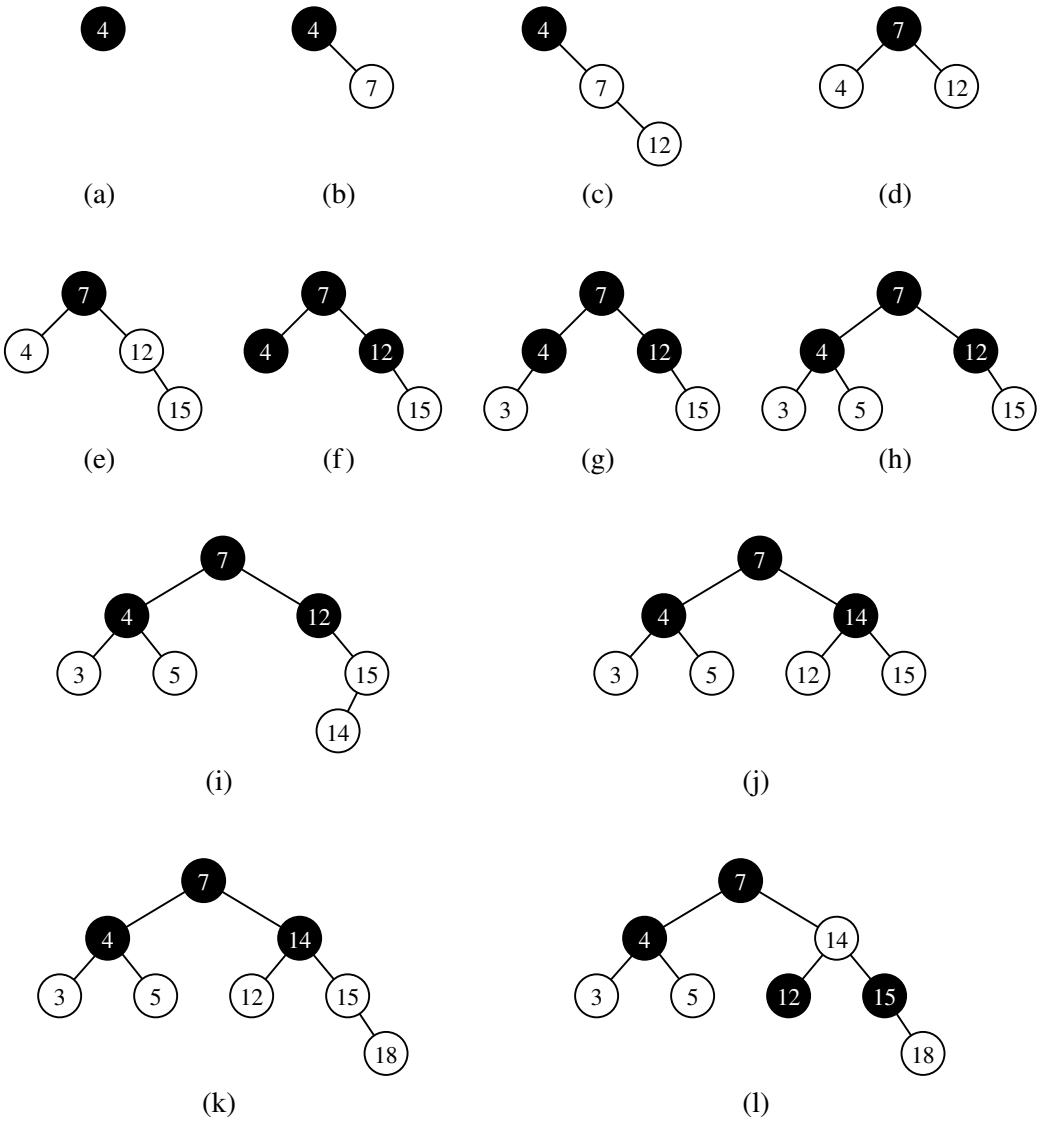



**Figure 11.35:** A sequence of insertions in a red-black tree: (a) initial tree; (b) insertion of 7; (c) insertion of 12, which causes a double red; (d) after restructuring; (e) insertion of 15, which causes a double red; (f) after recoloring (the root remains black); (g) insertion of 3; (h) insertion of 5; (i) insertion of 14, which causes a double red; (j) after restructuring; (k) insertion of 18, which causes a double red; (l) after recoloring. (Continues in Figure 11.36.)

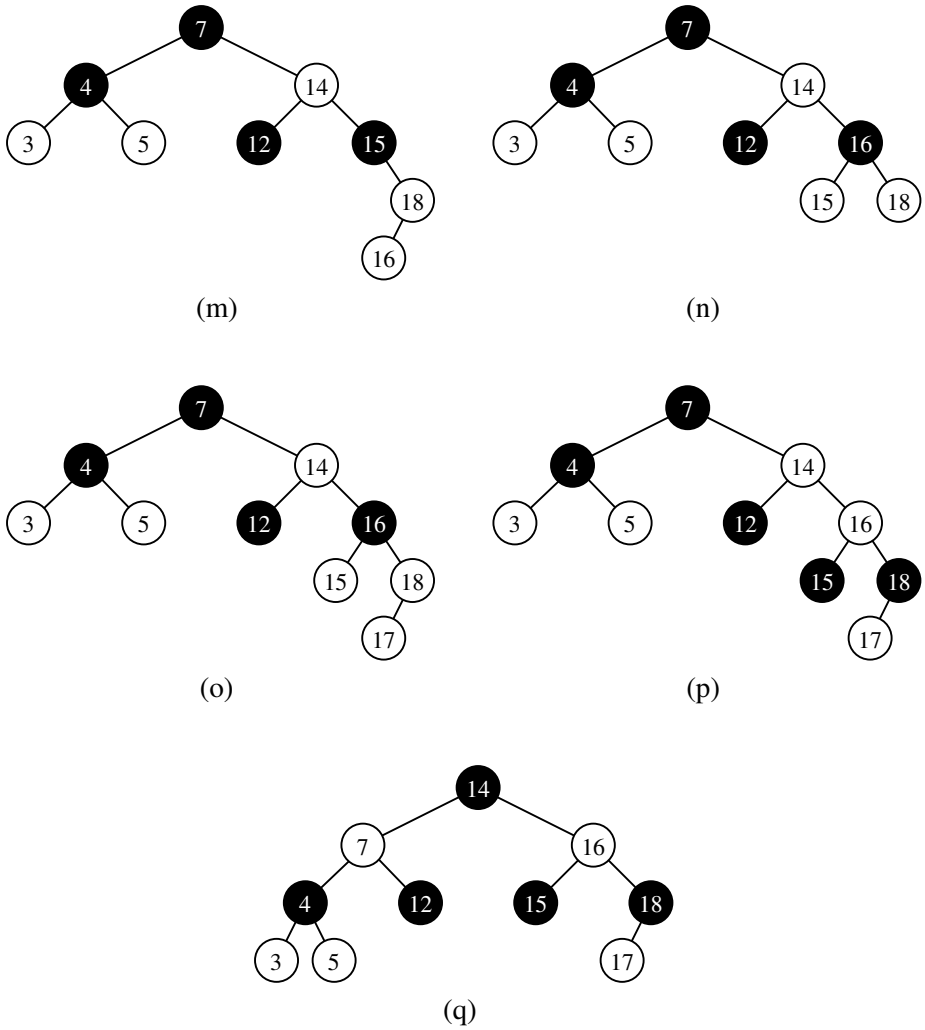



**Figure 11.36:** A sequence of insertions in a red-black tree: (m) insertion of 16, which causes a double red; (n) after restructuring; (o) insertion of 17, which causes a double red; (p) after recoloring there is again a double red, to be handled by a restructuring; (q) after restructuring. (Continued from Figure 11.35.)

## Deletion

Deleting an item with key $k$ from a red-black tree $T$ initially proceeds as for a binary search tree (Section 11.1.3). Structurally, the process results in the removal a node that has at most one child (either that originally containing key $k$ or its inorder predecessor) and the promotion of its remaining child (if any).

If the removed node was red, this structural change does not affect the black depths of any paths in the tree, nor introduce any red violations, and so the resulting tree remains a valid red-black tree. In the corresponding $(2,4)$ tree $T'$, this case denotes the shrinking of a 3-node or 4-node. If the removed node was black, then it either had zero children or it had one child that was a red leaf (because the null subtree of the removed node has black height 0). In the latter case, the removed node represents the black part of a corresponding 3-node, and we restore the red-black properties by recoloring the promoted child to black.

The more complex case is when a (nonroot) black leaf is removed. In the corresponding $(2,4)$ tree, this denotes the removal of an item from a 2-node. Without rebalancing, such a change results in a deficit of one for the black depth along the path leading to the deleted item. By necessity, the removed node must have a sibling whose subtree has black height 1 (given that this was a valid red-black tree prior to the deletion of the black leaf).

To remedy this scenario, we consider a more general setting with a node $z$ that is known to have two subtrees, $T_{\text{heavy}}$ and $T_{\text{light}}$, such that the root of $T_{\text{light}}$ (if any) is black and such that the black depth of $T_{\text{heavy}}$ is exactly one more than that of $T_{\text{light}}$, as portrayed in Figure 11.37. In the case of a removed black leaf, $z$ is the parent of that leaf and $T_{\text{light}}$ is trivially the empty subtree that remains after the deletion. We describe the more general case of a deficit because our algorithm for rebalancing the tree will, in some cases, push the deficit higher in the tree (just as the resolution of a deletion in a $(2,4)$ tree sometimes cascades upward). We let $y$ denote the root of $T_{\text{heavy}}$. (Such a node exists because $T_{\text{heavy}}$ has black height at least one.)

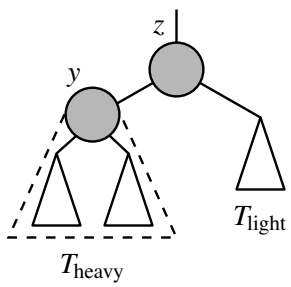



**Figure 11.37:** Portrayal of a deficit between the black heights of subtrees of node $z$. The gray color in illustrating $y$ and $z$ denotes the fact that these nodes may be colored either black or red.

We consider three possible cases to remedy a deficit.

**Case 1:** ***Node y Is Black and Has a Red Child x***. (See Figure 11.38.)

We perform a ***trinode restructuring***, as originally described in Section 11.2. The operation restructure$(x)$ takes the node $x$, its parent $y$, and grandparent $z$, labels them temporarily left to right as $a$, $b$, and $c$, and replaces $z$ with the node labeled $b$, making it the parent of the other two. We color $a$ and $c$ black, and give $b$ the former color of $z$.

Notice that the path to $T_{\text{light}}$ in the result includes one additional black node after the restructure, thereby resolving its deficit. In contrast, the number of black nodes on paths to any of the other three subtrees illustrated in Figure 11.38 remains unchanged.

Resolving this case corresponds to a transfer operation in the $(2,4)$ tree $T'$ between the two children of the node with $z$. The fact that $y$ has a red child assures us that it represents either a 3-node or a 4-node. In effect, the item previously stored at $z$ is demoted to become a new 2-node to resolve the deficiency, while an item stored at $y$ or its child is promoted to take the place of the item previously stored at $z$.

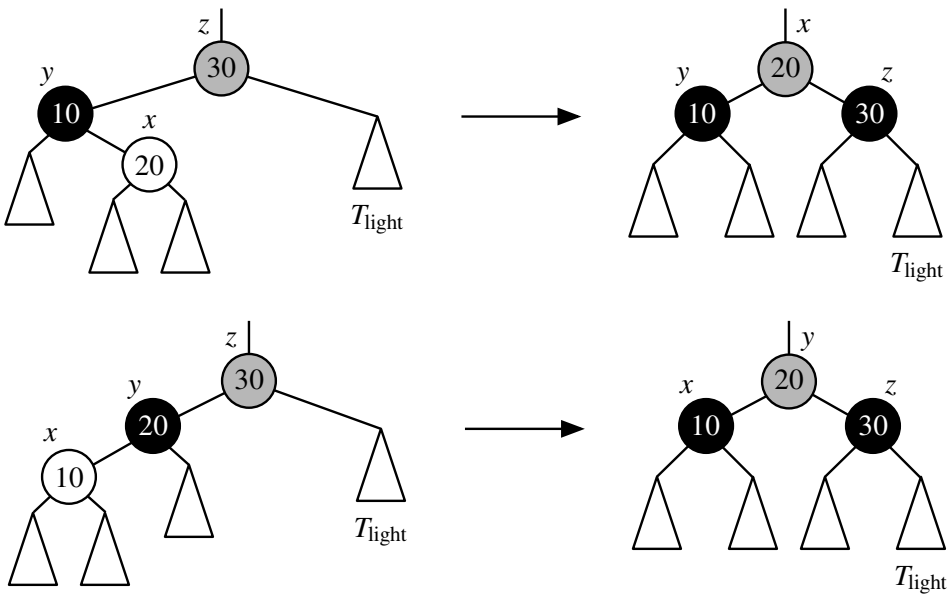



**Figure 11.38:** Resolving a black deficit in $T_{\text{light}}$ by performing a trinode restructuring as restructure$(x)$. Two possible configurations are shown (two other configurations are symmetric). The gray color of $z$ in the left figures denotes the fact that this node may be colored either red or black. The root of the restructured portion is given that same color, while the children of that node are both colored black in the result.

**Case 2:** ***Node y Is Black and Both Children of y Are Black (or None).***

Resolving this case corresponds to a fusion operation in the corresponding $(2,4)$ tree $T'$, as $y$ must represent a 2-node. We do a ***recoloring***; we color $y$ red, and, if $z$ is red, we color it black. (See Figure 11.39). This does not introduce any red violation, because $y$ does not have a red child.

In the case that $z$ was originally red, and thus the parent in the corresponding $(2,4)$ tree is a 3-node or 4-node, this recoloring resolves the deficit. (See Figure 11.39a.) The path leading to $T_{\text{light}}$ includes one additional black node in the result, while the recoloring did not affect the number of black nodes on the path to the subtrees of $T_{\text{heavy}}$.

In the case that $z$ was originally black, and thus the parent in the corresponding $(2,4)$ tree is a 2-node, the recoloring has not increased the number of black nodes on the path to $T_{\text{light}}$; in fact, it has *reduced* the number of black nodes on the path to $T_{\text{heavy}}$. (See Figure 11.39b.) After this step, the two children of $z$ will have the same black height. However, the entire tree rooted at $z$ has become deficient, thereby propogating the problem higher in the tree; we must repeat consideration of all three cases at the parent of $z$ as a remedy.

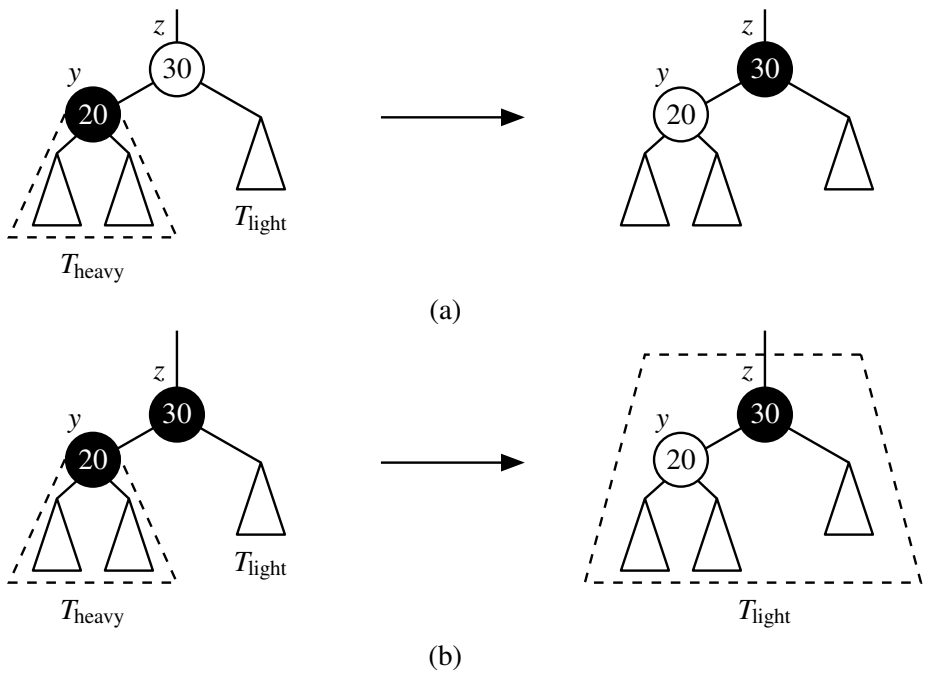



**Figure 11.39:** Resolving a black deficit in $T_{\text{light}}$ by a recoloring operation: (a) when $z$ is originally red, reversing the colors of $y$ and $z$ resolves the black deficit in $T_{\text{light}}$, ending the process; (b) when $z$ is originally black, recoloring $y$ causes the entire subtree of $z$ to have a black deficit, requiring a cascading remedy.

**Case 3:** ***Node y Is Red***. (See Figure 11.40.)

Because $y$ is red and $T_{\text{heavy}}$ has black depth at least 1, $z$ must be black and the two subtrees of $y$ must each have a black root and a black depth equal to that of $T_{\text{heavy}}$. In this case, we perform a rotation about $y$ and $z$, and then recolor $y$ black and $z$ red. This denotes a reorientation of a 3-node in the corresponding $(2,4)$ tree $T'$.

This does not immediately resolve the deficit, as the new subtree of $z$ is an old subtree of $y$ with black root $y'$ and black height equal to that of the original $T_{\text{heavy}}$. We reapply the algorithm to resolve the deficit at $z$, knowing that the new child $y'$, that is the root of $T_{\text{heavy}}$ is now black, and therefore that either Case 1 applies or Case 2 applies. Furthermore, the next application will be the last, because Case 1 is always terminal and Case 2 will be terminal given that $z$ is red.

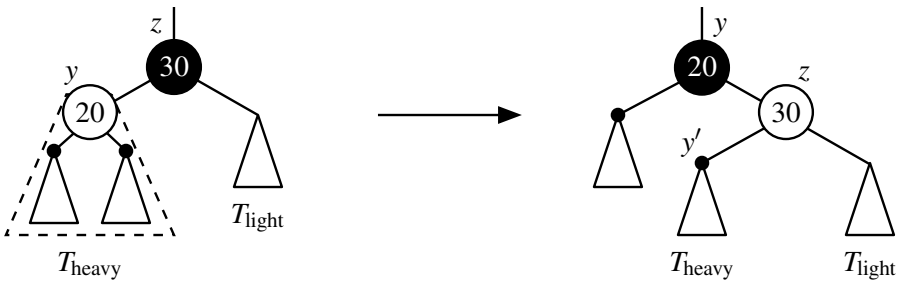



**Figure 11.40:** A rotation and recoloring about red node $y$ and black node $z$, assuming a black deficit at $z$. This amounts to a change of orientation in the corresponding 3-node of a $(2,4)$ tree. This operation does not affect the black depth of any paths through this portion of the tree. Furthermore, because $y$ was originally red, the new subtree of $z$ must have a black root $y'$ and must have black height equal to the original $T_{\text{heavy}}$. Therefore, a black deficit remains at node $z$ after the transformation.

In Figure 11.41, we show a sequence of deletions on a red-black tree. A dashed edge in those figures, such as to the right of 7 in part (c), represents a branch with a black deficiency that has not yet been resolved. We illustrate a Case 1 restructuring in parts (c) and (d). We illustrate a Case 2 recoloring in parts (f) and (g). Finally, we show an example of a Case 3 rotation between parts (i) and (j), concluding with a Case 2 recoloring in part (k).

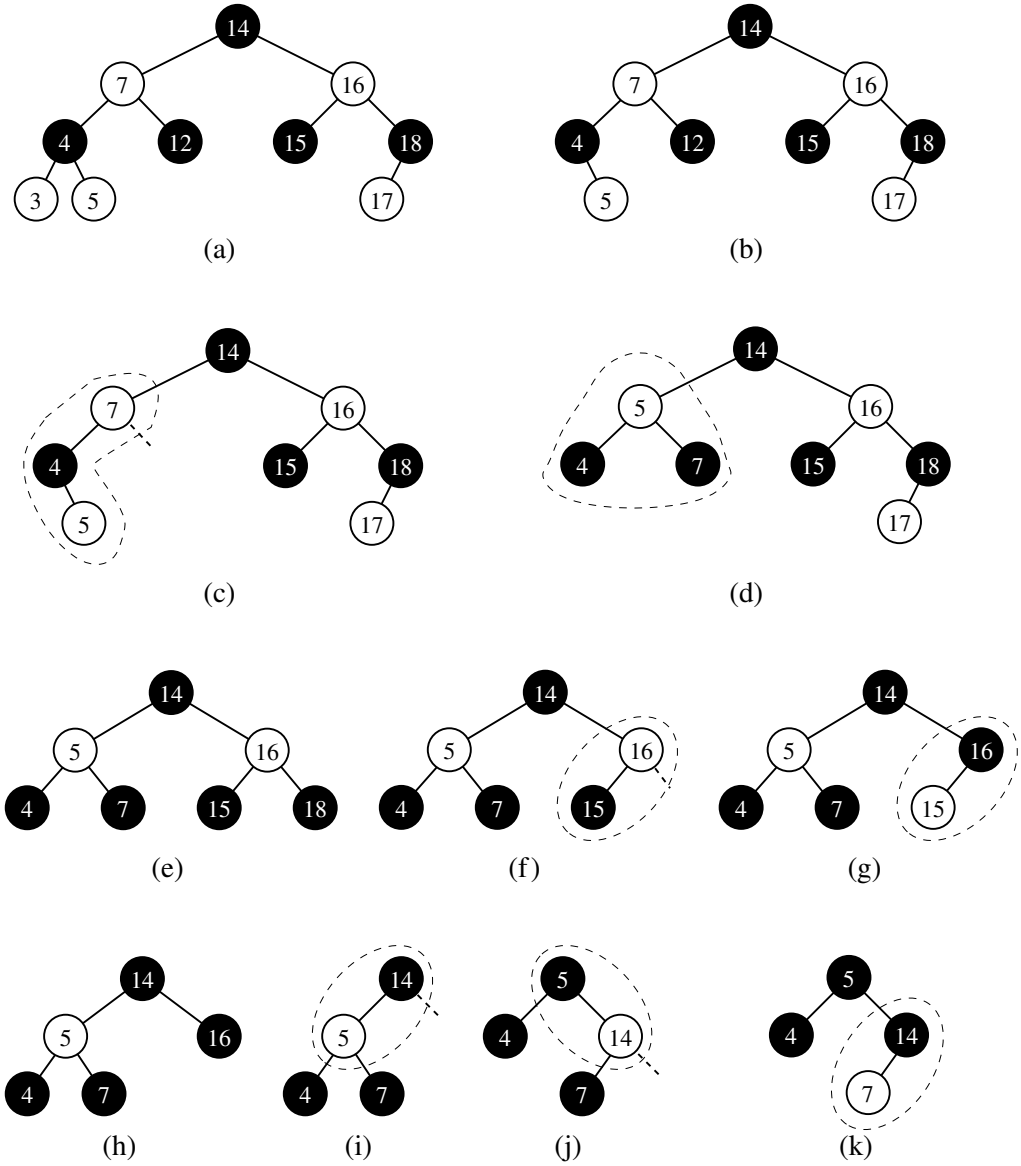



**Figure 11.41:** A sequence of deletions from a red-black tree: (a) initial tree; (b) removal of 3; (c) removal of 12, causing a black deficit to the right of 7 (handled by restructuring); (d) after restructuring; (e) removal of 17; (f) removal of 18, causing a black deficit to the right of 16 (handled by recoloring); (g) after recoloring; (h) removal of 15; (i) removal of 16, causing a black deficit to the right of 14 (handled initially by a rotation); (j) after the rotation the black deficit needs to be handled by a recoloring; (k) after the recoloring.

## Performance of Red-Black Trees

The asymptotic performance of a red-black tree is identical to that of an AVL tree or a $(2,4)$ tree in terms of the sorted map ADT, with guaranteed logarithmic time bounds for most operations. (See Table 11.2 for a summary of the AVL performance.) The primary advantage of a red-black tree is that an insertion or deletion requires only a ***constant number of restructuring operations***. (This is in contrast to AVL trees and $(2,4)$ trees, both of which require a logarithmic number of structural changes per map operation in the worst case.) That is, an insertion or deletion in a red-black tree requires logarithmic time for a search, and may require a logarithmic number of recoloring operations that cascade upward. Yet we show, in the following propositions, that there are a constant number of rotations or restructure operations for a single map operation.

**Proposition 11.10:** *The insertion of an item in a red-black tree storing $n$ items can be done in $O(\log n)$ time and requires $O(\log n)$ recolorings and at most one trinode restructuring.*

**Justification:** Recall that an insertion begins with a downward search, the creation of a new leaf node, and then a potential upward effort to remedy a double-red violation. There may be logarithmically many recoloring operations due to an upward cascading of Case 2 applications, but a single application of the Case 1 action eliminates the double-red problem with a trinode restructuring. Therefore, at most one restructuring operation is needed for a red-black tree insertion. ■

**Proposition 11.11:** *The algorithm for deleting an item from a red-black tree with $n$ items takes $O(\log n)$ time and performs $O(\log n)$ recolorings and at most two restructuring operations.*

**Justification:** A deletion begins with the standard binary search tree deletion algorithm, which requires time proportional to the height of the tree; for red-black trees, that height is $O(\log n)$. The subsequent rebalancing takes place along an upward path from the parent of a deleted node.

We considered three cases to remedy a resulting black deficit. Case 1 requires a trinode restructuring operation, yet completes the process, so this case is applied at most once. Case 2 may be applied logarithmically many times, but it only involves a recoloring of up to two nodes per application. Case 3 requires a rotation, but this case can only apply once, because if the rotation does not resolve the problem, the very next action will be a terminal application of either Case 1 or Case 2.

In the worst case, there will be $O(\log n)$ recolorings from Case 2, a single rotation from Case 3, and a trinode restructuring from Case 1. ■

### 11.6.2 Python Implementation

A complete implementation of a RedBlackTreeMap class is provided in Code Fragments 11.15 through 11.17. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1.

We begin, in Code Fragment 11.15, by overriding the definition of the nested _Node class to introduce an additional Boolean field to denote the current color of a node. Our constructor intentionally sets the color of a new node to red to be consistent with our approach for inserting items. We define several additional utility functions, at the top of Code Fragment 11.16, that aid in setting the color of nodes and querying various conditions.

When an element has been inserted as a leaf in the tree, the _rebalance_insert hook is called, allowing us the opportunity to modify the tree. The new node is red by default, so we need only look for the special case of the new node being the root (in which case it should be colored black), or the possibility that we have a double-red violation because the new node's parent is also red. To remedy such violations, we closely follow the case analysis described in Section 11.6.1.

The rebalancing after a deletion also follows the case analysis described in Section 11.6.1. An additional challenge is that by the time the _rebalance_hook is called, the old node has already been removed from the tree. That hook is invoked on the *parent* of the removed node. Some of the case analysis depends on knowing about the properties of the removed node. Fortunately, we can reverse engineer that information by relying on the red-black tree properties. In particular, if $p$ denotes the parent of the removed node, it must be that:

- If $p$ has no children, the removed node was a red leaf. (Exercise R-11.26.)
- If $p$ has one child, the removed node was a black leaf, causing a deficit, unless that one remaining child is a red leaf. (Exercise R-11.27.)
- If $p$ has two children, the removed node was a black node with one red child, which was promoted. (Exercise R-11.28.)

```
1  class RedBlackTreeMap(TreeMap):
2    """Sorted map implementation using a red-black tree."""
3    class _Node(TreeMap._Node):
4      """Node class for red-black tree maintains bit that denotes color."""
5      __slots__ = '_red'      # add additional data member to the Node class
6
7      def __init__(self, element, parent=None, left=None, right=None):
8        super().__init__(element, parent, left, right)
9        self._red = True      # new node red by default
```

**Code Fragment 11.15:** Beginning of the RedBlackTreeMap class. (Continued in Code Fragment 11.16.)

```
10    #------------------------ positional-based utility methods ------------------------
11    # we consider a nonexistent child to be trivially black
12    def _set_red(self, p): p._node._red = True
13    def _set_black(self, p): p._node._red = False
14    def _set_color(self, p, make_red): p._node._red = make_red
15    def _is_red(self, p): return p is not None and p._node._red
16    def _is_red_leaf(self, p): return self._is_red(p) and self.is_leaf(p)
17
18    def _get_red_child(self, p):
19      """Return a red child of p (or None if no such child)."""
20      for child in (self.left(p), self.right(p)):
21        if self._is_red(child):
22          return child
23      return None
24
25    #------------------------ support for insertions ------------------------
26    def _rebalance_insert(self, p):
27      self._resolve_red(p)                          # new node is always red
28
29    def _resolve_red(self, p):
30      if self.is_root(p):
31        self._set_black(p)                          # make root black
32      else:
33        parent = self.parent(p)
34        if self._is_red(parent):                    # double red problem
35          uncle = self.sibling(parent)
36          if not self._is_red(uncle):               # Case 1: misshapen 4-node
37            middle = self._restructure(p)           # do trinode restructuring
38            self._set_black(middle)                 # and then fix colors
39            self._set_red(self.left(middle))
40            self._set_red(self.right(middle))
41          else:                                     # Case 2: overfull 5-node
42            grand = self.parent(parent)
43            self._set_red(grand)                    # grandparent becomes red
44            self._set_black(self.left(grand))       # its children become black
45            self._set_black(self.right(grand))
46            self._resolve_red(grand)                # recur at red grandparent
```

**Code Fragment 11.16:** Continuation of the RedBlackTreeMap class. (Continued from Code Fragment 11.15, and concluded in Code Fragment 11.17.)

```
47    #------------------------ support for deletions ------------------------
48    def _rebalance_delete(self, p):
49      if len(self) == 1:
50        self._set_black(self.root( ))      # special case: ensure that root is black
51      elif p is not None:
52        n = self.num_children(p)
53        if n == 1:                         # deficit exists unless child is a red leaf
54          c = next(self.children(p))
55          if not self._is_red_leaf(c):
56            self._fix_deficit(p, c)
57        elif n == 2:                       # removed black node with red child
58          if self._is_red_leaf(self.left(p)):
59            self._set_black(self.left(p))
60          else:
61            self._set_black(self.right(p))
62
63    def _fix_deficit(self, z, y):
64      """Resolve black deficit at z, where y is the root of z's heavier subtree."""
65      if not self._is_red(y): # y is black; will apply Case 1 or 2
66        x = self._get_red_child(y)
67        if x is not None: # Case 1: y is black and has red child x; do "transfer"
68          old_color = self._is_red(z)
69          middle = self._restructure(x)
70          self._set_color(middle, old_color)       # middle gets old color of z
71          self._set_black(self.left(middle))       # children become black
72          self._set_black(self.right(middle))
73        else: # Case 2: y is black, but no red children; recolor as "fusion"
74          self._set_red(y)
75          if self._is_red(z):
76            self._set_black(z)                     # this resolves the problem
77          elif not self.is_root(z):
78            self._fix_deficit(self.parent(z), self.sibling(z)) # recur upward
79      else: # Case 3: y is red; rotate misaligned 3-node and repeat
80        self._rotate(y)
81        self._set_black(y)
82        self._set_red(z)
83        if z == self.right(y):
84          self._fix_deficit(z, self.left(z))
85        else:
86          self._fix_deficit(z, self.right(z))
```

**Code Fragment 11.17:** Conclusion of the RedBlackTreeMap class. (Continued from Code Fragment 11.16.)

## 11.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

### Reinforcement

**R-11.1** If we insert the entries $(1,A)$, $(2,B)$, $(3,C)$, $(4,D)$, and $(5,E)$, in this order, into an initially empty binary search tree, what will it look like?

**R-11.2** Insert, into an empty binary search tree, entries with keys 30, 40, 24, 58, 48, 26, 11, 13 (in this order). Draw the tree after each insertion.

**R-11.3** How many different binary search trees can store the keys $\{1,2,3\}$?

**R-11.4** Dr. Amongus claims that the order in which a fixed set of entries is inserted into a binary search tree does not matter—the same tree results every time. Give a small example that proves he is wrong.

**R-11.5** Dr. Amongus claims that the order in which a fixed set of entries is inserted into an AVL tree does not matter—the same AVL tree results every time. Give a small example that proves he is wrong.

**R-11.6** Our implementation of the TreeMap._subtree_search utility, from Code Fragment 11.4, relies on recursion. For a large unbalanced tree, Python's default limit on recursive depth may be prohibitive. Give an alternative implementation of that method that does not rely on the use of recursion.

**R-11.7** Do the trinode restructurings in Figures 11.12 and 11.14 result in single or double rotations?

**R-11.8** Draw the AVL tree resulting from the insertion of an entry with key 52 into the AVL tree of Figure 11.14b.

**R-11.9** Draw the AVL tree resulting from the removal of the entry with key 62 from the AVL tree of Figure 11.14b.

**R-11.10** Explain why performing a rotation in an $n$-node binary tree when using the array-based representation of Section 8.3.2 takes $\Omega(n)$ time.

**R-11.11** Give a schematic figure, in the style of Figure 11.13, showing the heights of subtrees during a deletion operation in an AVL tree that triggers a trinode restructuring for the case in which the two children of the node denoted as $y$ start with equal heights. What is the net effect of the height of the rebalanced subtree due to the deletion operation?

**R-11.12** Repeat the previous problem, considering the case in which $y$'s children start with different heights.

**R-11.13** The rules for a deletion in an AVL tree specifically require that when the two subtrees of the node denoted as $y$ have equal height, child $x$ should be chosen to be "aligned" with $y$ (so that $x$ and $y$ are both left children or both right children). To better understand this requirement, repeat Exercise R-11.11 assuming we picked the misaligned choice of $x$. Why might there be a problem in restoring the AVL property with that choice?

**R-11.14** Perform the following sequence of operations in an initially empty splay tree and draw the tree after each set of operations.

a. Insert keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.
b. Search for keys 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, in this order.
c. Delete keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.

**R-11.15** What does a splay tree look like if its entries are accessed in increasing order by their keys?

**R-11.16** Is the search tree of Figure 11.23(a) a $(2,4)$ tree? Why or why not?

**R-11.17** An alternative way of performing a split at a node $w$ in a $(2,4)$ tree is to partition $w$ into $w'$ and $w''$, with $w'$ being a 2-node and $w''$ a 3-node. Which of the keys $k_1$, $k_2$, $k_3$, or $k_4$ do we store at $w$'s parent? Why?

**R-11.18** Dr. Amongus claims that a $(2,4)$ tree storing a set of entries will always have the same structure, regardless of the order in which the entries are inserted. Show that he is wrong.

**R-11.19** Draw four different red-black trees that correspond to the same $(2,4)$ tree.

**R-11.20** Consider the set of keys $K = \{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15\}$.

a. Draw a $(2,4)$ tree storing $K$ as its keys using the fewest number of nodes.
b. Draw a $(2,4)$ tree storing $K$ as its keys using the maximum number of nodes.

**R-11.21** Consider the sequence of keys $(5,16,22,45,2,10,18,30,50,12,1)$. Draw the result of inserting entries with these keys (in the given order) into

a. An initially empty $(2,4)$ tree.
b. An initially empty red-black tree.

**R-11.22** For the following statements about red-black trees, provide a justification for each true statement and a counterexample for each false one.

a. A subtree of a red-black tree is itself a red-black tree.
b. A node that does not have a sibling is red.
c. There is a unique $(2,4)$ tree associated with a given red-black tree.
d. There is a unique red-black tree associated with a given $(2,4)$ tree.

**R-11.23** Explain why you would get the same output in an inorder listing of the entries in a binary search tree, $T$, independent of whether $T$ is maintained to be an AVL tree, splay tree, or red-black tree.

**R-11.24** Consider a tree $T$ storing 100,000 entries. What is the worst-case height of $T$ in the following cases?

a. $T$ is a binary search tree.
b. $T$ is an AVL tree.
c. $T$ is a splay tree.
d. $T$ is a $(2,4)$ tree.
e. $T$ is a red-black tree.

**R-11.25** Draw an example of a red-black tree that is not an AVL tree.

**R-11.26** Let $T$ be a red-black tree and let $p$ be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if $p$ has zero children, the removed node was a red leaf.

**R-11.27** Let $T$ be a red-black tree and let $p$ be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if $p$ has one child, the deletion has caused a black deficit at $p$, except for the case when the one remaining child is a red leaf.

**R-11.28** Let $T$ be a red-black tree and let $p$ be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if $p$ has two children, the removed node was black and had one red child.

## Creativity

**C-11.29** Explain how to use an AVL tree or a red-black tree to sort $n$ comparable elements in $O(n \log n)$ time in the worst case.

**C-11.30** Can we use a splay tree to sort $n$ comparable elements in $O(n \log n)$ time in the ***worst case***? Why or why not?

**C-11.31** Repeat Exercise C-10.28 for the TreeMap class.

**C-11.32** Show that any $n$-node binary tree can be converted to any other $n$-node binary tree using $O(n)$ rotations.

**C-11.33** For a key $k$ that is not found in binary search tree $T$, prove that both the greatest key less than $k$ and the least key greater than $k$ lie on the path traced by the search for $k$.

**C-11.34** In Section 11.1.2 we claim that the find_range method of a binary search tree executes in $O(s+h)$ time where $s$ is the number of items found within the range and $h$ is the height of the tree. Our implementation, in Code Fragment 11.6 begins by searching for the starting key, and then repeatedly calling the after method until reaching the end of the range. Each call to after is guaranteed to run in $O(h)$ time. This suggests a weaker $O(sh)$ bound for find_range, since it involves $O(s)$ calls to after. Prove that this implementation achieves the stronger $O(s+h)$ bound.

**C-11.35** Describe how to perform an operation remove_range(start, stop) that removes all the items whose keys fall within range(start, stop) in a sorted map that is implemented with a binary search tree $T$, and show that this method runs in time $O(s+h)$, where $s$ is the number of items removed and $h$ is the height of $T$.

**C-11.36** Repeat the previous problem using an AVL tree, achieving a running time of $O(s \log n)$. Why doesn't the solution to the previous problem trivially result in an $O(s+\log n)$ algorithm for AVL trees?

**C-11.37** Suppose we wish to support a new method count_range(start, stop) that determines how many keys of a sorted map fall in the specified range. We could clearly implement this in $O(s+h)$ time by adapting our approach to find_range. Describe how to modify the search tree structure to support $O(h)$ worst-case time for count_range.

**C-11.38** If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain support for the new method?

**C-11.39** Draw a schematic of an AVL tree such that a single remove operation could require $\Omega(\log n)$ trinode restructurings (or rotations) from a leaf to the root in order to restore the height-balance property.

**C-11.40** In our AVL implementation, each node stores the height of its subtree, which is an arbitrarily large integer. The space usage for an AVL tree can be reduced by instead storing the ***balance factor*** of a node, which is defined as the height of its left subtree minus the height of its right subtree. Thus, the balance factor of a node is always equal to $-1$, 0, or 1, except during an insertion or removal, when it may become ***temporarily*** equal to $-2$ or $+2$. Reimplement the AVLTreeMap class storing balance factors rather than subtree heights.

**C-11.41** If we maintain a reference to the position of the leftmost node of a binary search tree, then operation find_min can be performed in $O(1)$ time. Describe how the implementation of the other map methods need to be modified to maintain a reference to the leftmost position.

**C-11.42** If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to accurately maintain the reference to the leftmost position?

**C-11.43** Describe a modification to the binary search tree implementation having worst-case $O(1)$-time performance for methods after(p) and before(p) without adversely affecting the asymptotics of any other methods.

**C-11.44** If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain the efficiency?

**C-11.45** For a standard binary search tree, Table 11.1 claims $O(h)$-time performance for the delete(p) method. Explain why delete(p) would run in $O(1)$ time if given a solution to Exercise C-11.43.

**C-11.46** Describe a modification to the binary search tree data structure that would support the following two index-based operations for a sorted map in $O(h)$ time, where $h$ is the height of the tree.

**at_index(i):** Return the position $p$ of the item at index $i$ of a sorted map.

**index_of(p):** Return the index $i$ of the item at position $p$ of a sorted map.

**C-11.47** Draw a splay tree, $T_1$, together with the sequence of updates that produced it, and a red-black tree, $T_2$, on the same set of ten entries, such that a preorder traversal of $T_1$ would be the same as a preorder traversal of $T_2$.

**C-11.48** Show that the nodes that become temporarily unbalanced in an AVL tree during an insertion may be nonconsecutive on the path from the newly inserted node to the root.

**C-11.49** Show that at most one node in an AVL tree becomes temporarily unbalanced after the immediate deletion of a node as part of the standard __delitem__ map operation.

**C-11.50** Let $T$ and $U$ be $(2,4)$ trees storing $n$ and $m$ entries, respectively, such that all the entries in $T$ have keys less than the keys of all the entries in $U$. Describe an $O(\log n + \log m)$-time method for ***joining*** $T$ and $U$ into a single tree that stores all the entries in $T$ and $U$.

**C-11.51** Repeat the previous problem for red-black trees $T$ and $U$.

**C-11.52** Justify Proposition 11.7.

**C-11.53** The Boolean indicator used to mark nodes in a red-black tree as being "red" or "black" is not strictly needed when we have distinct keys. Describe a scheme for implementing a red-black tree without adding any extra space to standard binary search tree nodes.

**C-11.54** Let $T$ be a red-black tree storing $n$ entries, and let $k$ be the key of an entry in $T$. Show how to construct from $T$, in $O(\log n)$ time, two red-black trees $T'$ and $T''$, such that $T'$ contains all the keys of $T$ less than $k$, and $T''$ contains all the keys of $T$ greater than $k$. This operation destroys $T$.

**C-11.55** Show that the nodes of any AVL tree $T$ can be colored "red" and "black" so that $T$ becomes a red-black tree.

**C-11.56** The standard splaying step requires two passes, one downward pass to find the node $x$ to splay, followed by an upward pass to splay the node $x$. Describe a method for splaying and searching for $x$ in one downward pass. Each substep now requires that you consider the next two nodes in the path down to $x$, with a possible zig substep performed at the end. Describe how to perform the zig-zig, zig-zag, and zig steps.

**C-11.57** Consider a variation of splay trees, called ***half-splay trees***, where splaying a node at depth $d$ stops as soon as the node reaches depth $\lfloor d/2 \rfloor$. Perform an amortized analysis of half-splay trees.

**C-11.58** Describe a sequence of accesses to an $n$-node splay tree $T$, where $n$ is odd, that results in $T$ consisting of a single chain of nodes such that the path down $T$ alternates between left children and right children.

**C-11.59** As a positional structure, our TreeMap implementation has a subtle flaw. A position instance $p$ associated with an key-value pair $(k,v)$ should remain valid as long as that item remains in the map. In particular, that position should be unaffected by calls to insert or delete other items in the collection. Our algorithm for deleting an item from a binary search tree may fail to provide such a guarantee, in particular because of our rule for using the inorder predecessor of a key as a replacement when deleting a key that is located in a node with two children. Given an explicit series of Python commands that demonstrates such a flaw.

**C-11.60** How might the TreeMap implementation be changed to avoid the flaw described in the previous problem?

## Projects

**P-11.61** Perform an experimental study to compare the speed of our AVL tree, splay tree, and red-black tree implementations for various sequences of operations.

**P-11.62** Redo the previous exercise, including an implementation of skip lists. (See Exercise P-10.53.)

**P-11.63** Implement the Map ADT using a $(2,4)$ tree. (See Section 10.1.1.)

**P-11.64** Redo the previous exercise, including all methods of the Sorted Map ADT. (See Section 10.3.)

**P-11.65** Redo Exercise P-11.63 providing *positional* support, as we did for binary search trees (Section 11.1.1), so as to include methods first( ), last( ), before(p), after(p), and find_position(k). Each item should have a distinct position in this abstraction, even though several items may be stored at a single node of a tree.

**P-11.66** Write a Python class that can take any red-black tree and convert it into its corresponding $(2,4)$ tree and can take any $(2,4)$ tree and convert it into its corresponding red-black tree.

**P-11.67** In describing multisets and multimaps in Section 10.5.3, we describe a general approach for adapting a traditional map by storing all duplicates within a secondary container as a value in the map. Give an alternative implementation of a multimap using a binary search tree such that each entry of the map is stored at a distinct node of the tree. With the existence of duplicates, we redefine the search tree property so that all items in the left subtree of a position $p$ with key $k$ have keys that are less than *or equal to* $k$, while all items in the right subtree of $p$ have keys that are greater than *or equal to* $k$. Use the public interface given in Code Fragment 10.17.

**P-11.68** Prepare an implementation of splay trees that uses top-down splaying as described in Exercise C-11.56. Perform extensive experimental studies to compare its performance to the standard bottom-up splaying implemented in this chapter.

**P-11.69** The ***mergeable heap*** ADT is an extension of the priority queue ADT consisting of operations add(k, v), min( ), remove_min( ) and merge(h), where the merge(h) operations performs a union of the mergeable heap $h$ with the present one, incorporating all items into the current one while emptying $h$. Describe a concrete implementation of the mergeable heap ADT that achieves $O(\log n)$ performance for all its operations, where $n$ denotes the size of the resulting heap for the merge operation.

**P-11.70** Write a program that performs a simple $n$-body simulation, called "Jumping Leprechauns." This simulation involves $n$ leprechauns, numbered 1 to $n$. It maintains a gold value $g_i$ for each leprechaun $i$, which begins with each leprechaun starting out with a million dollars worth of gold, that is, $g_i = 1\,000\,000$ for each $i = 1,2,\ldots,n$. In addition, the simulation also maintains, for each leprechaun, $i$, a place on the horizon, which is represented as a double-precision floating-point number, $x_i$. In each iteration of the simulation, the simulation processes the leprechauns in order. Processing a leprechaun $i$ during this iteration begins by computing a new place on the horizon for $i$, which is determined by the assignment

$$x_i = x_i + rg_i,$$

where $r$ is a random floating-point number between $-1$ and 1. The leprechaun $i$ then steals half the gold from the nearest leprechauns on either side of him and adds this gold to his gold value, $g_i$. Write a program that can perform a series of iterations in this simulation for a given number, $n$, of leprechauns. You must maintain the set of horizon positions using a sorted map data structure described in this chapter.

# Chapter Notes

Some of the data structures discussed in this chapter are extensively covered by Knuth in his *Sorting and Searching* book [65], and by Mehlhorn in [76]. AVL trees are due to Adel'son-Vel'skii and Landis [2], who invented this class of balanced search trees in 1962. Binary search trees, AVL trees, and hashing are described in Knuth's *Sorting and Searching* [65] book. Average-height analyses for binary search trees can be found in the books by Aho, Hopcroft, and Ullman [6] and Cormen, Leiserson, Rivest and Stein [29]. The handbook by Gonnet and Baeza-Yates [44] contains a number of theoretical and experimental comparisons among map implementations. Aho, Hopcroft, and Ullman [5] discuss $(2,3)$ trees, which are similar to $(2,4)$ trees. Red-black trees were defined by Bayer [10]. Variations and interesting properties of red-black trees are presented in a paper by Guibas and Sedgewick [48]. The reader interested in learning more about different balanced tree data structures is referred to the books by Mehlhorn [76] and Tarjan [95], and the book chapter by Mehlhorn and Tsakalidis [78]. Knuth [65] is excellent additional reading that includes early approaches to balancing trees. Splay trees were invented by Sleator and Tarjan [89] (see also [95]).